

关于本文档

文档名称：《Kubernetes服务发现和名称解析》

使用协议：《知识共享公共许可协议(CCPL)》

贡献者

贡献者名称	贡献度	文档变更记录	个人主页
马哥（马永亮）	主编		http://github.com/iKubernetes/
王晓春	作者	kubernetes-v1.32.3	http://www.wangxiaochun.com

文档协议

署名要求：使用本系列文档，您必须保留本页中的文档来源信息，具体请参考《知识共享 (Creative Commons) 署名4.0公共许可协议国际版》。

非商业化使用：遵循《知识共享公共许可协议(CCPL)》，并且您不能将本文档用于马哥教育相关业务之外的其他任何商业用途。

您的权利：遵循本协议后，在马哥教育相关业务之外的领域，您将有以下使用权限：

共享 — 允许以非商业性质复制本作品。

改编 — 在原基础上修改、转换或以本作品为基础进行重新编辑并用于个人非商业使用。

致谢

本文档中，部分素材参考了相关项目的文档，以及通过搜索引擎获得的内容，这里先一并向相关的贡献者表示感谢。

Kubernetes 服务发现和域名解析

本章内容

- 服务访问
- 服务发现
- 域名解析
- 无头服务

1 服务访问

1.1 Service 介绍

1.1.1 Service 说明

<https://kubernetes.io/zh-cn/docs/concepts/services-networking/service/>

说到外部用户来访问集群内部的场景，肯定能想到传统的负载均衡的案例：

比如：nginx的upstream,Haproxy的Backend 或者 LVS的RS 功能



通过对Pod及其工作负载的控制资源RC和Deployment的学习，所有的应用服务都是工作在pod资源中，由于每个Pod都有独立的ip地址，而随着Pod的大量的动态创建和销毁操作后，会导致Pod的名称和IP很有可能发生了变化

假设有前端应用的pod和后端应用的pod，那么在频繁变动的场景中，集群中的应用之间该如何相互通信呢？难道是让我们以类似nginx负载均衡的方式手工定制pod ip然后进行——管理么？显然这是很难做到的。

Kubernetes集群提供了这样的一个资源对象Service，它定义了一组Pod的逻辑集合和一个用于访问它们的入口策略

Service 可以基于标签的方式自动找到对应的pod应用，而无需关心pod的ip地址变化与否，从而实现了类似负载均衡的效果

Service 本质上就是一个四层的反向代理，集群内和外的客户端可以通过如下流程最终实现访问Pod应用

集群内部Client --> service网络 --> Pod网络 --> 容器应用
集群外部Client --> 主机节点网络 --> service网络 --> Pod网络 --> 容器应用

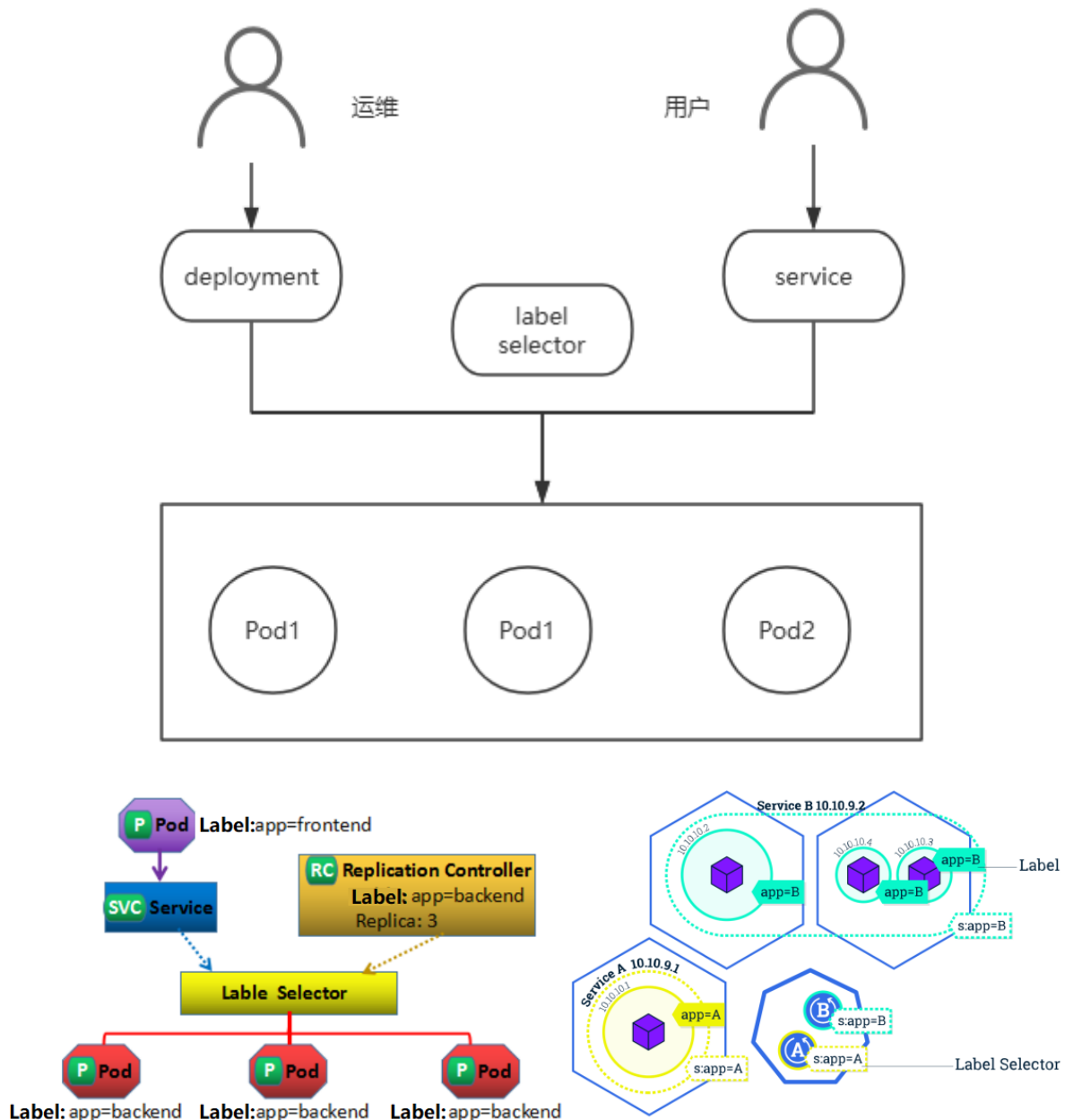
Service 资源在master端的Controller组件中，由 Service Controller 来进行统一管理。

service是Kubernetes里最核心的API资源对象之一，它是由coredns或者kube-dns组件提供的功能。

每一个Service通常都是对应一个完整的业务服务，之前的Pod、RC、Deployment等资源对象都是为Service服务的。

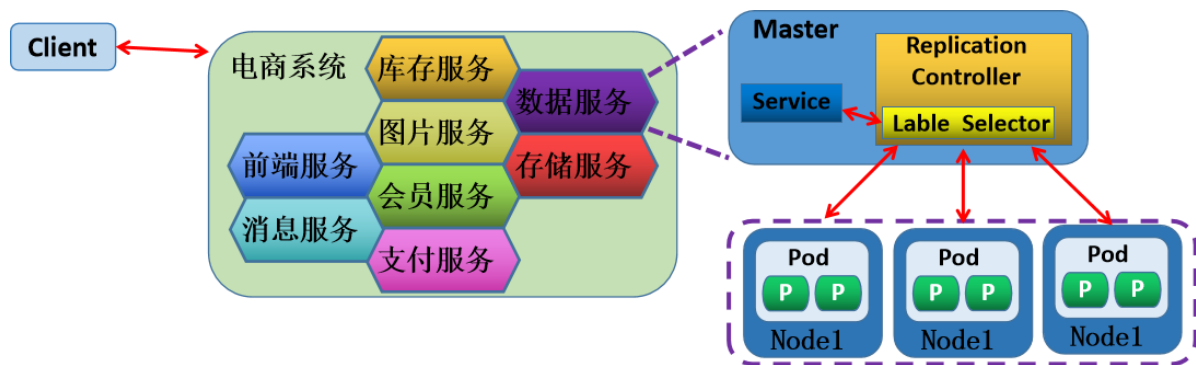
Service 是基于名称空间的资源

Service 和Pod 之间的关系



Kubernetes 的 Service定义了一个服务的访问入口地址，前端的应用Pod通过Service访问其背后一组有Pod副本组成的集群实例，Service通过Label Selector访问指定的后端Pod，RC保证Service的服务能力和服务质量处于预期状态。

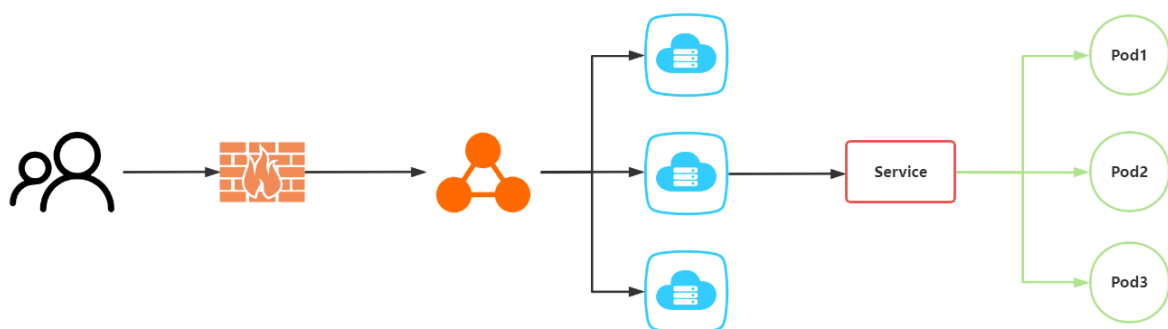
Service是Kubernetes中最高一级的抽象资源对象，每个Service提供一个独立的服务，集群Service彼此间使用TCP/IP进行通信，将不同的服务组合在一起运行起来，就形成了所谓的"系统"，效果如下图



每个Pod都有一个专用的IP地址，加上Pod内部容器的Port端口，就组成了一个访问Pod专用的EndPoint(Pod IP+Container Port)，从而实现了用户外部资源访问Pod内部应用的效果。

这个EndPoint资源在master端的Controller组件中，由EndPoint Controller 来进行统一管理。

当多个Pod组成了一个业务集群来提供外部服务，那么外部客户端怎么才能访问业务集群服务呢？



所有的Pod都有专用的Pod IP，多个Pod副本就组成了一个Kubernetes集群的后端服务，需要配置一个vip。当客户端访问的请求过来后，Master节点通过VIP把客户端的请求，负载均衡到后端的Pod副本即可。

Pod是工作在不同的Node节点上，而Node节点上有一个kube-proxy组件，它就是一个软件负载均衡器，在内部有一套专用的负载均衡与会话保持机制，可以达到接收到所有对Service请求，进而通过RANDOM算法调度转发到后端的某个具体的Pod实例上来处理该请求。

kube-proxy 本质就是 Service Controller位于各节点上的agent。

根据Pod所处的Node场景，有两种情况：

- 所有Pod副本都在同一Node节点上：对于这种情况，将Node物理节点专用的IP，作为负载均衡器的VIP，即可实现负载均衡后端服务的效果。
- 所有Pod副本不在同一Node节点上：Node节点的物理ip就没有办法作为负载均衡器的VIP了，这也是更为常见的情况。

而Kubernetes是一个分布式的资源管理平台，所以Kubernetes集群资源对象肯定不能保证在同一台Node节点，所以就不能使用Node节点的物理ip地址作为负载均衡器VIP地址了。

那么应该怎么解决这种问题呢？

Kubernetes发明了一种设计，给Service分配一个全局唯一的虚拟ip地址--cluster IP，它不存在任何网络设备，Service通过内部的标签选择器，指定相应该Service的Pod资源，当请求发给cluster IP，后端的Pod资源收到请求后，就会响应请求。

这种情况下，每个Service都有一个全局唯一通信地址，整个系统的内部服务间调用就变成了最基础的TCP/IP网络通信问题。如果我们的集群内部的服务想要和外部的网络进行通信，可以有多种方法，比如：

- NodePort类型，通过在所有节点上增加一个对外的端口，用于接入集群外部请求

- ingress类型, 通过集群附加服务功能, 将外部的域名流量转交到集群内部。

Service 核心功能

- 服务发现: 利用标签选择器, 在同一个namespace中筛选符合条件的Pod, 从而发现一组提供了相同服务的Pod, 可以结合readiness探针实现健康性检查
- 负载均衡: Service作为流量入口和负载均衡器, 其入口为ClusterIP, 这组筛选出的Pod的IP地址, 将作为该Service的后端服务器
- 名称解析: 利用Cluster DNS, 为该组Pod所代表的服务提供一个名称, 在DNS中 对于每个Service, 自动生成一个A、PTR和SRV记录

范例: 查看Kubernetes 内置的Service

```
[root@master1 ~]#kubectl get svc
NAME                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
kubernetes          ClusterIP     10.96.0.1     <none>         443/TCP          72d
```

```
[root@master1 ~]#kubectl get svc kubernetes -o yaml
```

```
apiVersion: v1
kind: Service
metadata:
  creationTimestamp: ""
  labels:
    component: apiserver
    provider: kubernetes
  name: kubernetes
  namespace: default
  resourceVersion: "193"
  uid: 6f24f80c-9ccc-4102-8ab8-0667fee18edd
spec:
  clusterIP: 10.96.0.1
  clusterIPs:
  - 10.96.0.1
  internalTrafficPolicy: Cluster
  ipFamilies:
  - IPV4
  ipFamilyPolicy: SingleStack
  ports:
  - name: https
    port: 443
    protocol: TCP
    targetPort: 6443
  sessionAffinity: None
  type: ClusterIP
status:
  loadBalancer: {}
```

#对应同名的Endpoint

```
[root@master1 ~]#kubectl describe ep kubernetes
```

```
Name:                kubernetes
Namespace:           default
Labels:               endpointslice.kubernetes.io/skip-mirror=true
Annotations:          <none>
Subsets:
  Addresses:          10.0.0.200
  NotReadyAddresses:  <none>
  Ports:
    Name  Port  Protocol
```

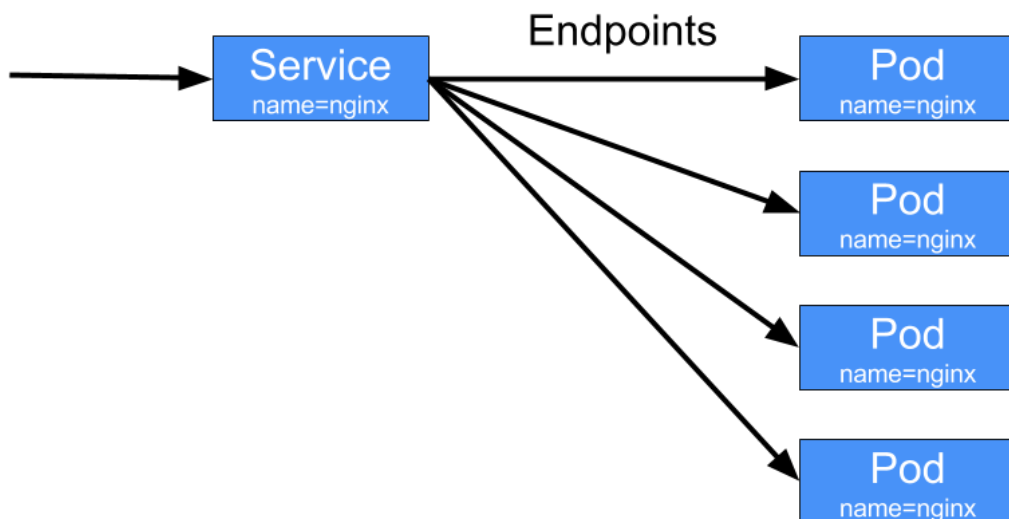
1.1.2 Endpoints

当创建 Service资源的时候，最重要的就是为Service指定能够提供服务的标签选择器

Service Controller就会根据标签选择器会自动创建一个同名的Endpoint资源对象，Kubernetes新版中还增加了endpointslices资源

- Endpoint Controller使用Endpoint的标签选择器(继承自Service标签选择器)，筛选符合条件(包括符合标签选择器条件和处于Ready 状态)的pod资源
- Endpoint Controller 将符合要求的pod资源绑定到Endpoint上，并告知给Service资源谁可以正常提供服务
- Service 会自动获取一个固定的 cluster IP向外提供由Endpoint提供的服务资源
- Service 其实就是为动态的一组 pod 资源对象提供一个固定的访问入口。即 Service实现了后端Pod应用服务的发现功能

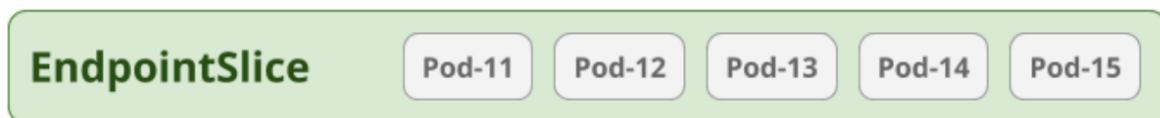
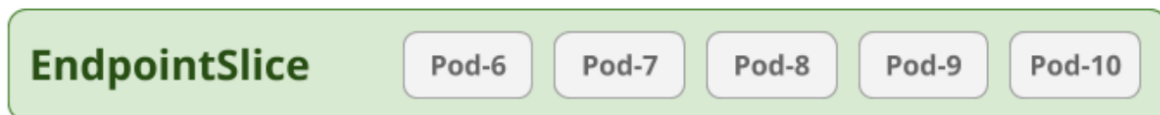
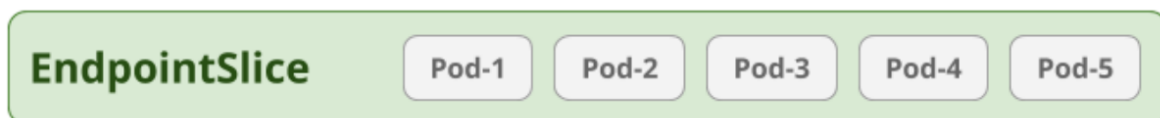
Endpoints



- 每创建一个Service ,自动创建一个和之同名的API 资源类型 Endpoints
- Endpoints负责维护由相关Service标签选择器匹配的Pod对象
- Endpoints对象上保存Service匹配到的所有Pod的IP和Port信息,称之为端点
- ETCD是K/V数据库, 而一个Endpoints对象对应一个Key,所有后端Pod端点信息为其Value
- 当一个Endpoints对象对应后端每个Pod的每次变动，都需更新整个Endpoints对象，并将新的Endpoints对象重新保存至API Server和ETCD
- 此外还需要将该对象同步至每个节点的kube-proxy
- 在ETCD中的对象默认最大为1.5MB,一个Endpoints对象至多可以存储5000个左右的端点信息,这意味着平均每端点占300KB

1.1.3 EndpointSlice

新版Kubernetes为什么需要引用EndpointSlice呢?



- 基于Endpoints机制，即便只有一个Pod的IP等信息发生变动，就需要向集群中的每个节点上的 kube-proxy发送整个endpoints对象
- 比如: 一个由2000个节点组成的集群中，更新一个有5000个Pod IP占用1.5MB空间的Endpoints 对象，就需要发送3GB的数据

若以滚动更新机制，一次只升级更新一个Pod的信息，这将导致更新这个Endpoints对象需要发送15T的数据

- EndpointSlice资源通过将Endpoints切分为多片来解决上述问题
- 自Kubernetes v1.16引入EndpointSlice
- 每个端点信息的变动，仅需要更新和发送一个EndpointSlice对象,而非整个Endpoints对象
- 每个EndpointSlice默认存储100个端点信息，不会超过 etcd对单个对象的存储限制
- 可在kube-controller-manager程序上使用 --max-endpoints-per-slice 选项进行配置
- EndpointSlice并未取代Endpoints，二者同时存在

范例: 查看 Service, Endpoints 和 EndpointSlices

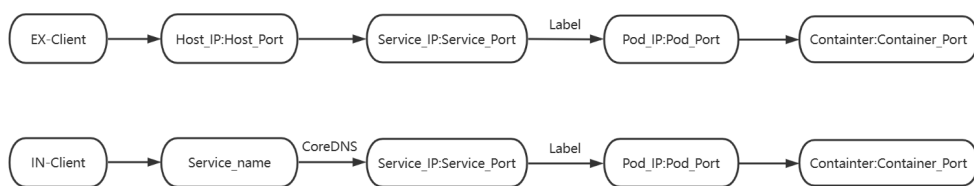
```
[root@master1 ~]#kubectl get svc,ep,endpointlices -n demo
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
service/nfs-server	ClusterIP	10.99.201.239	<none>	2049/TCP,111/UDP

NAME	ENDPOINTS	AGE
endpoints/nfs-server	10.244.1.3:2049,10.244.1.3:111	2d2h

NAME	ENDPOINTS	AGE	ADDRESSTYPE	PORTS
endpointslice.discovery.k8s.io/nfs-server-mtzqk	10.244.1.3	2d2h	IPv4	111,2049

1.1.4 Service 的访问流程



1.2 Service 工作模型

1.2.1 代理模型

一个Service对象最终体现为工作节点上的一些iptables或ipvs规则，这些规则是由kube-proxy进行实时生成和维护

针对某一特定服务，如何将集群中的每个节点都变成其均衡器：

- 在每个节点上运行一个kube-proxy，由kube-proxy注册监视API Server的Service资源的创建、修改和删除
- 将Service的定义，转为本地负载均衡功能的落地实现

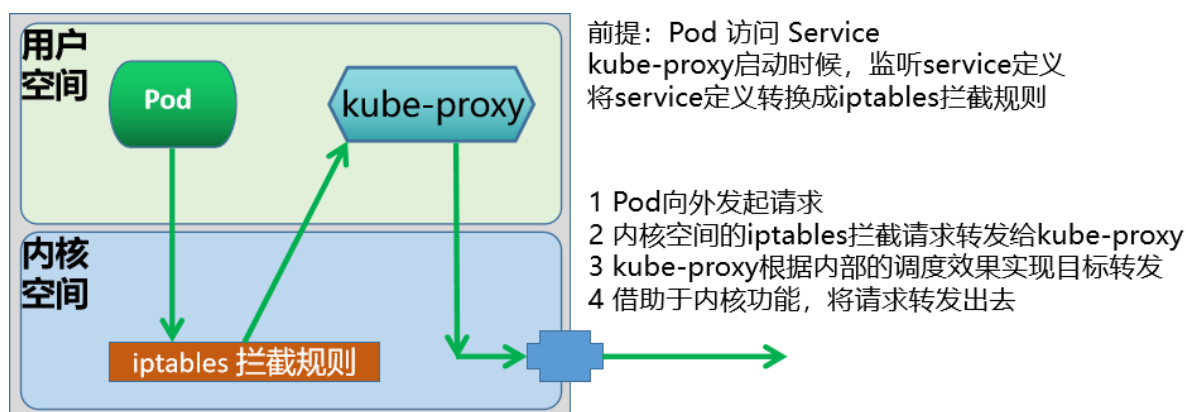
kube-proxy 将请求代理至相应端点的实现方式有五种：

- userspace 早期k8s实现
- iptables: 较旧的内核，iptables工具
- nftables: 依赖较新的内核，从kubernetes-v1.29.0 之后版本支持,nft 工具
- ipvs
- kernelspace: 仅Windows使用

这些方法的目的是：kube-proxy如何确保service能在每个节点实现并正常工作

注意: 一个集群只能选择使用一种Mode，即集群中所有节点要使用相同的方式

1.2.1.1 Userspace



userspace模型是Kubernetes最早的一种工作模型，从kubernetes v1.1版本之前使用,在kubernetes v1.2以后淘汰

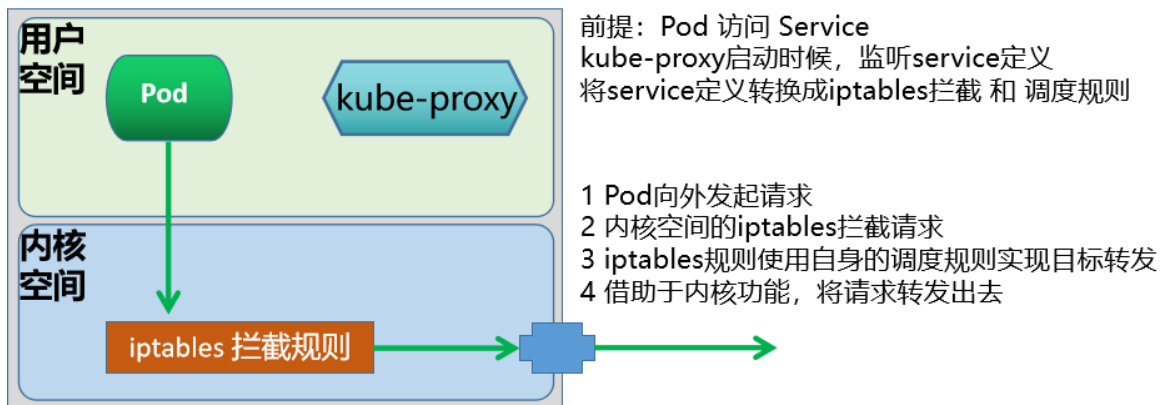
userspace模式早由 Kube-proxy 自己来负责将service 策略转换成iptables规则

这些iptables规则仅做请求的拦截，而不对请求进行调度处理。

流量处理过程

- Pod向外的请求流量先到达内核空间后，经过Iptables的规则后，再由套接字送往用户空间的kube-proxy
- 之后由kube-proxy将流量送回内核空间，并调度至后端Pod。因为涉及到来回转发，效率低下

1.2.1.2 Iptables



Iptables模式在kubernetes v1.1版本开始使用。kubernetes v1.2 版本 iptables 成为默认代理模式

Kube-proxy 自动为Service创建iptables规则

将由iptables规则直接捕获Pod向外发往ClusterIP和Port的请求流量，并重定向至当前Service的后端

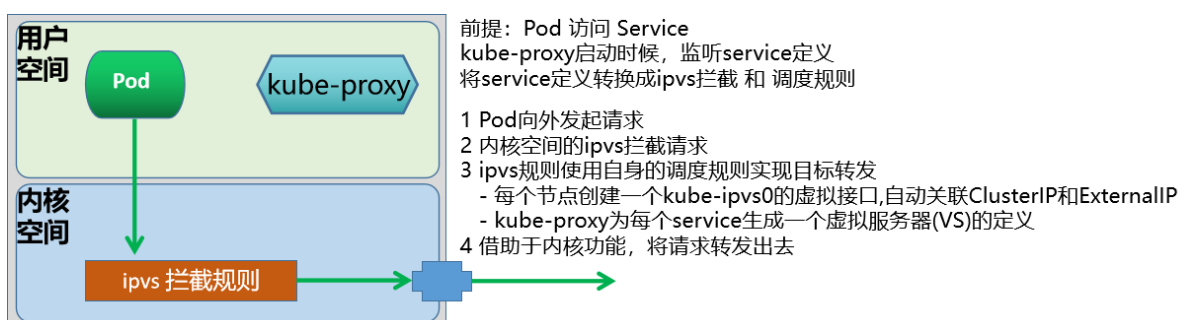
对每个Endpoints对象，Service资源会为其创建iptables规则并关联至挑选的后端Pod资源

优点：性能比userspace更加高效和可靠

缺点：

- 不会在后端Pod无响应时自动重定向，而userspace可以
- 一个service一般对应生成20条左右iptables规则
- 中等规模的Kubernetes集群(几百个service)能够承受，但是大规模的Kubernetes集群(几千个service)可达几万条规则，性能较差
- 集群节点数少于50个,服务数量少于2000个,可以使用此模式,否则使用IPVS模式性能更好
- iptables 默认基于random算法调度
- kube-proxy在 iptables模式下的复杂度程度为O(n)，即规模越大，性能越差

1.2.1.3 IPVS



kubernetes v1.8 引入 ipvs 代理模块

kubernetes v1.9 ipvs 代理模块成为 beta 版本

kubernetes v1.11 ipvs 代理模式 GA，成为默认设置

请求流量的调度功能由ipvs实现，余下的其他功能仍由iptables完成

ipvs流量转发速度快，规则同步性能好

IPVS默认基于rr算法调度，同时也支持众多调度算法，如rr/lc/dh/sh/sed/nq等

二进制Kubernetes的安装方式修改模式：

```
echo 'KUBE_PROXY_MODE=ipvs' >> /etc/sysconfig/kubelet
```

优点：

- 每条ipvs规则包含了两个功能：转发功能和调度功能
- kube-proxy在IPVS模式下是使用hash表作为底层数据结构，复杂度程度为O(1)，所以当服务很多时，性能更优
- 据测试，当后端服务达到5000以上且不使用保持连接keepalive时的往返响应时间，和iptables模式性能相对已经有较大优势，当达到服务达到10000时，甚至可以达到iptables模式的性能两倍，此外iptables模式的CPU也会消耗更多

注意：对于kubeadm方式安装Kubernetes集群来说，它会首先检测当前主机上是否已经包含了ipvs模块，如果加载了，就直接用ipvs中NAT模式，如果没有加载ipvs模块的话，会自动使用iptables模式。

范例: kubeadm方式将默认的iptables模式修改为IPVS模式

```
#查看当前模式,默认为iptables
[root@master1 ~]#curl 127.0.0.1:10249/proxyMode
iptables

[root@master1 ~]#apt install ipvsadm
[root@master1 ~]#ipvsadm -Ln
IP Virtual Server version 1.2.1 (size=4096)
Prot LocalAddress:Port Scheduler Flags
  -> RemoteAddress:Port           Forward Weight ActiveConn InActConn

[root@master1 ~]#kubectl edit configmap kube-proxy -n kube-system
mode: "ipvs"    #修改此行,默认为""

#删除所有kube-proxy对应的Pod,建议reboot生效
[root@master1 ~]#kubectl delete pod -n kube-system -l k8s-app=kube-proxy

#查看Pod 自动创建
[root@master1 ~]#kubectl get pod -n kube-system -l k8s-app=kube-proxy
NAME                READY   STATUS    RESTARTS   AGE
kube-proxy-6jrp9    1/1     Running   0           42s
kube-proxy-fr9q4    1/1     Running   0           42s
kube-proxy-g6svk    1/1     Running   0           42s
kube-proxy-lzlbn    1/1     Running   0           43s
kube-proxy-m55ts    1/1     Running   0           43s
kube-proxy-t94ff    1/1     Running   0           43s

#验证结果
[root@master1 ~]#curl 127.0.0.1:10249/proxyMode
ipvs

#每个节点都生成kube-ipvs0的虚拟网卡
[root@master1 ~]#ip a show kube-ipvs0
7: kube-ipvs0: <BROADCAST,NOARP> mtu 1500 qdisc noop state DOWN group default
    link/ether 02:01:44:76:7b:65 brd ff:ff:ff:ff:ff:ff
    inet 10.96.0.10/32 scope global kube-ipvs0
        valid_lft forever preferred_lft forever
    inet 10.106.205.83/32 scope global kube-ipvs0
        valid_lft forever preferred_lft forever
```

```

inet 10.0.0.11/32 scope global kube-ipvs0
    valid_lft forever preferred_lft forever
inet 10.107.246.153/32 scope global kube-ipvs0
    valid_lft forever preferred_lft forever
inet 10.0.0.99/32 scope global kube-ipvs0
    valid_lft forever preferred_lft forever
inet 10.109.41.99/32 scope global kube-ipvs0
    valid_lft forever preferred_lft forever
inet 10.96.0.1/32 scope global kube-ipvs0
    valid_lft forever preferred_lft forever
inet 10.110.50.151/32 scope global kube-ipvs0
    valid_lft forever preferred_lft forever
inet 10.0.0.10/32 scope global kube-ipvs0
    valid_lft forever preferred_lft forever
inet 10.105.92.154/32 scope global kube-ipvs0
    valid_lft forever preferred_lft forever

```

#再次查看出现IPVS规则

[root@master1 ~]#ipvsadm -Ln

IP Virtual Server version 1.2.1 (size=4096)

Prot LocalAddress:Port Scheduler Flags

	-> RemoteAddress:Port	Forward	Weight	ActiveConn	InActConn
TCP	10.96.0.1:443 rr				
->	10.0.0.101:6443	Masq	1	0	0
->	10.0.0.102:6443	Masq	1	0	0
->	10.0.0.103:6443	Masq	1	0	0
TCP	10.96.0.10:53 rr				
->	10.244.0.4:53	Masq	1	0	0
->	10.244.0.5:53	Masq	1	0	0
TCP	10.96.0.10:9153 rr				
->	10.244.0.4:9153	Masq	1	0	0
->	10.244.0.5:9153	Masq	1	0	0
UDP	10.96.0.10:53 rr				
->	10.244.0.4:53	Masq	1	0	0
->	10.244.0.5:53	Masq	1	0	0

范例: 二进制安装Kubernetes修改默认的iptables模式修改为IPVS模式

#修改文件路径

[root@node1 ~]#vim /var/lib/kube-proxy/kube-proxy-config.yaml

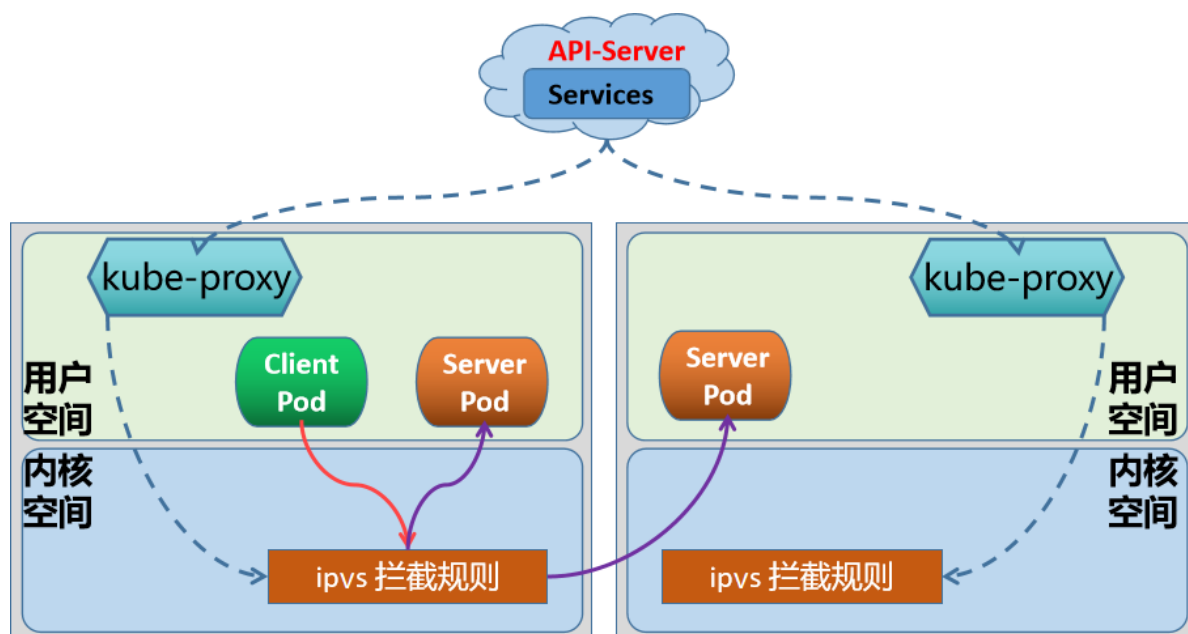
.....

mode: "ipvs"

ipvs:

 scheduler: sh

1.2.2 Service 和 kube-proxy 关联关系



- Service作为一个独立的API资源对象，它会在在API Service服务中定义出来的
- 在创建任何存在标签选择器的Service时，都会被自动创建一个同名的Endpoints资源，Endpoints对象会使用Label Selector自动发现后端端点，并各端点的IP配置为可用地址列表的元素
- Service Controller 触发每个节点上的kube-proxy，由kube-proxy实时的转换为本地节点上面的ipvs/iptables规则。
- 默认情况下，内核中的ipvs或iptables规则，仅仅是负责本地节点用户空间pod客户端发出请求时的拦截或者转发规则
- 如果Pod客户端向Service发出请求,客户端向内核发出请求，根据ipvs或iptables规则，匹配目标service
- 如果service匹配，会返回当前service随对应的后端endpoint有哪些
- iptables或ipvs会根据情况挑选一个合适的endpoint地址
 - 如果endpoint是本机上的，则会转发给本机的endpoint
 - 如果endpoint是其他主机上的，则转发给其他主机上的endpoint

1.2.3 Service 类型

对于Kubernetes 可以实现内部服务的自由通信,也可以将平台内部的服务发布到外部环境

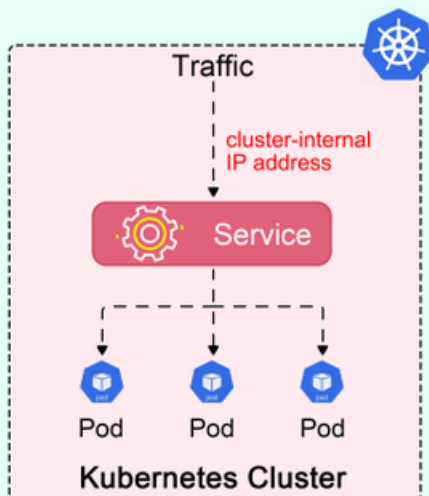
Service主要有四种类型，实现不同的网络通信功能

<https://kubernetes.io/docs/concepts/services-networking/service/#publishing-services-service-types>

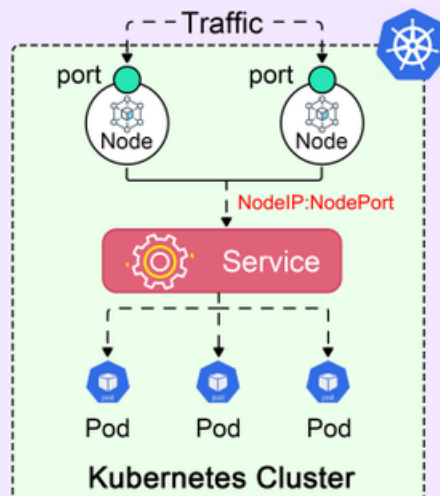
4 K8S Service Types

ByteByteGo

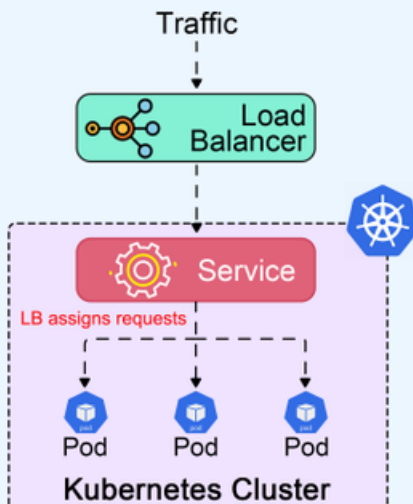
ClusterIP



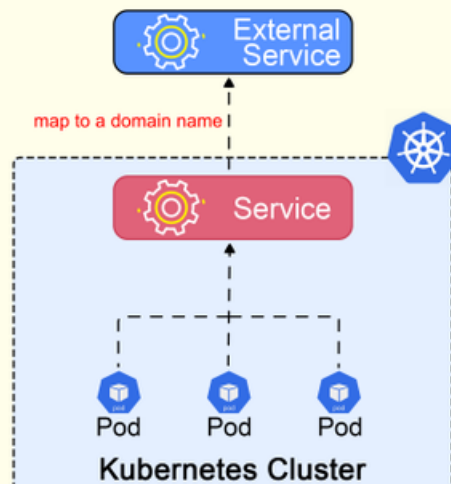
NodePort



LoadBalancer



ExternalName



类型	解析
ClusterIP	此为Service的默认类型 为集群内部的客户端访问,包括节点和Pod等, 外部网络无法访问 In client --> clusterIP: ServicePort (Service) --> PodIP: PodPort
NodePort	本质上在ClusterIP模式基础上,再多加了一层端口映射的封装,相当于增强版的ClusterIP 通过NodeIP:NodePort对外部网络提供服务, 默认随机端口范围30000~32767, 可指定为固定端口 NodePort是一个随机的端口, 以防止端口冲突,在所有安装kube-proxy的节点上都会打开此相同的端口 可通过访问ClusterIP实现集群内部访问,也可以通过NodeIP:NortPort的方式实现从集群外部至内部的访问 Ex Client --> NodeIP:NodePort (Service) --> PodIP:PodPort
LoadBalancer	基于NodePort基础之上, 使用集群外部的运营商负载均衡器方式实现对外提供服务,增强版的NodePort 基于云运营商IaaS云创建一个Kubernetes云, 云平台也支持LBaaS(Load Balance as a Service)产品服务 Master 借助cloud-manager向LBaaS的API请求动态创建软件LB,即支持和Kubernetes API Server 进行交互 如果没有云服务,将无法获取EXTERNAL-IP,显示Pending状态,则降级为NodePort类型 Ex Client --> LB_IP:LB_PORT --> NodeIP:NodePort(Service)--> PodIP:PodPort
ExternalName	当Kubernetes集群需要访问集群外部服务时, 需要通过externalName将外部主机引入到集群内部 外部主机名以 DNS方式解析为一个 CNAME记录给Kubernetes集群的其他主机来使用 这种Service既没有ClusterIP, 也没有NodePort.而且依赖于内部的CoreDNS功能 In client -->Cluster ServiceName --> CName --> External Service Name 本方式的service没有selector,所以也不会创建同名的Endpoints资源对象 示例: Service名称: MySQL服务内部域名 mysql.default.svc.cluster.local MySQL服务外部域名: mysql.wang.org In Client --> mysql.default.svc.cluster.local --> mysql.wang.org --> MYSQL IP

1.2.4 ExternalIP

<https://kubernetes.io/docs/concepts/services-networking/service/#external-ips>
<https://kubernetes.io/zh-cn/docs/concepts/services-networking/service/#external-ips>

从外部访问 Kubernetes的Pod,除了使用 NodePort 和 LoadBalancer 的Service 类型,也可以使用 ExternalIP 实现

如果有外部 IP 能够路由到一个或多个集群节点上, 则 Kubernetes 可以通过 `externalIPs` 将Service公开出去。

当网络流量进入集群时，如果 `externalIPs` 作为目的 IP 地址和端口都与该 Service 匹配，Kubernetes 所配置的规则和路由会确保流量被路由到该 Service 的端点之一。

Service 可通过使用节点上配置的辅助 IP 地址接入集群外部客户端流量

流量入口仅能是配置有该 IP 地址的节点，其它节点无效，因而此时在节点间无负载均衡的效果

external IP 所在的节点故障后，该流量入口失效，除非将该 IP 地址转移配置到其它节点

external IP 是除了 LoadBalancer 和 NodePort 外，接入外部流量的又一种方式

定义 Service 时，你可以为任何服务类型 externalIPs。

由于 externalIPs 是绑定在某一个节点上，而此节点存在单点问题，可以通过 Keepalived 等方式实现高可用

在下面的例子中，名为 `my-service` 的服务可以在 "`198.51.100.32:80`"（根据 `.spec.externalIPs[]` 和 `.spec.ports[].port` 得出）上被客户端使用 TCP 协议访问。

还需要在某个节点绑定此 IP 1.2.3.4 到网卡上

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app.kubernetes.io/name: MyApp
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 49152
  externalIPs:
    - 1.2.3.4
```

1.3 Service 管理

1.3.1 创建 Service 方式说明

<https://kubernetes.io/zh-cn/docs/concepts/services-networking/service/>

对于 Service 的创建有两种方法：

- 命令行方法
- YAML 文件方法

1.3.1.1 命令行方法

<http://docs.kubernetes.org.cn/564.html>

基础语法

```
#创建命令1:
kubectl create service [flags] NAME [--tcp=port:targetPort] [--dry-run]

#作用：单独创建一个service服务
flags 参数详解：
```


clusterip	Create a ClusterIP service .将集群专用服务接口
nodeport	创建一个 NodePort service .将集群内部服务以端口形式对外提供
loadbalancer	创建一个 LoadBalancer service .主要针对公有云服务
externalname	Create an ExternalName service .将集群外部服务引入集群内部

#创建命令2:

```
kubectl expose (-f FILENAME | TYPE NAME) [--port=port] [--protocol=TCP|UDP|SCTP]
[--target-port=number-or-name] [--name=name] [--external-ip=external-ip-of-
service] [--type=type] [options]
```

#作用: 针对一个已存在的deployment、pod、ReplicaSet等创建一个service

#参数详解:

```
--port=''          #设定service对外的端口信息
--target-port=''    #设定容器的端口,默认和service的端口相同
--type=''           #设定类型,支持四种: ClusterIP(默认), NodePort,
LoadBalancer, ExternalName
--cluster-ip=''     #设定对外的ClusterIP地址
--name=''           #创建service对外的svc名称
```

#创建命令3: 创建自主式Pod时创建Service

```
kubectl run <Pod_name> --image 镜像 --expose --port <容器端口>
```

#查看命令:

```
kubectl get svc
```

#查看更多信息的命令是一个通用的命令,一般常用两种方法显示更多的内容: yaml和json

#语法:

```
kubectl get 资源类型 [类型名称] -o yaml
kubectl get 资源类型 [类型名称] -o json
```

#yaml格式查看更多Service的信息

```
kubectl get svc <service-name> -o yaml
```

#注意:

在spec.ports的定义中, targetPort属性就是用来确定提供该服务的容器所暴露(EXPORT)的端口号,即通过targetPort端口来访问业务进程在容器。

默认情况下targetPort跟我们容器定义的port一致。

#删除 service

```
kubectl delete svc <svc_name> [ -n <namespace>[] [--all]
```

范例: 基于自主式Pod自动创建Service

```
[root@master1 ~]#kubectl run myapp --image registry.cn-
beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1 --expose --port 80
service/myapp created
pod/myapp created
```

```
[root@master1 ~]#kubectl get svc,pod
```

NAME	PORT(S)	AGE	TYPE	CLUSTER-IP	EXTERNAL-IP
service/myapp	80/TCP	99s	ClusterIP	10.109.23.81	<none>

NAME	READY	STATUS	RESTARTS
AGE			
pod/myapp	1/1	Running	0
99s			


```
[root@master1 ~]#kubectl get ep myapp
NAME      ENDPOINTS          AGE
myapp     10.244.1.114:80    3m8s
```

```
[root@master1 ~]#curl 10.109.23.81
kubernetes pod-test v0.1!! ClientIP: 10.244.0.0, ServerName: myapp, ServerIP:
10.244.1.114!
```

范例: kubectl create 创建 service

```
#简单示例
[root@master1 ~]#kubectl create service clusterip my-service1 --tcp=5678:80
[root@master1 ~]#kubectl create service clusterip my-service2 --clusterip="None"
[root@master1 ~]#kubectl create service clusterip my-service3 --
clusterip="192.168.100.100" --tcp="999"
```

```
#查看效果
[root@master1 ~]#kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	192.168.0.1	<none>	443/TCP	21h
my-service1	ClusterIP	192.168.146.176	<none>	5678/TCP	94s
my-service2	ClusterIP	None	<none>	<none>	58s
my-service3	ClusterIP	192.168.100.100	<none>	999/TCP	2s

```
#结果显示:
cluster-ip是自动生成的, 如果不需要, 值指定为None
如果要自定义cluster-ip, 必须是集群初始化指定的IP范围, 默认是 10.96.0.0/12
```

范例: kubectl expose 创建 service

```
#对已存在Deployment创建service
[root@master1 ~]#kubectl create deployment myweb01 --image registry.cn-
beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1 --replicas 3

[root@master1 ~]#kubectl create deployment myweb02 --image registry.cn-
beijing.aliyuncs.com/wangxiaochun/pod-test:v0.2 --replicas 3

[root@master1 ~]#kubectl expose deployment myweb01 --port=81 --target-port=80
[root@master1 ~]#kubectl create svc clusterip myweb01 --tcp=81:80
[root@master1 ~]#kubectl expose deployment myweb02 --port=82 --target-port=80 --
type=NodePort
```

```
[root@master1 ~]#kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	4m22s
myweb01	ClusterIP	10.98.236.20	<none>	81/TCP	11s
myweb02	NodePort	10.99.187.62	<none>	82:30084/TCP	3s

```
#自动生成同名的Endpoint
[root@master1 ~]#kubectl get endpoints
```

NAME	ENDPOINTS	AGE
kubernetes	10.0.0.200:6443	5m54s
myweb01	10.244.1.92:81,10.244.2.63:81,10.244.3.31:81	103s
myweb02	10.244.1.93:82,10.244.2.64:82,10.244.3.32:82	95s

```
[root@master1 ~]#kubectl describe service myweb01
Name: myweb01
Namespace: default
Labels: app=myweb01
Annotations: <none>
Selector: app=myweb01
Type: ClusterIP
IP Family Policy: SingleStack
IP Families: IPv4
IP: 10.98.236.20
IPs: 10.98.236.20
Port: <unset> 81/TCP
TargetPort: 81/TCP
Endpoints: 10.244.1.92:81,10.244.2.63:81,10.244.3.31:81
Session Affinity: None
Events: <none>
```

```
[root@master1 ~]#kubectl describe service myweb02
Name: myweb02
Namespace: default
Labels: app=myweb02
Annotations: <none>
Selector: app=myweb02
Type: NodePort
IP Family Policy: SingleStack
IP Families: IPv4
IP: 10.99.187.62
IPs: 10.99.187.62
Port: <unset> 82/TCP
TargetPort: 82/TCP
NodePort: <unset> 30084/TCP
Endpoints: 10.244.1.93:82,10.244.2.64:82,10.244.3.32:82
Session Affinity: None
External Traffic Policy: Cluster
Events: <none>
```

#清理环境

```
[root@master1 ~]#kubectl delete deployments.apps myweb0{1,2}
[root@master1 ~]#kubectl delete svc myweb0{1,2}
```

范例:

```
#一步实现创建所有pod和相应的service
[root@master1 ~]#kubectl run pod-test --image=registry.cn-
beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1 --expose=true --port=80
service/pod-test created
pod/pod-test created
```

```
[root@master1 ~]#kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/pod-test	1/1	Running	0	7s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	192.168.0.1	<none>	443/TCP	21h
service/pod-test	ClusterIP	192.168.76.54	<none>	80/TCP	7s

```
[root@master1 ~]#kubectl get pod --show-labels -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED	NODE	READINESS	GATES	LABELS		
pod-test	1/1	Running	0	73s	172.16.2.90	node3.wang.org
<none>	<none>	run=pod-test				

```
[root@master1 ~]#curl 192.168.76.54
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: pod-test, ServerIP: 172.16.2.90!
```

范例:

```
#两步实现
[root@master1 ~]#kubectl create deployment web --image=registry.cn-
beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
[root@master1 ~]#kubectl expose deployment web --name=web-service --port=80
[root@master1 ~]#kubectl expose deployment web --name=web-nodeport --port=999 --
target-port='80' --type='NodePort'
```

```
[root@master1 ~]#kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/web-65655fc68-cr2z7	1/1	Running	0	116s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
AGE				
service/kubernetes	ClusterIP	192.168.0.1	<none>	443/TCP
21h				
service/web-nodeport	NodePort	192.168.86.207	<none>	999:30959/TCP
3s				
service/web-service	ClusterIP	192.168.108.144	<none>	80/TCP
62s				

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/web	1/1	1	1	116s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/web-65655fc68	1	1	1	116s

```
[root@master1 ~]#curl 192.168.108.144
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: web-65655fc68-
cr2z7, ServerIP: 172.16.2.91!
```

#注意: NodePort类型的Service会在当前集群的所有主机上开放一个服务端口, 这个服务端口是随机的

```
[root@master1 ~]#curl 192.168.86.207:999
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: web-65655fc68-
cr2z7, ServerIP: 172.16.2.91!
[root@master1 ~]#curl 10.0.0.101:30959
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: web-65655fc68-
cr2z7, ServerIP: 172.16.2.91!
[root@master1 ~]#curl 10.0.0.106:30959
kubernetes pod-test v0.1!! ClientIP: 172.16.2.1, ServerName: web-65655fc68-
cr2z7, ServerIP: 172.16.2.91!
```

#清理还原环境

```
[root@master1 ~]#kubectl delete deployments.apps web
[root@master1 ~]#kubectl delete svc web-nodeport web-service
```

范例:

```
[root@master1 ~]#kubectl create deployment deployment-test --image registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1 --replicas 2
```

#查看pod信息

```
[root@master1 ~]#kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP
NODE	NOMINATED	READINESS	GATES		
deployment-test-f96bdb9-cxbm5	1/1	Running	0	57s	
10.244.2.86	node2.wang.org	<none>	<none>		
deployment-test-f96bdb9-rhwrc	1/1	Running	0	57s	
10.244.3.62	node3.wang.org	<none>	<none>		

#对已有的Deployment创建对应的service

```
[root@master1 ~]#kubectl expose deployment deployment-test --port 80
```

#查看service

```
[root@master1 ~]#kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
deployment-test	ClusterIP	10.111.116.76	<none>	80/TCP	4s
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	22d

#可以看到后端的两个Pod的IP地址

```
[root@master1 ~]#kubectl get ep
```

NAME	ENDPOINTS	AGE
deployment-test	10.244.2.86:80,10.244.3.62:80	9s
kubernetes	10.0.0.200:6443	22d

#测试名称解析, 10.96.0.10为coredns服务器地址

```
[root@master1 ~]#dig -t A deployment-test.default.svc.cluster.local @10.96.0.10
+short
10.97.30.140
```

#通过service的IP访问后端的Pod

```
[root@master1 ~]#curl 10.111.116.76
```

```
kubernetes pod-test v0.1!! ClientIP: 10.244.0.0, ServerName: deployment-test-f96bdb9-rhwrc, ServerIP: 10.244.3.62!
```

```
[root@master1 ~]#curl 10.111.116.76
```

```
kubernetes pod-test v0.1!! ClientIP: 10.244.0.0, ServerName: deployment-test-f96bdb9-cxbm5, ServerIP: 10.244.2.86!
```

#创建测试的Pod

```
[root@master1 ~]#kubectl run busybox --image busybox:1.30.0 -- sleep 3600
```

#测试访问service

```
[root@master1 ~]#kubectl exec -it busybox -- sh
```

#无法ping通

```
/ # ping -c1 deployment-test
```

```
PING deployment-test (10.111.116.76): 56 data bytes
```

```
--- deployment-test ping statistics ---
```

```
1 packets transmitted, 0 packets received, 100% packet loss
```

#可以通过service名称访问http服务

```
/ # wget -qO - deployment-test
```

```
kubernetes pod-test v0.1!! ClientIP: 10.244.3.59, ServerName: deployment-test-f96bdbc9-cx5m5, ServerIP: 10.244.2.86!  
/ # wget -qO - deployment-test  
kubernetes pod-test v0.1!! ClientIP: 10.244.3.59, ServerName: deployment-test-f96bdbc9-rhwrc, ServerIP: 10.244.3.62!
```

1.3.1.2 文件方式

<https://kubernetes.io/docs/concepts/services-networking/service/>

语法解析

```
apiVersion: v1  
kind: Service  
metadata:  
  name: ...  
  namespace: ...  
  labels:  
    key1: value1  
    key2: value2  
spec:  
  type: <string> # Service类型，默认为ClusterIP  
  selector: <map[string]string> # 指定用于过滤出service所代理的后端Pod的标签，只支持  
    等值类型的标签选择器，多个条件内含“与”逻辑  
  ports:  
    # Service的端口对象列表  
    - name: <string> # 端口名称，需要保证唯一性  
      protocol: <string> # 协议，目前仅支持TCP、UDP和SCTP，默认为TCP  
      port: <integer> # Service的端口号  
      targetPort: <string> # 后端Pod的端口号或名称，名称需由Pod中的规范定义  
      nodePort: <integer> # 节点端口号，仅适用NodePort和LoadBalancer类型，范  
    围：30000-32768，建议不指定而由系统自动分配  
    - name: <string>  
      .....  
  clusterIP: <string> # 指定Service的集群IP，建议不指定而由系统自动分配  
  internalTrafficPolicy: <string> # 内部流量策略处理方式，Local表示由当前节点处理，  
    Cluster表示向集群范围调度，默认值为Cluster  
  externalTrafficPolicy: <string> # 外部流量策略处理方式，默认为Cluster，当为Local  
    时，表示由当前节点处理，性能较好，但无负载均衡功能，且可以看到真实客户端IP，Cluster表示向集群范  
    围调度，和Local相反，基于性能原因，生产更建议Local，此方式只支持type是NodePort和  
    LoadBalancer类型或者ExternalIPs  
  loadBalancerIP: <string> # 外部负载均衡器使用的IP地址，仅适用于LoadBalancer，  
    此字段未来可能被删除  
  externalName: <string> # 外部服务名称，该名称将作为Service的DNS CNAME值  
  externalIPs <[]string> # 群集中的节点将接受此服务的流量的 IP 地址列表。这些  
    IP不由 Kubernetes管理。用户负责确保流量到达具有此 IP 的节点。常见的是不属于 Kubernetes 系统  
    的外部负载均衡器，注意：此IP和Type类型无关
```

1.3.2 ClusterIP Service 实现

范例：查看集群内默认 Service

#在集群内kubernetes对应的SVC默认占用Service 网络的第一个IP

```
[root@master1 ~]#kubectl get svc kubernetes
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	410d

#DNS对应的SVC，默认占用Service网络的第10个IP

```
[root@master1 ~]#kubectl get svc -n kube-system
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kube-dns	ClusterIP	10.96.0.10	<none>	53/UDP, 53/TCP, 9153/TCP	410d

范例：单端口应用

#如果基于资源配置文件创建资源，依赖于后端pod的标签，才可以关联后端的资源

#准备多个后端pod对象

```
[root@master1 ~]#kubectl create deployment myweb --image=registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1 --replicas=3
```

#检查效果

```
[root@master1 ~]#kubectl get pod --show-labels -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
myweb-5df49fdd5-cd8b4	1/1	Running	0	18s	172.16.1.56	node2.wang.org
myweb-5df49fdd5-mvmrt	1/1	Running	0	18s	172.16.3.106	node1.wang.org
myweb-5df49fdd5-s2k5w	1/1	Running	0	18s	172.16.2.93	node3.wang.org

#创建service对象

```
[root@master1 ~]#cat service-clusterip-test.yaml
```

```
kind: Service
```

```
apiVersion: v1
```

```
metadata:
```

```
  name: service-clusterip-test
```

```
spec:
```

```
  #type: ClusterIP #默认即为ClusterIP，此行可省略
```

```
  #clusterIP: 192.168.64.100 #可以手动指定IP，但一般都是系统自动指定而无需添加此行
```

```
  selector:
```

```
    app: myweb #引用上面deployment的名称，同时也是Pod的Label中app的值，实现
```

service代理指定Pod的功能

```
    #version: v1.0 #可以添加多个label与关系，即只过滤出有此处label匹配的pod，而非
```

deployment的名称

```
  ports:
```

```
    - name: http
```

```
      protocol: TCP
```

```
      port: 80
```

```
      targetPort: 80
```

#Service的端口

#后端Pod对应的端口

```
[root@master1 ~]#kubectl apply -f service-clusterip-test.yaml
```

#查看

```
[root@master1 ~]#kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/myweb-5df49fdd5-cd8b4	1/1	Running	0	2m16s
pod/myweb-5df49fdd5-mvmrt	1/1	Running	0	2m16s
pod/myweb-5df49fdd5-s2k5w	1/1	Running	0	2m16s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
service/kubernetes 443/TCP 21h	ClusterIP	192.168.0.1	<none>
service/service-clusterip-test 80/TCP 2s	ClusterIP	192.168.64.158	<none>

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/myweb	3/3	3	3	2m16s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/myweb-5df49fdd5	3	3	3	2m16s

[root@master1 ~]#kubectl get svc

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	192.168.0.1	<none>	443/TCP	21h
service-cluster-test	ClusterIP	192.168.64.158	<none>	80/TCP	75s

#自动创建同名的endpoint

[root@master1 ~]#kubectl get ep

NAME	ENDPOINTS	AGE
kubernetes	10.0.0.200:6443	9d
service-clusterip-test	172.16.1.56:80,172.16.2.93:80,172.16.3.106:80	5s

#查看service信息

[root@master1 ~]#kubectl describe svc service-clusterip-test

```

Name:          service-clusterip-test
Namespace:     default
Labels:        <none>
Annotations:   <none>
Selector:      app=myweb
Type:          ClusterIP
IP Family Policy: SingleStack
IP Families:   IPv4
IP:            192.168.64.158    #对应的ClusterIP
IPs:           192.168.64.158
Port:          http 80/TCP
TargetPort:    80/TCP
Endpoints:     172.16.1.56:80,172.16.2.93:80,172.16.3.106:80  #对应后端Pod的地址
Session Affinity: None
Events:        <none>

```

#结果显示: service 自动关联了四个Endpoints

#访问验证

[root@master1 ~]#curl 192.168.64.158

kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-cd8b4, ServerIP: 172.16.1.56!

[root@master1 ~]#curl 192.168.64.158

kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-mvmrt, ServerIP: 172.16.3.106!

[root@master1 ~]#curl 192.168.64.158

```
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-s2k5w, ServerIP: 172.16.2.93!
```

#结果显示: 后端随机代理到不同的pod应用了

#在测试从其它Pod通过service名称访问,内部通过修改目标IP的DNAT实现,但源地址不变

```
[root@master1 ~]#kubectl run client-test-$RANDOM --image registry.cn-beijing.aliyuncs.com/wangxiaochun/admin-box:v0.1 --restart=Never --rm -it --command -- /bin/bash
```

```
root@client-test-12345~# curl service-clusterip-test
```

```
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-cd8b4, ServerIP: 172.16.1.56!
```

#测试从Service的自身的Pod访问Service

#注意:当访问Pod访问自己所对应的Service时,恰恰被调度到自身Pod时会修改源IP为其它IP(相当于FULLNAT),而被调度到其它的Pod时则保留其源IP,相当于只DNAT

#Pod自身发起访问最终至自身的流量称为发夹流量

```
[root@master1 ~]#kubectl exec -it myweb-5df49fdd5-cd8b4 --command -- /bin/bash
```

```
root@myweb-5df49fdd5-cd8b4~# curl service-clusterip-test
```

```
kubernetes pod-test v0.1!! ClientIP: 172.16.1.1, ServerName: myweb-5df49fdd5-cd8b4, ServerIP: 172.16.1.56!
```

```
kubernetes pod-test v0.1!! ClientIP: 172.16.1.56, ServerName: myweb-5df49fdd5-mvmrt, ServerIP: 172.16.3.106!
```

```
kubernetes pod-test v0.1!! ClientIP: 172.16.1.56, ServerName: myweb-5df49fdd5-s2k5w, ServerIP: 172.16.2.93!
```

#如果跨namespace,可以用<service>.<namespace>格式访问,一般用于公共的基础服务,比如:数据库,MQ服务等

```
root@client-test-12345~# curl service-clusterip-test.default
```

```
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-mvmrt, ServerIP: 172.16.3.106!
```

#删除并重建Pod

```
[root@master1 ~]#kubectl delete myweb-5df49fdd5-cd8b4 myweb-5df49fdd5-mvmrt myweb-5df49fdd5-s2k5w
```

#通过原来的service名称和IP都还可访问,说明service比Pod地址更加稳定,可以用于Pod之间的访问

```
root@client-test-12345~# / # wget -qO - http://service-test
```

```
kubernetes pod-test v0.1!! ClientIP: 10.244.3.11, ServerName: myweb-f95974cf4-zqbzd, ServerIP: 10.244.3.12!
```

#kubernetes-v1.29版之后,采用nftable方式查看各种规则

```
[root@node1 ~]#nft list ruleset|head
```

```
table ip nat {
    chain DOCKER {
        iifname "docker0" counter packets 0 bytes 0 return
    }

    chain POSTROUTING {
        type nat hook postrouting priority srcnat; policy accept;
        counter packets 66 bytes 5249 jump KUBE-POSTROUTING
        oifname != "docker0" ip saddr 172.17.0.0/16 counter packets 0 bytes 0
        masquerade
        counter packets 56 bytes 4649 jump FLANNEL-POSTRTG
    }
}
```

#查看Pod的调度算法RANDOM和DNAT的策略

```
[root@master1 ~]#iptables -vnL -t nat |grep random
```

```
0      0 MASQUERADE  all  --  *      *      10.244.0.0/16      !224.0.0.0/4
/* flannelD masq */ random-fully
```



```

0 0 MASQUERADE all -- * * !10.244.0.0/16
10.244.0.0/16 /* flanneld masq */ random-fully
0 0 MASQUERADE all -- * * 0.0.0.0/0 0.0.0.0/0
/* kubernetes service traffic requiring SNAT */ random-fully
0 0 KUBE-SEP-LM3EPTFUMTUQ2AGO all -- * * 0.0.0.0/0
0.0.0.0/0 /* default/service-nodeport:http -> 10.244.1.76:80 */
statistic mode random probability 0.33333333349
0 0 KUBE-SEP-W56VQGQCLB74M2RA all -- * * 0.0.0.0/0
0.0.0.0/0 /* default/service-nodeport:http -> 10.244.1.77:80 */
statistic mode random probability 0.50000000000
0 0 KUBE-SEP-ZJFRUAU4WMFGDV6X all -- * * 0.0.0.0/0
0.0.0.0/0 /* kube-system/kube-dns:dns-tcp -> 10.244.1.74:53 */
statistic mode random probability 0.50000000000
0 0 KUBE-SEP-MQJLPRJYFMDPYHQF all -- * * 0.0.0.0/0
0.0.0.0/0 /* kube-system/kube-dns:metrics -> 10.244.1.74:9153 */
statistic mode random probability 0.50000000000
0 0 KUBE-SEP-W3WEMR74BBVQWX4B all -- * * 0.0.0.0/0
0.0.0.0/0 /* kube-system/kube-dns:dns -> 10.244.1.74:53 */
statistic mode random probability 0.50000000000

#清理环境
[root@master1 ~]#kubectl delete -f service-clusterip-test.yaml

```

范例: 多端口实现

有很多服务都会同时开启多个端口, 典型的比如: tomcat三个端口, 接下来实现创建多端口的Service

```

#资源定义文件
[root@master1 ~]#cat service-clusterip-multi-port.yaml
kind: Service
apiVersion: v1
metadata:
  name: service-clusterip-multi-port
spec:
  selector:
    app: myweb
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 80
    - name: https
      protocol: TCP
      port: 443
      targetPort: 443

```

#关键点: 只能有一个ports属性, 多了会覆盖, 每一个子port必须有一个name属性, 由于service是基于标签的方式来管理pod的, 所以必须有标签选择器

#创建服务service

```

[root@master1 ~]#kubectl apply -f service-clusterip-multi-port.yaml

[root@master1 ~]#kubectl get svc

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
kubernetes	ClusterIP	192.168.0.1	<none>	443/TCP

21h

```
service-clusterip-multi-port ClusterIP 192.168.134.142 <none>
80/TCP,443/TCP 3s
```

范例: ClusterIP 类型同时指定externalIPs

ClusterIP 类型只能支持集群内容访问, 如何想从集群外部访问, 可以使用 externalIPs实现

注意: externalIPs 可以应用在所有的service模式下的

#在集群中的某个节点添加一个公网ip

```
[root@node3 ~]#ip a a 10.0.0.66/24 dev eth0 label eth0:1
```

#注意: 这里的模拟外网ip, 必须被宿主机能够访问, 而且与Kubernetes的集群Pod网段不一样

#配置清单文件

```
[root@master1 ~]#cat service-clusterip-externalip.yaml
```

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: service-clusterip-externalip
```

```
spec:
```

```
  type: ClusterIP
```

```
  selector:
```

```
    app: myweb
```

```
  ports:
```

```
  - name: http
```

```
    protocol: TCP
```

```
    port: 80
```

```
    targetPort: 80
```

```
  externalIPs:
```

```
  - 10.0.0.66
```

#注意: 这里的externalIPs对于使用哪种Service类型, 其实关联度不大。

#创建

```
[root@master1 ~]#kubectl apply -f service-clusterip-externalip.yaml
```

#查看

```
[root@master1 ~]#kubectl get svc service-test-externalip
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
service-clusterip-externalip	ClusterIP	192.168.213.247	10.0.0.66

80:32267/TCP 9s

#结果显示: EXTERNAL-IP 已经指定了公网能够正常访问的ip地址

#使用externalIPs和标准端口即可访问

```
[root@master1 ~]#curl 10.0.0.66
```

```
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-mvmrt, ServerIP: 172.16.3.106!
```

```
[root@master1 ~]#curl 10.0.0.66
```

```
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-s2k5w, ServerIP: 172.16.2.93!
```

```
[root@master1 ~]#curl 10.0.0.66
```

```
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-cd8b4, ServerIP: 172.16.1.56!
```

#结果显示: 对于EXTERNAL-IP来说, 可以直接通过该ip和容器端口来进行服务访问, 而无需通过其他端口

#清理环境

```
[root@master1 ~]#kubectl delete -f service-clusterip-externalip.yaml
```

1.3.3 NodePort Service 实现

NodePort会在所有节点主机上，向外暴露一个指定或者随机的端口，供集群外部的应用能够访问Kubernetes集群内部的Pod资源。

注意：nodePort 属性范围为 30000-32767，且type 类型只支持 NodePort或LoadBalancer

注意：此方式有安全风险，端口为非标端口,而且性能一般,生产一般较少使用

生产中建议使用Ingress方式向外暴露集群内的服务

范例：修改NodePort端口范围

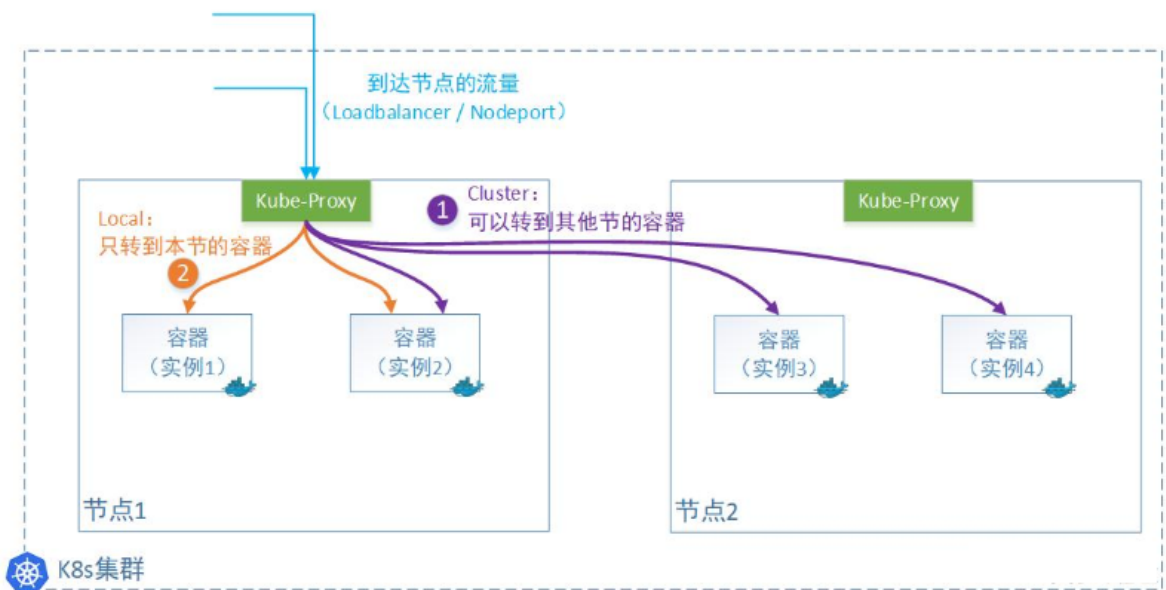
```
[root@master1 ~]#kubectl exec -it -n kube-system kube-apiserver-master1.wang.org
-- kube-apiserver --help |grep port-range
      --service-node-port-range portRange          A port range to reserve for
services with NodePort visibility. This must not overlap with the ephemeral port
range on nodes. Example: '30000-32767'. Inclusive at both ends of the range.
(default 30000-32767)

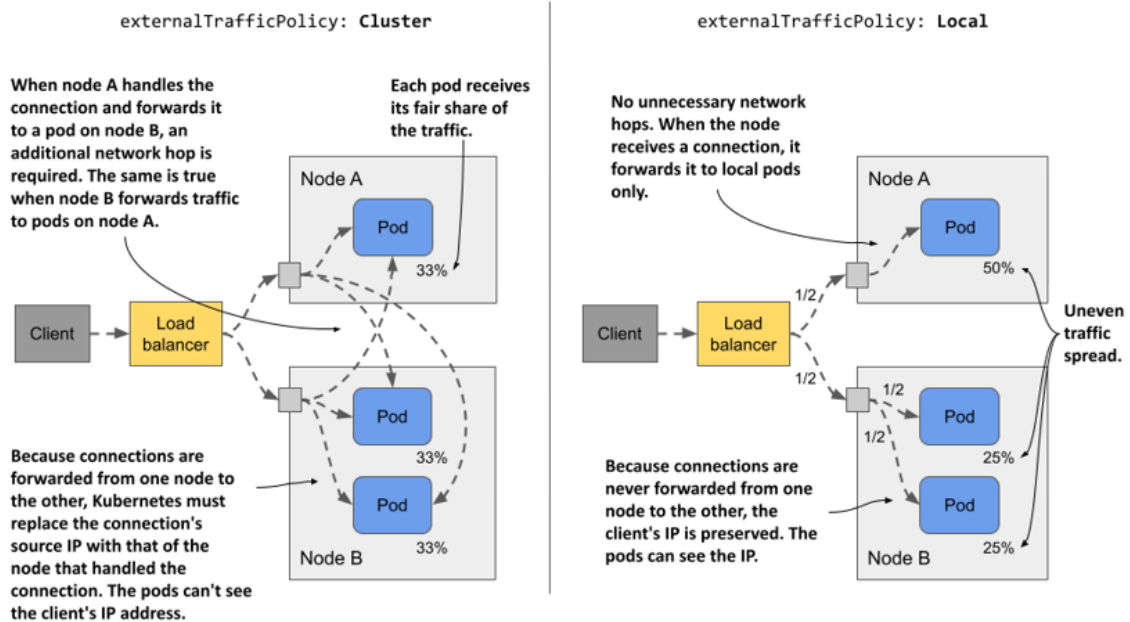
[root@master1 ~]#cp /etc/kubernetes/manifests/kube-apiserver.yaml .
[root@master1 ~]#vim /etc/kubernetes/manifests/kube-apiserver.yaml
.....
- --tls-private-key-file=/etc/kubernetes/pki/apiserver.key
- --service-node-port-range=40000-42767
image: registry.aliyuncs.com/google_containers/kube-apiserver:v1.32.0
.....
[root@master1 ~]#reboot

[root@master1 ~]#kubectl create svc nodeport myweb --tcp=80:80
service/myweb created
[root@master1 ~]#kubectl get svc myweb
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
myweb	NodePort	10.100.140.127	<none>	80:41044/TCP	7s

externaltrafficpolicy 有两种策略





- cluster 模式

此为默认模式

集群外的请求报文从某节点的NodePort进入,该节点的Service 可以将请求流量调度到其他节点上的Pod,无需关心Pod在哪个节点上

Kube-proxy转发外部请求时会替换掉报文的源IP和目标IP,相当于FULLNAT

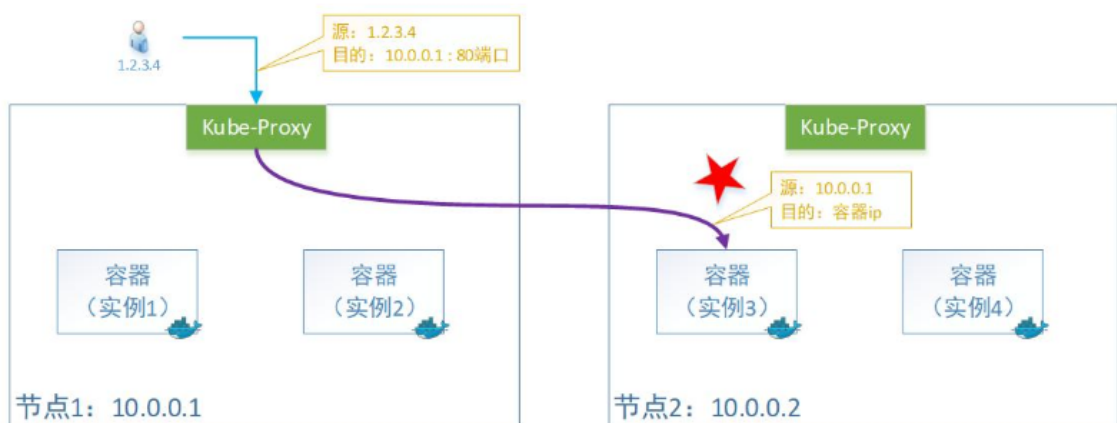
返回时需要从原路返回,可能会产生跨节点的跃点转发流量

此模式负载均衡效果好, 因为无论容器实例怎么分布在多个节点上, 它都会转发过去。

但是由于多了一次转发, 性能会损失

如果是NodePort类型,Pod无法获取外部客户端真实客户端IP

如果从集群内的Pod访问SVC,可以看到Pod源地址, 但本Pod访问SVC调度至自身时例外



- local 模式

集群外的请求报文从某节点的NodePort进入,该节点的Service 只会将请求流量调度到当前节点上的Pod

外部请求流量只发给本机的Pod, Kube-proxy转发时只会替换掉报文的目标IP,即只实现DNAT

例外: 当从Pod所在集群节点访问时, 也是通过FULLNAT来实现的

即: 容器收到的报文, 看到源IP地址还是用户的原有 IP

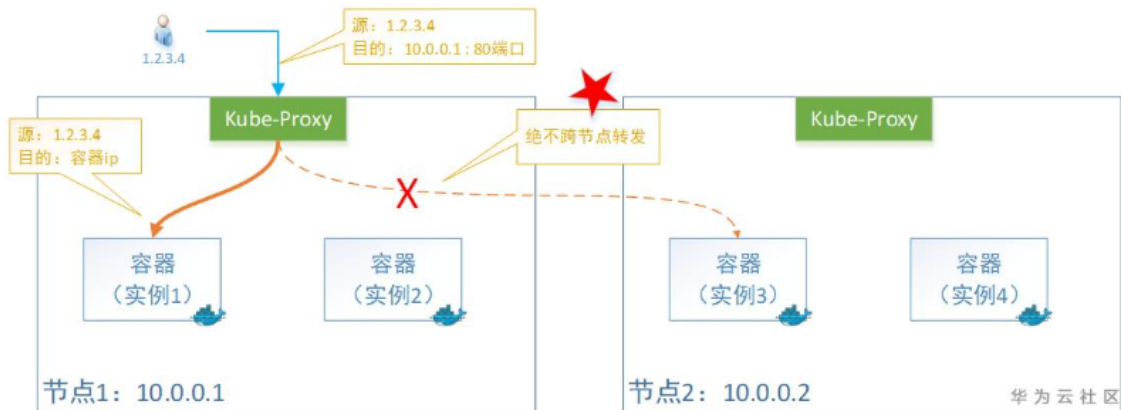
如果从集群内的Pod访问SVC,可以看到Pod源地址, 但本Pod访问SVC调度至自身时例外

此模式的负载均衡效果不是很好, 因为一旦容器实例分布在多个节点上, 它只转发给本机, 不产生跨节点的跃点转发流量。

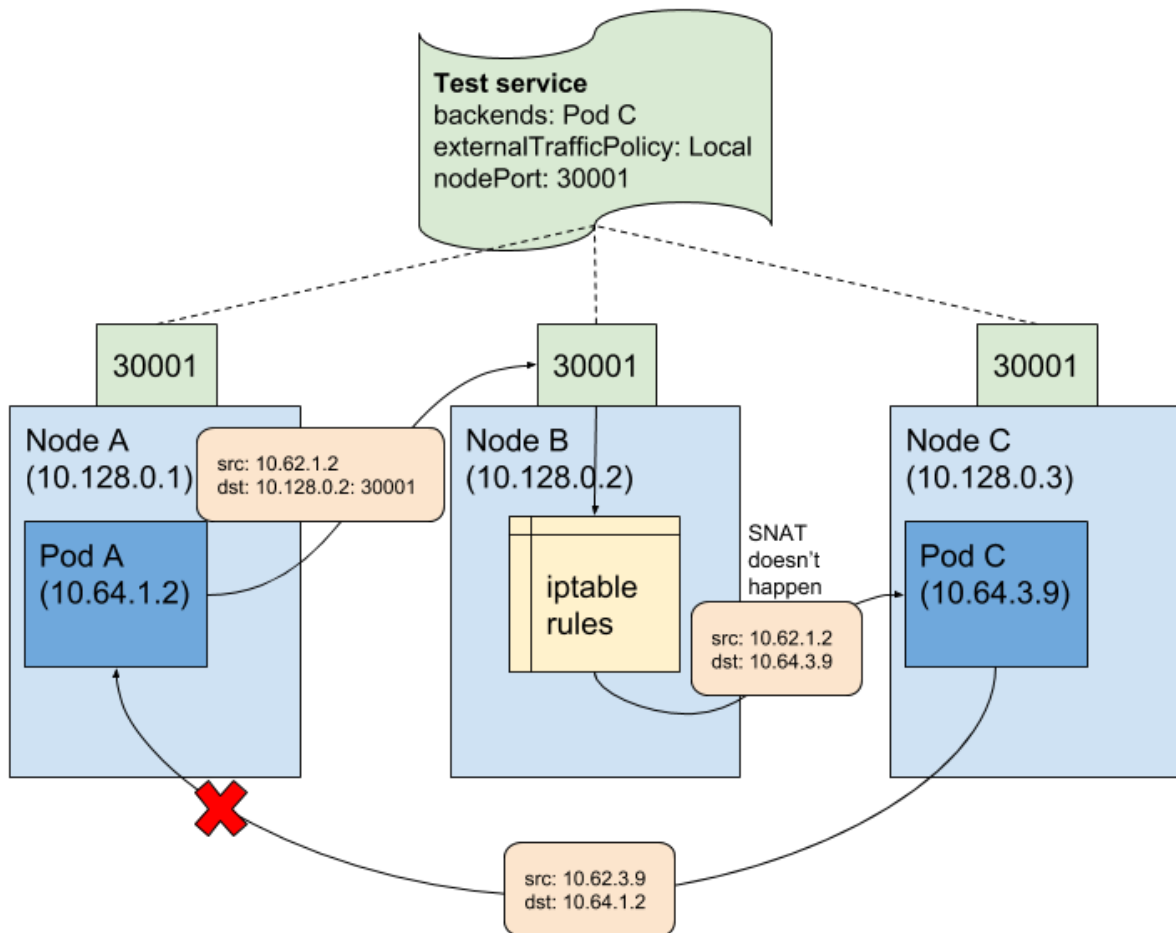
但是少了一次转发，性能会相对好

由于本机不会跨节点转发报文，所以要想对所有节点上的容器实现负载均衡，就需要借助外部的Loadbalancer来实现

因此使用Local 模式,一般会使用 LoadBalancer Service 类型结合 Loadbalancer 实现



如果使用Local模式，仍然转发至其它节点的Pod，因为是DNAT，会导致如下图示的通信异常



范例: nodePort 类型

```
[root@master1 ~]#cat service-nodeport.yaml
kind: Service
apiVersion: v1
metadata:
  name: service-nodeport
spec:
  type: NodePort    #指定service的类型,默认值为ClusterIP
```

#externalTrafficPolicy: Local #默认值为**Cluster**,如果是**Local**只能被当前运行Pod的节点处理流量.并且可以获取客户端真实IP

selector:

app: myweb

ports:

- name: http

protocol: TCP

port: 80

targetPort: 80

nodePort: 30066 #指定固定端口(30000-32767),使用NodePort类型且不指定nodeport,会自动分配随机端口向外暴露

#应用

[root@master1 ~]#kubectl apply -f service-nodeport.yaml

#查看有两个端口,集群内访问192.168.91.192:80,集群外访问任意节点的30066

[root@master1 ~]#kubectl get svc

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service-nodeport	NodePort	192.168.91.192	<none>	80:30066/TCP	9s

#访问service的地址

[root@master1 ~]#curl 192.168.91.192

kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-mvmrt, ServerIP: 172.16.3.106!

#默认externalTrafficPolicy: Cluster, 所以使用访问集群中任意一个物理节点的地址都可以

[root@master1 ~]#curl 10.0.0.101:30066

kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-mvmrt, ServerIP: 172.16.3.106!

[root@master1 ~]#curl 10.0.0.102:30066

kubernetes pod-test v0.1!! ClientIP: 172.16.4.0, ServerName: myweb-5df49fdd5-s2k5w, ServerIP: 172.16.2.93!

[root@master1 ~]#curl 10.0.0.106:30066

kubernetes pod-test v0.1!! ClientIP: 172.16.2.1, ServerName: myweb-5df49fdd5-s2k5w, ServerIP: 172.16.2.93!

#访问指定主机的指定端口,还会跳转到其他主机

externalTrafficPolicy <string> #外部流量策略处理方式, **Local**表示由当前节点处理, **Cluster**表示向集群范围调度,默认的外部流量调度策略是 **cluster**

#限制: **Local** 将外网访问的请求都转发到当前主机节点上,但是依赖于当前主机必须有对应的pod资源,否则会出现问题

#注意:如果当前节点有多个pod,仍然还会负载均衡

#从集群外访问,如果是externalTrafficPolicy为cluster,则Pod不能看到真实客户端IP

[root@master1 ~]#kubectl logs -f myweb-5d78b4dcdb-6rgv4

* Running on http://0.0.0.0:80/ (Press CTRL+C to quit)

10.244.1.0 - - [08/Apr/2023 03:55:57] "GET / HTTP/1.1" 200 -

10.244.2.1 - - [08/Apr/2023 03:56:42] "GET / HTTP/1.1" 200 -

#新版使用nft规则

[root@master1 ~]#nft list ruleset |grep 66

```
meta l4proto tcp ip daddr 127.0.0.0/8 tcp dport 30066 # nfacct-name
localhost_nps_accepted_pkts counter packets 0 bytes 0 jump KUBE-EXT-
75TCIBBYN24S2RET
```

```
meta l4proto tcp tcp dport 30066 counter packets 1 bytes 60 jump KUBE-
EXT-75TCIBBYN24S2RET
```

```
ct state new counter packets 90567 bytes 6601175 jump KUBE-PROXY-
FIREWALL
```

```
#查看生成的iptables规则，旧版
[root@master1 ~]#echo 'obase=16;30066'|bc
7572
[root@master1 ~]#iptables -vnL -t nat | grep -iE "30066|7572"

#环境清理
[root@master1 ~]#kubectl delete -f service-nodeport.yaml
```

```
[root@master1 ~]#kubectl get svc service-nodeport -o yaml | grep TrafficPolicy
externalTrafficPolicy: Cluster
internalTrafficPolicy: Cluster

#创建配置,指定流量调度策略是Local
[root@master1 ~]#cat service-nodeport-local.yaml
apiVersion: v1
kind: Service
metadata:
  name: service-nodeport-local
spec:
  type: NodePort
  externalTrafficPolicy: Local #指定Local
  ports:
    - port: 80
      nodePort: 30099
  selector:
    app: myweb

[root@master1 ~]#kubectl apply -f service-nodeport-local.yaml

[root@master1 ~]#kubectl get svc
NAME                                TYPE                CLUSTER-IP          EXTERNAL-IP          PORT(S)
kubernetes                         ClusterIP            192.168.0.1         <none>                443/TCP
22h
service-nodeport-local            NodePort             192.168.1.192       <none>                80:30099/TCP
6s

[root@master1 ~]#curl 10.0.0.101:30099
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-
mvmrt, ServerIP: 172.16.3.106!

[root@master1 ~]#kubectl get pod -o wide
NAME                                READY    STATUS    RESTARTS   AGE    IP                                NODE
NOMINATED NODE    READINESS GATES
myweb-5df49fdd5-cd8b4    1/1      Running    0          39m    172.16.1.56                      node2.wang.org    <none>          <none>
myweb-5df49fdd5-mvmrt    1/1      Running    0          39m    172.16.3.106                     node1.wang.org    <none>          <none>
myweb-5df49fdd5-s2k5w    1/1      Running    0          39m    172.16.2.93                      node3.wang.org    <none>          <none>
```



```
[root@master1 ~]#curl 10.0.0.106:30099
kubernetes pod-test v0.1!! ClientIP: 10.0.0.101, ServerName: myweb-5df49fdd5-s2k5w, ServerIP: 172.16.2.93!
[root@master1 ~]#curl 10.0.0.106:30099
kubernetes pod-test v0.1!! ClientIP: 10.0.0.101, ServerName: myweb-5df49fdd5-s2k5w, ServerIP: 172.16.2.93!
[root@master1 ~]#curl 10.0.0.106:30099
kubernetes pod-test v0.1!! ClientIP: 10.0.0.101, ServerName: myweb-5df49fdd5-s2k5w, ServerIP: 172.16.2.93!
```

#发现master2, 3节点无法访问

```
[root@master1 ~]#curl 10.0.0.102:30099
^C
[root@master1 ~]#curl 10.0.0.101:30099
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-s2k5w, ServerIP: 172.16.2.93!
[root@master1 ~]#curl 10.0.0.103:30099
^C
```

#访问其它节点,则

```
[root@master1 ~]#curl 10.0.0.104:30099
kubernetes pod-test v0.1!! ClientIP: 10.0.0.101, ServerName: myweb-5df49fdd5-mvmrt, ServerIP: 172.16.3.106!
[root@master1 ~]#curl 10.0.0.105:30099
kubernetes pod-test v0.1!! ClientIP: 10.0.0.101, ServerName: myweb-5df49fdd5-cd8b4, ServerIP: 172.16.1.56!
[root@master1 ~]#curl 10.0.0.106:30099
kubernetes pod-test v0.1!! ClientIP: 10.0.0.101, ServerName: myweb-5df49fdd5-s2k5w, ServerIP: 172.16.2.93!
```

#结果显示: 外部流量定向到当前主机, 而且不影响当前主机级别的负载均衡策略

#缩减Pod数

```
[root@master1 ~]#kubectl scale deployment myweb --replicas 1
```

#查看Pod所在的节点,此有Pod的节点可以访问,其它无Pod的节点,将无法访问

```
[root@master1 ~]#kubectl get pod -o wide
```

```
[root@master1 services]#kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP
		NOMINATED	myweb-5d775c669c-q6kq8	1/1	
Running	0	20m	10.244.1.121	node1.wang.org	
<none>	<none>				

```
C:\Users\wang>curl 10.0.0.104:30099
```

```
kubernetes pod-test v0.1!! ClientIP: 10.0.0.1, ServerName: myweb-5d775c669c-q6kq8, ServerIP: 10.244.1.121!
```

```
C:\Users\wang>curl 10.0.0.105:30099
```

```
^C
```

```
C:\Users\wang>curl 10.0.0.106:30099
```

```
^C
```

#从集群外访问, 如果是externalTrafficPolicy为Local, 则Pod可以看到真实客户端IP

```
C:\Users\lbt00>curl 10.0.0.104:30066
```

```
kubernetes pod-test v0.1!! ClientIP: 10.0.0.1, ServerName: myweb-5d78b4dcdbd-96r94, ServerIP: 10.244.1.85!
```

#访问日志显示真实客户端IP

```
[root@master1 ~]#kubectl logs -f myweb-5df49fdd5-mvmrt
```

```
* Running on http://0.0.0.0:80/ (Press CTRL+C to quit)
```

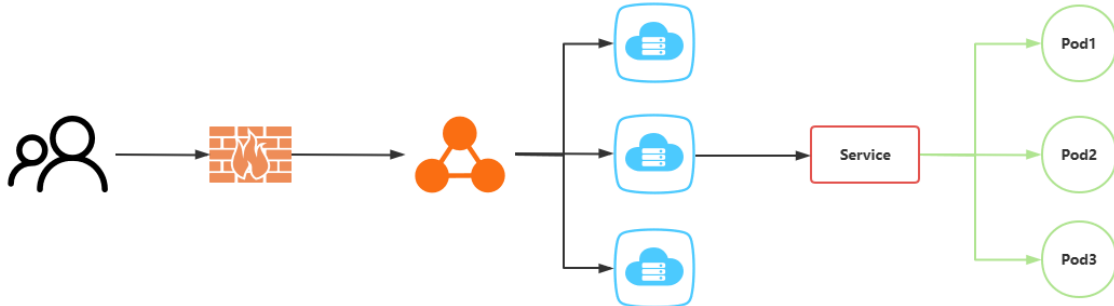


```
10.0.0.204 - - [08/Apr/2023 03:50:47] "GET / HTTP/1.1" 200 -  
10.0.0.1 - - [08/Apr/2023 03:51:44] "GET / HTTP/1.1" 200 -
```

#环境清理

```
[root@master1 ~]#kubectl delete -f service-nodeport-local.yaml
```

1.3.4 LoadBalancer Service 实现



只有Kubernetes集群是部署在一个LBaaS平台上且指供有一个集群外部的LB才适合使用LoadBalancer
一般的公有云都提供了此功能,但可能会有费用产生

如果在一个非公用云的普通的Kubernetes集群上, 创建了一个LoadBalancer类型的Service, 一般情况默认环境中是没有LBaaS的, 所以会导致由于找不到指定的服务, 状态会一直处于 Pending 状态

如果在私有云环境中使用 LoadBalancer Service, 可以使用云原生的开源项目实现负载均衡器, 比如 OpenELB, MetalLB 实现

Loadbalancer 可以获取用户访问的Service对应的Pod在哪个节点上,因此支持externaltrafficpolicy为 Local模式的流量转发

1.3.4.1 LoadBalancer 类型但没有 loadBalancerIP

范例: LoadBalancer 类型默认无法获取loadBalancerIP

#清单文件

```
[root@master1 ~]#cat service-loadbalancer.yaml
```

```
apiVersion: v1  
kind: Service  
metadata:  
  name: service-loadbalancer  
spec:  
  type: LoadBalancer  
  externalTrafficPolicy: Local  
  selector:  
    app: myweb  
  ports:  
    - name: http  
      protocol: TCP  
      port: 80  
      targetPort: 80
```

#loadBalancerIP: 6.6.6.6 #指定地址后, 还需要连接云服务商的LBaaS服务才能真正获得此地址, 否则为pending状态

```
[root@master1 ~]#kubectl apply -f service-loadbalancer.yaml
```

#结果显示: svc的外网负载EXTERNAL-IP一直在pending状态

```
[root@master1 ~]#kubectl get svc service-loadbalancer
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
service-loadbalancer	LoadBalancer	192.168.180.31	<pending>	80:31677/TCP

11s

#注意: LoadBalancer 本身是一个增强的NodePort类型的Service。

#注意: 虽然EXTERNAL-IP处理Pending状态, 但是CLUSTER-IP: 31677端口仍然可以访问

```
C:\Users\wang>curl 10.0.0.201:31677
kubernetes pod-test v0.1!! ClientIP: 10.244.2.0, ServerName: myweb-5d775c669c-q6kq8, ServerIP: 10.244.1.121!
```

```
C:\Users\wang>curl 10.0.0.202:31677
kubernetes pod-test v0.1!! ClientIP: 10.244.2.0, ServerName: myweb-5d775c669c-q6kq8, ServerIP: 10.244.1.121!
```

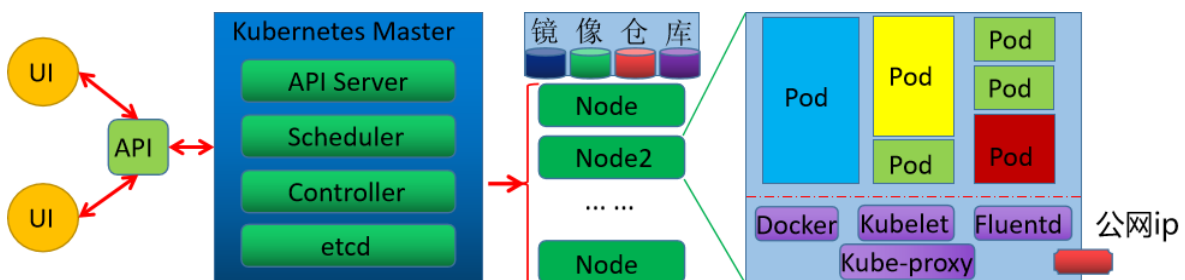
```
C:\Users\wang>curl 10.0.0.203:31677
kubernetes pod-test v0.1!! ClientIP: 10.244.2.0, ServerName: myweb-5d775c669c-q6kq8, ServerIP: 10.244.1.121!
```

#环境清理

```
[root@master1 ~]#kubectl delete -f service-loadbalancer.yaml
```

1.3.4.2 externalIPs

如果当前底层没有IaaS服务, 也没有LBaaS服务, 但是想直接通过众所周知的服务端口地址来进行访问, 可以通过 externalIPs 实现



当前的Kubernetes集群, 假如只有某个节点上绑定了一个对外的公网ip, 其他主机则没有, 那么我们可以指定此主机上的公网ip使用标准端口来接入外部流量。可以借助于 externalIPs 属性来为当前的 service指定公网ip地址即可

范例: LoadBalancer 类型同时指定externalIPs

```
#为局域网内的某台主机添加一个公网ip
[root@node3 ~]#ip a a 10.0.0.66/24 dev eth0 label eth0:1
#注意: 这里的模拟的外网ip, 必须被宿主机能够访问, 而且与Kubernetes的集群Pod网段不一样
```

```
#配置清单文件
[root@master1 ~]#cat service-loadbalancer-externalip.yml
apiVersion: v1
kind: Service
metadata:
  name: service-loadbalancer-externalip
spec:
  type: LoadBalancer
  ports:
    - port: 80
  selector:
```

```

    app: myweb
  externalIPs:
  - 10.0.0.66
#注意：这里的externalIPs对于使用哪种Service类型无关

#创建
[root@master1 ~]#kubectl apply -f service-loadbalancer-externalip.yml

#查看
[root@master1 ~]#kubectl get svc service-loadbalancer-externalip

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
service-loadbalancer-externalip	LoadBalancer	192.168.213.247	10.0.0.66

```

80:32267/TCP    9s
#结果显示：EXTERNAL-IP 已经指定了公网能够正常访问的ip地址

#使用externalIPs和标准端口即可访问
[root@master1 ~]#curl 10.0.0.66
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-
mvmrt, ServerIP: 172.16.3.106!
[root@master1 ~]#curl 10.0.0.66
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-
s2k5w, ServerIP: 172.16.2.93!
[root@master1 ~]#curl 10.0.0.66
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-
cd8b4, ServerIP: 172.16.1.56!
#结果显示：对于EXTERNAL-IP来说，可以直接通过该ip和容器端口来进行服务访问，而无需通过其他端口

#环境清理
[root@master1 ~]#kubectl delete -f service-loadbalancer-externalip.yml

```

1.3.4.3 OpenELB 实现 LBaaS 服务

1.3.4.3.1 OpenELB 介绍

OpenELB 是一个开源的云原生负载均衡器实现

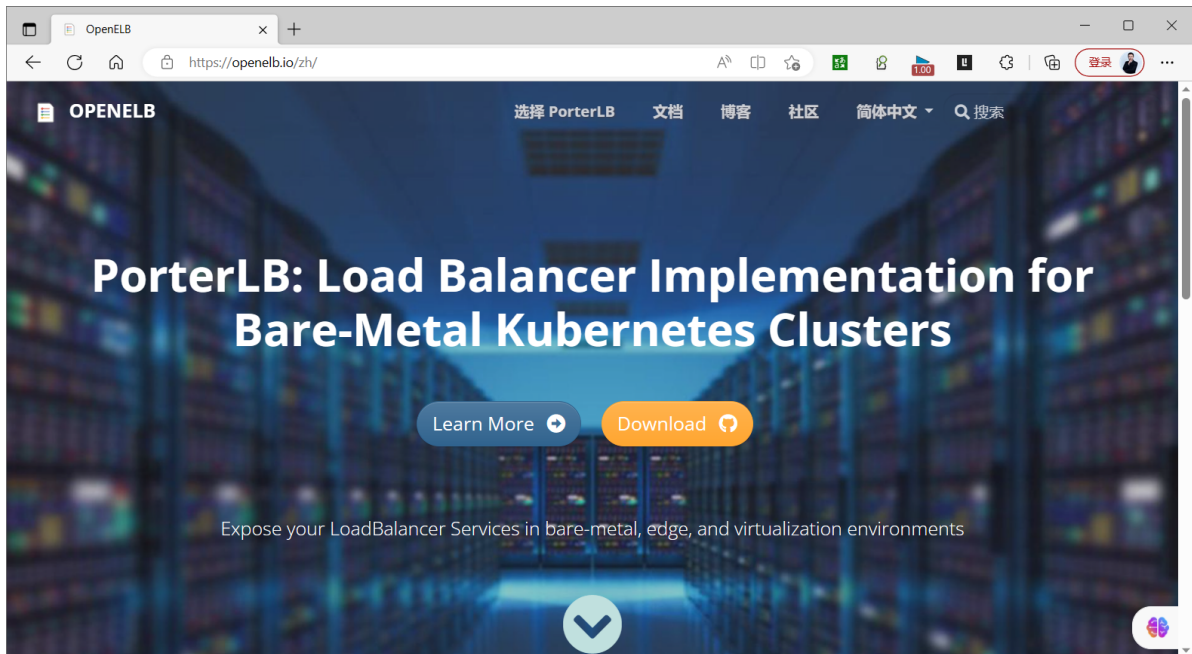
可以在基于裸金属服务器、边缘以及虚拟化的 Kubernetes 环境中使用 LoadBalancer 类型的 Service 对外暴露服务。

OpenELB 项目最初由国内青云的 KubeSphere 社区发起

目前已作为 CNCF 沙箱项目加入 CNCF 基金会，由 OpenELB 开源社区维护与支持。

官网:

<https://openlb.io/>
<https://github.com/openlb/openlb>



OpenELB 也拥有两种主要工作模式：Layer2 模式和 BGP 模式。OpenELB 的 BGP 模式目前暂不支持 IPv6。

无论是 Layer2 模式还是 BGP 模式，核心思路都是通过某种方式将特定 VIP 的流量引到 k8s 集群中，然后再通过 kube-proxy 将流量转发到后面的特定服务。

Layer2 模式需要 k8s 集群基础环境支持发送 anonymous ARP/NDP packets。

因为 OpenELB 是针对裸金属服务器设计的，因此如果是在云环境中部署，需要注意是否满足条件。

核心功能

- BGP模式和二层网络模式下的负载均衡
- ECMP路由和负载均衡
- IP地址池管理
- 基于CRD来管理BGP配置

注意: 此应用可能不支持 Openstack 等云环境

1.3.4.3.2 OpenELB 实现

安装说明

```
https://openelb.github.io/docs/getting-started/installation/install-openelb-on-kubernetes/
```

范例: 部署和使用 OpenELB

```
#使用kubectl部署OpenELB至Kubernetes集群
[root@master1 ~]#wget
https://raw.githubusercontent.com/openelb/openelb/master/deploy/openelb.yaml

#查看创建的资源类型
[root@master1 ~]#grep ^kind openelb.yaml
kind: Namespace
kind: CustomResourceDefinition
kind: CustomResourceDefinition
kind: CustomResourceDefinition
kind: ServiceAccount
kind: ServiceAccount
```

```
kind: Role
kind: Role
kind: ClusterRole
kind: ClusterRole
kind: ClusterRole
kind: RoleBinding
kind: RoleBinding
kind: ClusterRoleBinding
kind: ClusterRoleBinding
kind: ClusterRoleBinding
kind: Service
kind: Deployment
kind: Job
kind: Job
kind: MutatingWebhookConfiguration
kind: ValidatingWebhookConfiguration
```

#查看镜像地址,有些镜像国内可能无法下载,可以通过代理服务<https://dockerproxy.com/>下载镜像

```
[root@master1 ~]#grep image: openlb.yaml
    image: kubescape/openlb:v0.5.1
    image: registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.1.1
    image: registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.1.1
```

#镜像如果无法下载,修改镜像地址

```
[root@master1 ~]#vim openlb.yaml
#image: kubescape/openlb:v0.5.1
registry.cn-beijing.aliyuncs.com/wangxiaochun/kubescape/openlb:v0.5.1
#image: wangxiaochun/ingress-nginx-kube-webhook-certgen:v1.1.1
#image: registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.1.1
image: registry.cn-beijing.aliyuncs.com/wangxiaochun/ingress-nginx-kube-
webhook-certgen:v1.1.1
```

```
[root@master1 ~]#kubectl apply -f openlb.yaml
```

#查看创建的CRD自定义资源类型

```
[root@master1 ~]#kubectl get crd
NAME                                     CREATED AT
bgpconfs.network.kubescape.io          2022-11-23T01:57:23Z
bgppeers.network.kubescape.io          2022-11-23T01:57:23Z
eips.network.kubescape.io              2022-11-23T01:57:23Z
```

#确认openlb-manager Pod已经处于Running状态,且容器已经Ready。

```
[root@master1 ~]#kubectl get pods -n openlb-system
NAME                                READY   STATUS    RESTARTS   AGE
openlb-admission-create-p5h2p      0/1     Completed 0           5m33s
openlb-admission-patch-mxhm9        0/1     Completed 0           5m33s
openlb-keepalive-vip-5gmp2          1/1     Running   0           4m38s
openlb-keepalive-vip-bv8z9          1/1     Running   0           4m38s
openlb-keepalive-vip-s7vlp          1/1     Running   0           4m38s
openlb-manager-d6df4dfc4-tpwz9      1/1     Running   0           5m33s
```

#创建了一个Eip资源对象,它提供了一个地址池给LoadBalancer Service使用。

```
[root@master1 ~]#cat service-loadbalancer-eip-pool.yaml
apiVersion: network.kubescape.io/v1alpha2
kind: Eip
metadata:
```

```

name: eip-pool
annotations:
  eip.openelb.kubesphere.io/is-default-eip: "true" # 指定当前Eip作为向
LoadBalancer Server分配地址时使用默认的eip对象
spec:
  address: 10.0.0.10-10.0.0.50 #指定排除了主机节点之外的地址范围，可以使用单个IP或者带有掩
码长度的网络地址
  protocol: layer2                #指定OpenELB模式，支持bgp、layer2和vip三种，默认为bgp
  interface: eth0                 # OpenELB侦听ARP或NDP请求时使用的网络接口名称，仅layer2
模式下有效
  disable: false

```

#创建资源

```
[root@master1 ~]#kubectl apply -f service-loadbalancer-eip-pool.yaml
```

#查看结果

```
[root@master1 ~]#kubectl get eip
```

NAME	CIDR	USAGE	TOTAL
eip-pool	10.0.0.10-10.0.0.50		41

```
[root@master1 ~]#kubectl describe eip eip-pool
```

```

Name:          eip-pool
Namespace:
Labels:        <none>
Annotations:   eip.openelb.kubesphere.io/is-default-eip: true
API Version:   network.kubesphere.io/v1alpha2
Kind:          Eip
Metadata:
  Creation Timestamp:
  Finalizers:
    finalizer.ipam.kubesphere.io/v1alpha1
  Generation:      2
  Managed Fields:
    API Version:    network.kubesphere.io/v1alpha2
    Fields Type:    FieldsV1
    fieldsv1:
      f:metadata:
        f:annotations:
          .:
            f:eip.openelb.kubesphere.io/is-default-eip:
            f:kubectl.kubernetes.io/last-applied-configuration:
      f:spec:
        .:
          f:address:
          f:interface:
          f:protocol:
    Manager:        kubectl-client-side-apply
    Operation:      Update
    Time:
    API Version:    network.kubesphere.io/v1alpha2
    Fields Type:    FieldsV1
    fieldsv1:
      f:metadata:
        f:finalizers:
          .:
            v:"finalizer.ipam.kubesphere.io/v1alpha1":
    Manager:        openelb-manager
    Operation:      Update

```

```

Time:
API Version:  network.kubesphere.io/v1alpha2
Fields Type:  FieldsV1
fieldsv1:
  f:status:
    .:
    f:firstIP:
    f:lastIP:
    f:poolSize:
    f:ready:
    f:v4:
  Manager:      openelb-manager
  Operation:    Update
  Subresource:  status
  Time:
Resource Version: 1118656
UID:              c2da88da-e114-4203-8c96-e056b6bf79e5
Spec:
  Address:  10.0.0.10-10.0.0.50
  Interface: eth0
  Protocol:  layer2
Status:
  First IP:  10.0.0.10
  Last IP:   10.0.0.50
  Pool Size: 41
  Ready:     true
  v4:        true
Events:      <none>

```

```

[root@master1 ~]#kubectl get eip eip-pool -o yaml
apiVersion: network.kubesphere.io/v1alpha2
kind: Eip
metadata:
  annotations:
    eip.openelb.kubesphere.io/is-default-eip: "true"
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"network.kubesphere.io/v1alpha2","kind":"Eip","metadata":
{"annotations":{"eip.openelb.kubesphere.io/is-default-eip":"true"},"name":"eip-
pool"},"spec":{"address":"10.0.0.50-
10.0.0.60","disable":false,"interface":"eth0","protocol":"layer2"}}
    creationTimestamp: ""
  finalizers:
  - finalizer.ipam.kubesphere.io/v1alpha1
  generation: 2
  name: eip-pool
  resourceVersion: "986554"
  uid: 590addaa-9a57-4d85-a066-f0c263d5fb50
spec:
  address: 10.0.0.10-10.0.0.50
  interface: eth0
  protocol: layer2
status:
  firstIP: 10.0.0.10
  lastIP: 10.0.0.50
  poolSize: 11
  ready: true
  v4: true

```

#创建Deployment和LoadBalancer类型的Service，测试地址池是否能给Service分配LoadBalancer IP

```
[root@master1 ~]#kubectl create deployment myapp --image=registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1 --replicas=3
```

#方法1

#清单文件

```
[root@master1 ~]#cat service-loadbalancer-lbaas.yaml
```

```
apiVersion: v1
kind: Service
metadata:
  name: service-loadbalancer-lbaas
spec:
  type: LoadBalancer
  externalTrafficPolicy: Local
  selector:
    app: myapp
  ports:
  - name: http
    protocol: TCP
    port: 80
    targetPort: 80
```

```
[root@master1 ~]#kubectl apply -f service-loadbalancer-lbaas.yaml
```

#方法2

```
[root@master1 ~]#kubectl create service loadbalancer myapp --tcp=80:80
```

#查看service资源对象myapp是否自动获得了External IP。

```
[root@master1 ~]#kubectl get svc myapp
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
myapp	LoadBalancer	10.110.50.151	10.0.0.10	80:31491/TCP	36s

```
[root@master1 ~]#kubectl get svc myapp -o yaml
```

```
apiVersion: v1
kind: Service
metadata:
  annotations:
    eip.openlb.kubesphere.io/v1alpha2: eip-pool
    layer2.openlb.kubesphere.io/v1alpha1: master1.wang.org
    lb.kubesphere.io/v1alpha1: openlb
    protocol.openlb.kubesphere.io/v1alpha1: layer2
  creationTimestamp: ""
  finalizers:
  - finalizer.lb.kubesphere.io/v1alpha1
  labels:
    app: myapp
    eip.openlb.kubesphere.io/v1alpha2: eip-pool
  name: myapp
  namespace: default
  resourceVersion: "986749"
  uid: 347c510c-84c0-4d97-b766-aa874cce8cb2
spec:
  allocateLoadBalancerNodePorts: true
  clusterIP: 10.97.180.0
  clusterIPs:
  - 10.97.180.0
  externalTrafficPolicy: Cluster
```



```

internalTrafficPolicy: Cluster
ipFamilies:
- IPv4
ipFamilyPolicy: SingleStack
ports:
- name: 80-80
  nodePort: 31464
  port: 80
  protocol: TCP
  targetPort: 80
selector:
  app: myapp
sessionAffinity: None
type: LoadBalancer
status:
  loadBalancer:
    ingress:
    - ip: 10.0.0.10

```

#查看Service对应后端Pod的地址

```
[root@master1 ~]#kubectl get ep
```

NAME	ENDPOINTS	AGE
kubernetes	10.0.0.200:6443	76d
myapp	10.244.1.79:80,10.244.2.85:80,10.244.3.75:80	3m28s

#从集群外部的客户端上通过Service对应EXTERNAL-IP地址10.0.0.10对myapp服务发起访问

```
C:\Users\wang>curl 10.0.0.10
```

```
kubernetes pod-test v0.1!! ClientIP: 10.244.0.0, ServerName: myapp-7dbd45d496-qffnr, ServerIP: 10.244.1.79!
```

```
C:\Users\wang>curl 10.0.0.10
```

```
kubernetes pod-test v0.1!! ClientIP: 10.244.0.0, ServerName: myapp-7dbd45d496-f7fbk, ServerIP: 10.244.2.85!
```

```
C:\Users\wang>curl 10.0.0.10
```

```
kubernetes pod-test v0.1!! ClientIP: 10.244.0.0, ServerName: myapp-7dbd45d496-vmdb2, ServerIP: 10.244.3.75!
```

#缩减Pod

```
[root@master1 ~]#kubectl scale deployment myapp --replicas 1
```

```
[root@master1 ~]#kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP
myapp-7dbd45d496-kvvjk	1/1	Running	0	6m12s	10.244.3.96
node3.wang.org	<none>	<none>			

#所有节点的NodePort都可以访问

```
C:\Windows\System32>curl 10.0.0.201:31491
```

```
kubernetes pod-test v0.1!! ClientIP: 10.0.0.1, ServerName: myapp-7dbd45d496-kvvjk, ServerIP: 10.244.3.96!
```

```
C:\Windows\System32>curl 10.0.0.202:31491
```

```
kubernetes pod-test v0.1!! ClientIP: 10.0.0.1, ServerName: myapp-7dbd45d496-kvvjk, ServerIP: 10.244.3.96!
```

```
C:\Windows\System32>curl 10.0.0.203:31491
```

```
kubernetes pod-test v0.1!! ClientIP: 10.0.0.1, ServerName: myapp-7dbd45d496-kvvjk, ServerIP: 10.244.3.96!
```

```
#修改配置externalTrafficPolicy为Local
[root@master1 ~]#kubectl edit svc service-loadbalancer
externalTrafficPolicy: Local

#集群内所有节点访问都可以
[root@master1 ~]#curl 10.0.0.10

#集群外访问失败
C:\Users\lbt00>curl 10.0.0.10
curl: (28) Failed to connect to 10.0.0.10 port 80 after 21043 ms: Couldn't
connect to server
```

1.3.4.4 MetalLB 实现 LBaaS 服务

1.3.4.4.1 MetalLB 介绍



<https://github.com/metallb/metallb>
<https://metallb.universe.tf/>

A network load-balancer implementation for Kubernetes using standard routing protocols

MetalLB 是由 Google 开源提供, 当前属于 CNCF 的 sandbox 项目

MetalLB 功能实现依赖于两种机制

- Address Allocation 地址分配: 基于用户配置的地址池, 为用户创建的LoadBalancer分配IP地址, 并配置在节点上
- External Announcement 对外公告: 让集群外部的网络了解新分配的IP地址, MetalLB使用ARP、NDP或BGP实现

MetalLB 可配置为在二层模式或BGP模式下运行

- 二层模式(ARP/NDP)

LoadBalancer IP地址配置在某一个节点上, 并使用ARP(IPv4)或NDP(IPv6)对外公告

拥有LoadBalancer IP 地址的节点将成为Service流量的惟一入口, 并在节点故障时自动进行故障转移

并未真正实现负载均衡, 存在性能瓶颈, 且故障转移存在秒级的延迟

- BGP模式

集群中的所有节点与本地网络中的BGP Router建立BGP对等会话, 通告LoadBalancer IP, 从而告知Router如何进行流量

路由

可以实现跨多个节点的真实意义上的负载均衡

注意: 此应用可能不支持 Openstack 等云环境

1.3.4.4.2 MetalLB 实现

安装说明

```
https://metallb.universe.tf/installation/  
https://metallb.io/installation/
```

- 部署MetalLB

kube-proxy工作于ipvs模式时，必须要使用严格ARP（StrictARP）模式，因此，若有必要，先运行如下命令，配置kube-proxy。

```
kubectl get configmap kube-proxy -n kube-system -o yaml | \  
sed -e "s/strictARP: false/strictARP: true/" | \  
kubectl apply -f - -n kube-system
```

部署MetalLB至Kubernetes集群

```
METALLB_VERSION='v0.14.9'  
METALLB_VERSION='v0.14.4'  
kubectl apply -f  
https://raw.githubusercontent.com/metallb/metallb/${METALLB_VERSION}/config/manifests/metallb-native.yaml
```

- 创建地址池

注意: IPAddressPool 必须使用Kubernetes集群节点的IP地址段

```
apiVersion: metallb.io/v1beta1  
kind: IPAddressPool  
metadata:  
  name: localip-pool  
  namespace: metallb-system  
spec:  
  addresses:  
  - 10.0.0.10-10.0.0.50  
  autoAssign: true  
  avoidBuggyIPs: true
```

- 创建二层公告机制

```
apiVersion: metallb.io/v1beta1  
kind: L2Advertisement  
metadata:  
  name: localip-pool-l2a  
  namespace: metallb-system  
spec:  
  ipAddressPools:  
  - localip-pool  
  interfaces:  
  - eth0 #用于发送免费ARP公告
```

- 创建 Service 和 Deployment

创建Deployment和LoadBalancer类型的Service，测试地址池是否能给Service分配LoadBalancer IP

范例：

```
#下载最新版
[root@master1 ~]#wget
https://raw.githubusercontent.com/metallb/metallb/main/config/manifests/metallb-
native.yaml
#加速
[root@master1 ~]#wget
https://mirror.ghproxy.com/https://raw.githubusercontent.com/metallb/metallb/main/config/manifests/metallb-native.yaml

#指定版本
[root@master1 ~]#METALLB_VERSION='v0.14.9'
[root@master1 ~]#METALLB_VERSION='v0.14.8'
[root@master1 ~]#METALLB_VERSION='v0.14.4'
[root@master1 ~]#kubectl apply -f
https://raw.githubusercontent.com/metallb/metallb/${METALLB_VERSION}/config/manifests/metallb-native.yaml
namespace/metallb-system created
customresourcedefinition.apiextensions.k8s.io/bfdprofiles.metallb.io created
customresourcedefinition.apiextensions.k8s.io/bgpadvertisements.metallb.io
created
customresourcedefinition.apiextensions.k8s.io/bgppeers.metallb.io created
customresourcedefinition.apiextensions.k8s.io/communities.metallb.io created
customresourcedefinition.apiextensions.k8s.io/ipaddresspools.metallb.io created
customresourcedefinition.apiextensions.k8s.io/l2advertisements.metallb.io
created
customresourcedefinition.apiextensions.k8s.io/service12statuses.metallb.io
created
serviceaccount/controller created
serviceaccount/speaker created
role.rbac.authorization.k8s.io/controller created
role.rbac.authorization.k8s.io/pod-lister created
clusterrole.rbac.authorization.k8s.io/metallb-system:controller created
clusterrole.rbac.authorization.k8s.io/metallb-system:speaker created
rolebinding.rbac.authorization.k8s.io/controller created
rolebinding.rbac.authorization.k8s.io/pod-lister created
clusterrolebinding.rbac.authorization.k8s.io/metallb-system:controller created
clusterrolebinding.rbac.authorization.k8s.io/metallb-system:speaker created
configmap/metallb-exclude12 created
secret/metallb-webhook-cert created
service/metallb-webhook-service created
deployment.apps/controller created
daemonset.apps/speaker created
validatingwebhookconfiguration.admissionregistration.k8s.io/metallb-webhook-
configuration created

#创建CRD
[root@master1 ~]#kubectl get crd
NAME                                     CREATED AT
bfdprofiles.metallb.io                 2024-04-18T12:25:37Z
bgpadvertisements.metallb.io          2024-04-18T12:25:37Z
bgppeers.metallb.io                  2024-04-18T12:25:37Z
communities.metallb.io               2024-04-18T12:25:37Z
```

```
ipaddresspools.metallb.io      2024-04-18T12:25:37Z
l2advertisements.metallb.io    2024-04-18T12:25:37Z
service12statuses.metallb.io   2024-04-18T12:25:37Z
```

#指定IP地址池

```
[root@master1 ~]#cat service-metallb-IPAddressPool.yaml
```

```
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: localip-pool
  namespace: metallb-system
spec:
  addresses:
  - 10.0.0.10-10.0.0.50
  autoAssign: true
  avoidBuggyIPs: true
```

```
[root@master1 ~]#cat service-metallb-L2Advertisement.yaml
```

```
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: localip-pool-l2a
  namespace: metallb-system
spec:
  ipAddressPools:
  - localip-pool
  interfaces:
  - eth0
```

```
[root@master1 ~]#kubectl apply -f service-metallb-IPAddressPool.yaml -f service-metallb-L2Advertisement.yaml
```

```
[root@master1 ~]#kubectl get all -n metallb-system
```

NAME	READY	STATUS	RESTARTS	AGE
pod/controller-5bcd8f48c-6t8zr	1/1	Running	0	6m59s
pod/speaker-2slfb	1/1	Running	0	6m59s
pod/speaker-7q5zd	1/1	Running	0	6m59s
pod/speaker-fzvdk	1/1	Running	0	11s
pod/speaker-k7tnm	1/1	Running	0	6m59s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
service/metallb-webhook-service	ClusterIP	10.104.137.39	<none>

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE
daemonset.apps/speaker	4	4	4	4	4
kubernetes.io/os=linux	6m59s				

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/controller	1/1	1	1	6m59s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/controller-5bcd8f48c	1	1	1	6m59s

```
[root@master1 ~]#kubectl get IPAddressPool -n metallb-system
```

NAME	AUTO ASSIGN	AVOID BUGGY IPS	ADDRESSES
------	-------------	-----------------	-----------

```
localip-pool    true          true          ["10.0.0.10-10.0.0.50"]
```

#创建Deployment和LoadBalancer类型的Service, 测试地址池是否能给Service分配LoadBalancer IP

```
[root@master1 ~]#kubectl create deployment myapp --image=registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1 --replicas=3
```

```
[root@master1 ~]#cat service-loadbalancer-lbaas.yaml
```

```
apiVersion: v1
kind: Service
metadata:
  name: service-loadbalancer-lbaas
spec:
  type: LoadBalancer
  externalTrafficPolicy: Local
  selector:
    app: myapp
  ports:
  - name: http
    protocol: TCP
    port: 80
    targetPort: 80
```

```
[root@master1 ~]#kubectl apply -f service-loadbalancer-lbaas.yaml
```

```
[root@master1 ~]#kubectl get ep
```

NAME	ENDPOINTS	AGE
service-loadbalancer-lbaas	10.244.1.43:80,10.244.2.41:80,10.244.3.31:80	6m8s

#查看到分配了外部IP: 10.0.0.11

```
[root@master1 ~]#kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
service-loadbalancer-lbaas	LoadBalancer	10.108.1.1	10.0.0.11
80:30729/TCP			

#生成iptables规则

```
[root@master1 ~]#iptables -vnL -t nat |grep 30729
```

```
    0      0 KUBE-EXT-RAUC225Z5NF5JX4F  tcp  --  *      *      0.0.0.0/0
    0.0.0.0/0                  /* default/service-loadbalancer-lbaas:http */ tcp
dpt:30729
```

#从集群外可以访问

```
C:\Users\lbt00>curl 10.0.0.11
```

```
kubernetes pod-test v0.1!! ClientIP: 10.0.0.1, ServerName: myapp-7dbd45d496-8d6nx, ServerIP: 10.244.3.31!
```

```
C:\Users\lbt00>curl 10.0.0.11
```

```
kubernetes pod-test v0.1!! ClientIP: 10.0.0.1, ServerName: myapp-7dbd45d496-8d6nx, ServerIP: 10.244.3.31!
```

```
C:\Users\lbt00>curl 10.0.0.11
```

```
kubernetes pod-test v0.1!! ClientIP: 10.0.0.1, ServerName: myapp-7dbd45d496-8d6nx, ServerIP: 10.244.3.31!
```

1.3.5 ExternalName Service 实现

Service 不仅可以实现Kubernetes集群内Pod应用之间的相互访问以及从集群外部访问集群中的Pod

ExternalName Service 可以支持做为外部服务的代理实现集群中Pod访问集群外的服务或者另一个集群内的Service

因此也可以称为其它服务的“别名”服务



Service代理Kubernetes外部应用的使用场景

- 在生产环境中Pod 希望使用某个固定的名称而非IP地址进行访问集群外部的服务
- 使用Service指向另一个Namespace中或其它Kubernetes集群中的服务
- 某个项目正在迁移至Kubernetes集群，但是一部分服务仍然在集群外部，此时可以使用service代理至k8s集群外部的服务

范例：使用 ExternalName Service 实现代理外部服务

```
#创建文件
[root@master1 ~]#cat service-externalname-web.yaml
kind: Service
apiVersion: v1
metadata:
  name: svc-externalname-web
  namespace: default
spec:
  type: ExternalName
  externalName: www.wangxiaochun.com  #外部服务的FQDN, 不支持IP
  #externalName: myapp.demo.svc.cluster.local  #也可以是另一个Service的名称
  ports:
    #以下行都可选
    - protocol: TCP
      port: 80
      targetPort: 80  #外部服务端口
      nodePort: 0
  selector: {}  #没有标签选择器，表示不关联任何Pod

#应用
[root@master1 ~]#kubectl apply -f service-externalname-web.yaml

#注意：service没有Cluster-IP,即为无头服务Headless Service
[root@master1 ~]#kubectl get svc
NAME                                TYPE                CLUSTER-IP  EXTERNAL-IP  PORT(S)  AGE
svc-externalname-web               ExternalName        <none>      www.wangxiaochun.com  80/TCP   10s

#注意：ExternalName没有对应同名的Endpoints
[root@master1 ~]#kubectl get endpoints
NAME            ENDPOINTS                     AGE
kubernetes     10.0.0.200:6443             34d

#解析
[root@master1 ~]#kubectl run pod-$RANDOM --image=registry.cn-
beijing.aliyuncs.com/wangxiaochun/admin-box:v0.1 -it --rm --command -- /bin/bash
root@pod-12181 /# host svc-externalname-web
svc-externalname-web.default.svc.cluster.local is an alias for
www.wangxiaochun.com.
```

```

www.wangxiaochun.com has address 123.56.174.200

root@pod-12181 /# nslookup svc-externalname-web
Server:      10.96.0.10
Address:     10.96.0.10#53

svc-externalname-web.default.svc.cluster.local canonical name =
www.wangxiaochun.com.
Name:   www.wangxiaochun.com
Address: 123.56.174.200

root@pod-12181 /# ping -c1 svc-externalname-web
PING svc-externalname-web (123.56.174.200): 56 data bytes
64 bytes from 123.56.174.200: seq=0 ttl=127 time=11.929 ms

--- svc-externalname-web ping statistics ---
1 packets transmitted, 1 packets received, 0% packet loss
round-trip min/avg/max = 11.929/11.929/11.929 ms

/ # ping -c1 svc-externalname-web
PING svc-externalname-web (123.56.174.200): 56 data bytes
64 bytes from 123.56.174.200: seq=0 ttl=127 time=10.082 ms

--- svc-externalname-web ping statistics ---
1 packets transmitted, 1 packets received, 0% packet loss
round-trip min/avg/max = 10.082/10.082/10.082 ms

root@pod-12181 /# curl svc-externalname-web
<html>
<head><title>302 Found</title></head>
<body>
<center><h1>302 Found</h1></center>
<hr><center>nginx</center>
</body>
</html>

#环境清理
[root@master1 ~]#kubectl delete -f service-externalname-web.yaml

```

范例: 使用自建的Endpoint实现基于ClusterIP类型的Service代理集群外部服务

- 手动创建一个Endpoints资源对象，直接把外部端点的IP地址，放入可用地址列表
- 额外创建一个不带selector的同名的Service对象

```

#在K8s集群外安装redis
[root@master1 ~]#apt update && apt -y install redis
[root@master1 ~]#vim /etc/redis/redis.conf
bind 0.0.0.0
requirepass 123456 #因为保护模式，需要设置密码

[root@master1 ~]#systemctl restart redis

[root@master1 ~]#cat service-endpoints.yaml
---
kind: Endpoints
apiVersion: v1

```



```

metadata:
  name: service-redis    #和下面的service必须同名
  namespace: default
subsets:
  - addresses:
    - ip: 10.0.0.101    #外部服务的FQDN或IP
      #- ip: 10.0.0.102    #可以有多个外部主机
    ports:
    - name: redis
      port: 6379
      protocol: TCP
---
```

```

apiVersion: v1
kind: Service
metadata:
  name: service-redis    #和上面的endpoints必须同名
  namespace: default
spec:
  type: ClusterIP        #注意: 没有selector
  #clusterIP: "None"    #可以指定为无头服务
  ports:
  - name: redis
    port: 6379
    protocol: TCP
    targetPort: 6379
```

```
[root@master1 ~]#kubectl apply -f service-endpoints.yaml
```

```
[root@master1 ~]#kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	46d
service-redis	ClusterIP	10.103.7.0	<none>	6379/TCP	45m

```
[root@master1 ~]#kubectl get ep
```

NAME	AGE	ENDPOINTS
service-redis	46m	10.0.0.101:6379

```
[root@master1 ~]#kubectl describe svc service-redis
```

```

Name:          service-redis
Namespace:     default
Labels:        <none>
Annotations:   <none>
Selector:      <none>
Type:          ClusterIP
IP Family Policy: SingleStack
IP Families:   IPv4
IP:            10.103.7.0
IPs:           10.103.7.0
Port:          redis 6379/TCP
TargetPort:    6379/TCP
Endpoints:     10.0.0.101:6379
Session Affinity: None
Events:        <none>
```

```
#测试访问
```

```

[root@master1 ~]#kubectl run pod-$RANDOM --image=registry.cn-
beijing.aliyuncs.com/wangxiaochun/admin-box:v0.1 -it --rm --command -- /bin/bash
If you don't see a command prompt, try pressing enter.
root@pod-4372 /# host service-redis           #将service解析为Cluster-IP
service-redis.default.svc.cluster.local has address 10.103.7.0
root@pod-4372 /# nc service-redis 6379       #访问外部服务
auth 123456
info
$3251
# Server
redis_version:5.0.7
redis_git_sha1:00000000
redis_git_dirty:0
redis_build_id:66bd629f924ac924
redis_mode:standalone
os:Linux 5.4.0-131-generic x86_64
.....

#环境清理
[root@master1 ~]#kubectl delete -f service-endpoints.yaml

```

范例：使用ClusterIP Service基于Headless Service 的自建的Endpoint代理外部服务

```

[root@master1 ~]#cat service-endpoints-headless.yaml
kind: Endpoints
apiVersion: v1
metadata:
  name: service-redis
  namespace: default
subsets:
  - addresses:
    - ip: 10.0.0.101
    ports:
    - name: redis
      port: 6379
      protocol: TCP
---
apiVersion: v1
kind: Service
metadata:
  name: service-redis
  namespace: default
spec:
  type: ClusterIP
  clusterIP: "None"           #添加此行，实现无头服务Headless Service 实现
  ports:
    - name: redis
      port: 6379
      protocol: TCP
      targetPort: 6379

[root@master1 ~]#kubectl apply -f service-endpoints-headless.yaml

#注意:CLUSTER-IP为None
[root@master1 ~]#kubectl get svc

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
AGE				
kubernetes	ClusterIP	10.96.0.1	<none>	
443/TCP 34d				
service-redis	ClusterIP	None	<none>	
6379/TCP 7s				

```
[root@master1 ~]#kubectl describe svc service-redis
```

```
Name:          service-redis
Namespace:     default
Labels:        <none>
Annotations:   <none>
Selector:      <none>
Type:          ClusterIP
IP Family Policy: RequireDualStack
IP Families:   IPv4,IPv6
IP:            None
IPs:           None
Port:          redis 6379/TCP
TargetPort:    6379/TCP
Endpoints:     10.0.0.101:6379
Session Affinity: None
Events:        <none>
```

```
[root@master1 ~]#kubectl get endpoints
```

NAME	ENDPOINTS	AGE
kubernetes	10.0.0.200:6443	34d
service-redis	10.0.0.101:6379	15s

#测试访问

```
[root@master1 ~]#kubectl run pod-$RANDOM --image=registry.cn-
```

```
beijing.aliyuncs.com/wangxiaochun/admin-box:v0.1 -it --rm --command -- /bin/bash
```

If you don't see a command prompt, try pressing enter.

```
root@pod-4321 /# host service-redis
```

```
service-redis.default.svc.cluster.local has address 10.0.0.101
```

```
root@pod-4321 /# nc service-redis 6379
```

```
info
```

```
$3253
```

Server

```
redis_version:5.0.7
```

```
redis_git_sha1:00000000
```

```
redis_git_dirty:0
```

```
redis_build_id:66bd629f924ac924
```

```
redis_mode:standalone
```

```
os:Linux 5.4.0-131-generic x86_64
```

```
arch_bits:64
```

```
....
```

#环境清理

```
[root@master1 ~]#kubectl delete -f service-endpoints-headless.yaml
```

1.3.6 会话粘滞

kubernetes的Service默认是按照轮询机制进行转发至后端的多个pod

如果用户请求有一定的会话要求,即希望同一个客户每次总是能访问同一个pod的时候,可以使用service的affinity机制来实现

它可将同一个客户端的请求始终转发至同一个后端Pod对象,它是由kube-proxy的ipvs机制来实现的。

默认情况下, service所提供的会话粘性效果默认在10800s(3小时)后会重新调度,而且仅能基于客户端IP进行识别,调度粒度较粗

属性解析

```
#属性信息
service.spec.sessionAffinity, #定义粘性会话的类型, 可为 None 和 ClientIP, 默认值为None即不开启会话粘滞
service.spec.sessionAffinityConfig.clientIP.timeoutSeconds #配置会话保持时长, 默认10800s 范围 1-86400

#配置格式
sessionAffinity: ClientIP
sessionAffinityConfig:
  clientIP:
    timeoutSeconds: 10800
```

范例: 默认没有开启会话粘滞

```
#准备效果
[root@master1 ~]#kubectl create deployment myweb --image=registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1 --replicas=3

[root@master1 ~]#cat service-test.yaml
kind: Service
apiVersion: v1
metadata:
  name: service-test
spec:
  selector:
    app: myweb
  ports:
  - name: http
    protocol: TCP
    port: 80
    targetPort: 80

[root@master1 ~]#kubectl apply -f service-test.yaml

#默认效果
[root@master1 ~]#kubectl get svc service-test
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP  PORT(S)    AGE
service-test        ClusterIP   192.168.193.236 <none>       80/TCP     23h

[root@master1 ~]#kubectl get pod
NAME                READY    STATUS    RESTARTS   AGE
myweb-5df49fdd5-brxcp 1/1      Running   1 (84m ago) 23h
myweb-5df49fdd5-xnrt4 1/1      Running   1 (84m ago) 23h
```

```
myweb-5df49fdd5-z7d67 1/1 Running 1 (84m ago) 23h
```

#结果显示: 每次访问的都不一样

```
[root@master1 ~]#for i in {1..3};do curl 192.168.193.236;done
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-
brxcp, ServerIP: 172.16.1.59!
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-
xnrt4, ServerIP: 172.16.3.108!
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-
z7d67, ServerIP: 172.16.2.95!
```

范例: 实现会话粘滞

#修改service对象的会话配置

```
[root@master1 ~]#cat service-session.yaml
apiVersion: v1
kind: Service
metadata:
  name: service-session
spec:
  type: LoadBalancer
  ports:
  - port: 80
  selector:
    app: myweb
  sessionAffinity: ClientIP #客户端IP的亲缘性,实现1800s内的会话保持
  sessionAffinityConfig:
    clientIP:
      timeoutSeconds: 1800 #默认值为10800,即3小时
```

#应用资源效果

```
[root@master1 ~]#kubectl apply -f service-session.yaml
service/service-session created
[root@master1 ~]#kubectl get svc service-session
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
AGE				
service-session	LoadBalancer	192.168.122.22	<pending>	80:30380/TCP
8s				

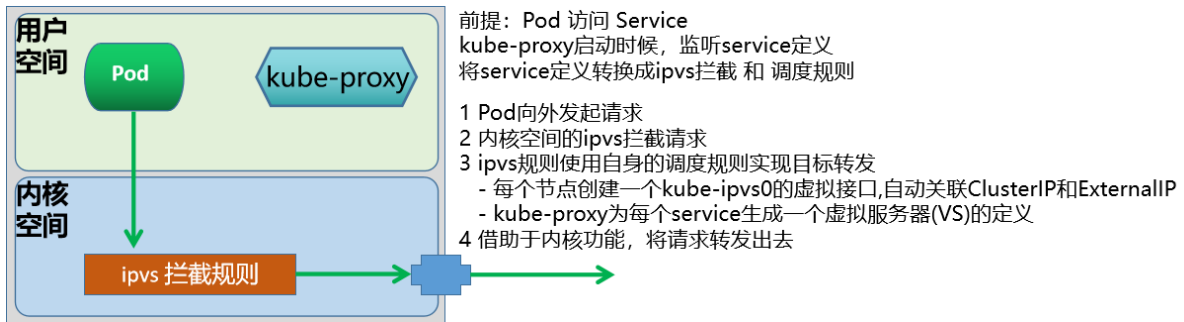
#查看效果,每次访问是同一个pod

```
[root@master1 ~]#for i in {1..5};do curl 192.168.122.22;done
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-
xnrt4, ServerIP: 172.16.3.108!
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-
xnrt4, ServerIP: 172.16.3.108!
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-
xnrt4, ServerIP: 172.16.3.108!
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-
xnrt4, ServerIP: 172.16.3.108!
kubernetes pod-test v0.1!! ClientIP: 172.16.0.0, ServerName: myweb-5df49fdd5-
xnrt4, ServerIP: 172.16.3.108!
```

#清理环境

```
[root@master1 ~]#kubectl delete -f service-session.yaml
```

1.3.7 IPVS 模式



ipvs会在每个节点上创建一个名为kube-ipvs0的虚拟接口, 并将集群所有Service对象的ClusterIP和ExternalIP都配置在该接口; 所以每增加一个ClusterIP 或者ExternalIP, 就相当于为 kube-ipvs0 关联了一个地址罢了。

kube-proxy为每个service生成一个虚拟服务器(IPVS Virtual Server)的定义。

基本流程:

当前节点接收到外部流量后, 如果该数据包是交给当前节点上的clusterIP, 则会直接将数据包交给 kube-ipvs0, 而这个接口是内核虚拟出来的, 而kube-proxy定义的VS直接关联到kube-ipvs0上。

如果是本地节点pod发送的请求, 基本上属于本地通信, 效率是非常高的。

默认情况下, 这里的ipvs使用的是nat转发模型, 而且支持更多的后端调度算法。仅仅在涉及到源地址转换的场景中, 会涉及到极少量的iptables规则(应该不会超过20条)

对于Kubernetes来说, 默认情况下, 支持的规则是 iptables, 可以通过多种方式对代理模式进行更改, 因为这些规则都是基于kube-proxy来定制的, 所以, 如果要更改代理模式的话, 就需要调整kube-proxy的属性。

注意: Kubernetes-v1.29版本后, 不再使用iptables, 而使用nftables framework, 工具使用nft list ruleset 查看

更改kube-proxy为IPVS模式方法说明

```
#方法1: 在集群创建的时候, 修改kubeadm-init.yml 添加如下配置, 此方法是生产中推荐
[root@master1 ~]#kubeadm config print init-defaults > kubeadm-init.yaml

#在文件最后面添加以下几行
[root@master1 ~]#vim  kubeadm-init.yaml
---
apiVersion: kubeproxy.config.kubernetes.io/v1alpha1
kind: KubeProxyConfiguration
featureGates:
  SupportIPVSProxyMode: true
mode: ipvs

#方法2: 在测试环境中, 临时修改一下configmap中proxy的基本属性, 此方式在测试环境中推荐使用
root@master1:~# kubectl edit configmap kube-proxy -n kube-system
...
mode: "ipvs" #修改此行, 默认为空""表示iptables模式
...

#所有的规则都是 ipvs本身的调度规则, 可以在node节点上查看效果(master也可以)
ipvsadm -Ln
nft list ruleset
```

范例：查看默认模式 iptables

```
#查看所有节点的当前模式
[root@master1 ~]#curl 127.0.0.1:10249/proxyMode
iptables
[root@node1 ~]#curl 127.0.0.1:10249/proxyMode
iptables

[root@master1 ~]#kubectl get pod -n kube-system -l k8s-app=kube-proxy
NAME                READY   STATUS    RESTARTS   AGE
kube-proxy-gh528    1/1     Running   0           2m28s
kube-proxy-hpl88    1/1     Running   0           2m28s
kube-proxy-nglzg    1/1     Running   0           2m27s
kube-proxy-tg4ss    1/1     Running   0           23s

#查看日志，可以看到默认使用iptables模式
[root@master1 ~]#kubectl logs kube-proxy-gh528 -n kube-system
I0409 03:29:31.160269      1 node.go:163] Successfully retrieved node IP:
10.0.0.202
I0409 03:29:31.160512      1 server_others.go:109] "Detected node IP"
address="10.0.0.202"
I0409 03:29:31.160583      1 server_others.go:535] "Using iptables proxy"
I0409 03:29:31.175304      1 server_others.go:176] "Using iptables Proxier"
I0409 03:29:31.175353      1 server_others.go:183] "kube-proxy running in dual-
stack mode" ipFamily=IPv4
I0409 03:29:31.175361      1 server_others.go:184] "Creating dualStackProxier
for iptables"
.....

#在Kubernetes集群中，关于kube-proxy的所有属性信息，可以通过 configmap 的资源对象实现
[root@master1 ~]#kubectl describe configmap kube-proxy -n kube-system

Name:                kube-proxy
Namespace:            kube-system
Labels:               app=kube-proxy
...
iptables:
  masqueradeAll: false           # 这个属性打开，会对所有的请求都进行源地址转换
  masqueradeBit: null
  minSyncPeriod: 0s
  syncPeriod: 0s
ipvs:
  excludeCIDRs: null
  minSyncPeriod: 0s
  scheduler: ""                # 调度算法，默认是random robin
  strictARP: false
  syncPeriod: 0s
  tcpFinTimeout: 0s
  tcpTimeout: 0s
  udpTimeout: 0s
kind: KubeProxyConfiguration
metricsBindAddress: ""
mode: ""                     # 默认没有指定，就使用 iptables 规则
...
```

范例：修改为IPVS模式

#查看当前模式

```
[root@master1 ~]#curl 127.0.0.1:10249/proxyMode
iptables
[root@node1 ~]#curl 127.0.0.1:10249/proxyMode
iptables
```

#所有节点查看，当前没有IPVS规则

```
[root@master1 ~]#apt update && apt -y install ipvsadm && ipvsadm -Ln
IP Virtual Server version 1.2.1 (size=4096)
Prot LocalAddress:Port Scheduler Flags
  -> RemoteAddress:Port           Forward Weight ActiveConn InActConn
[root@master1 ~]#
```

#查看默认的configmap

```
[root@master1 ~]#kubectl get cm -n kube-system
```

NAME	DATA	AGE
coredns	1	419d
extension-apiserver-authentication	6	419d
kube-proxy	2	419d
kube-root-ca.crt	1	419d
kubeadm-config	1	419d
kubelet-config	1	419d

#方法1:

```
[root@master1 ~]#kubectl get configmap kube-proxy -n kube-system > kube-
proxy.yaml
[root@master1 ~]#vim kube-proxy.yaml
...
mode: "ipvs" #修改此行，默认为 "" 表示iptables
...
[root@master1 ~]#kubectl apply -f kube-proxy.yaml
```

#方法2: 在线修改一下configmap中proxy的基本属性

```
[root@master1 ~]#kubectl edit configmap kube-proxy -n kube-system
```

```
...
ipvs:
  excludeCIDRs: null
  minSyncPeriod: 0s
  scheduler: ""          #默认RR调度算法
  strictARP: false
  syncPeriod: 0s
  tcpFinTimeout: 0s
  tcpTimeout: 0s
  udpTimeout: 0s
kind: KubeProxyConfiguration
metricsBindAddress: ""
mode: "ipvs"            #修改此行，默认为 "" 表示iptables
...
```

#查看kube-proxy对应Pod的标签

```
[root@master1 ~]#kubectl get pod -n kube-system --show-labels |grep kube-proxy
kube-proxy-7p8vr                1/1      Running   5 (99m ago)    33d
  controller-revision-hash=k8s-app=kube-proxy,pod-template-
generation=1
kube-proxy-fn6mt                1/1      Running   8 (10h ago)    33d
  controller-revision-hash=k8s-app=kube-proxy,pod-template-
generation=1
```



```
kube-proxy-tb7gh          1/1      Running   9 (10h ago)    33d
  controller-revision-hash=85648747bd,k8s-app=kube-proxy,pod-template-
generation=1
kube-proxy-xpxnt          1/1      Running   9 (10h ago)    33d
  controller-revision-hash=85648747bd,k8s-app=kube-proxy,pod-template-
generation=1
```

#需要稍等一会儿才能生效

#如果想立即生效可以重启所有的kube-proxy pod对象

#方法1: 重启相关Pod

```
[root@master1 ~]#kubectl get ds -n kube-system
```

```
NAME                DESIRED    CURRENT    READY    UP-TO-DATE    AVAILABLE    NODE SELECTOR
AGE
kube-proxy          4          4          4        4             4
kubernetes.io/os=linux 412d
```

```
[root@master1 ~]#kubectl get pod -n kube-system -l k8s-app=kube-proxy
```

```
NAME                READY    STATUS    RESTARTS    AGE
kube-proxy-7p8vr    1/1     Running   8 (7h4m ago) 412d
kube-proxy-fn6mt    1/1     Running   8 (7h4m ago) 412d
kube-proxy-tb7gh    1/1     Running   8 (7h5m ago) 412d
kube-proxy-xpxnt    1/1     Running   7 (7h5m ago) 412d
```

```
[root@master1 ~]#kubectl rollout restart daemonset -n kube-system kube-proxy
daemonset.apps/kube-proxy restarted
```

#查看Pod重启

```
[root@master1 ~]#kubectl get pod -n kube-system -l k8s-app=kube-proxy
```

```
NAME                READY    STATUS    RESTARTS    AGE
kube-proxy-cl6zt    1/1     Running   0            1s
kube-proxy-km245    1/1     Running   0            5s
kube-proxy-lrdcv    1/1     Running   0            7s
kube-proxy-vm49z    1/1     Running   0            3s
```

#方法2:

#新版

```
[root@master1 ~]#kubectl delete pod -n kube-system -l k8s-app=kube-proxy
```

#旧版

```
[root@master1 ~]#kubectl delete pod -n kube-system -l kubernetes-app=kube-proxy
```

#检查效果

#新版

```
[root@master1 ~]#kubectl get pod -n kube-system -l k8s-app=kube-proxy -o wide
```

#旧版

```
[root@master1 ~]#kubectl get pod -n kube-system -l kubernetes-app=kube-proxy -o
wide
```

```
NAME                READY    STATUS    RESTARTS    AGE    IP            NODE
NOMINATED NODE    READINESS GATES
kube-proxy-bflzm    1/1     Running   0            50s    10.0.0.106
node3.wang.org      <none>   <none>
kube-proxy-czlg2    1/1     Running   0            50s    10.0.0.103
master3.cluster.local <none>   <none>
kube-proxy-j7szq    1/1     Running   0            50s    10.0.0.104
node1.wang.org      <none>   <none>
kube-proxy-mb2hn    1/1     Running   0            50s    10.0.0.105
node2.wang.org      <none>   <none>
kube-proxy-rlwx9    1/1     Running   0            51s    10.0.0.102
master2.cluster.local <none>   <none>
```

```
kube-proxy-w4swc 1/1 Running 0 50s 10.0.0.101
master1.cluster.local <none> <none>
```

#验证模式修改IPVS

```
[root@master1 ~]#curl 127.0.0.1:10249/proxyMode
ipvs
[root@node1 ~]#curl 127.0.0.1:10249/proxyMode
ipvs
```

#查看日志

```
[root@master1 ~]#kubectl logs kube-proxy-bflzm -n kube-system
I0320 13:57:28.952304 1 node.go:172] Successfully retrieved node IP:
10.0.0.106
I0320 13:57:28.952410 1 server_others.go:140] Detected node IP 10.0.0.106
I0320 13:57:28.970159 1 server_others.go:206] kube-proxy running in dual-
stack mode, IPv4-primary
I0320 13:57:28.970305 1 server_others.go:274] Using ipvs Proxier.
I0320 13:57:28.970394 1 server_others.go:276] creating dualStackProxier
for ipvs.
.....
```

#查看默认RR算法

```
[root@master1 ~]#ipvsadm -Ln
IP Virtual Server version 1.2.1 (size=4096)
Prot LocalAddress:Port Scheduler Flags
-> RemoteAddress:Port Forward Weight ActiveConn InActConn
TCP 192.168.0.1:443 rr
-> 10.0.0.101:6443 Masq 1 0 0
-> 10.0.0.102:6443 Masq 1 0 0
-> 10.0.0.103:6443 Masq 1 0 0
TCP 192.168.0.10:53 rr
-> 172.16.1.27:53 Masq 1 0 0
-> 172.16.3.71:53 Masq 1 0 0
TCP 192.168.0.10:9153 rr
-> 172.16.1.27:9153 Masq 1 0 0
-> 172.16.3.71:9153 Masq 1 0 0
UDP 192.168.0.10:53 rr
-> 172.16.1.27:53 Masq 1 0 0
-> 172.16.3.71:53 Masq 1 0 0
```

#创建service观察结果

```
[root@master1 ~]#cat service-test.yaml
kind: Service
apiVersion: v1
metadata:
  name: service-test
spec:
  selector:
    app: myweb
  ports:
  - name: http
    protocol: TCP
    port: 80
    targetPort: 80
```

```
[root@master1 ~]#kubectl apply -f service-test.yaml
```

#查看IPVS规则

```
[root@master1 ~]#ipvsadm -Ln
IP Virtual Server version 1.2.1 (size=4096)
Prot LocalAddress:Port Scheduler Flags
  -> RemoteAddress:Port           Forward Weight ActiveConn InActConn
TCP  192.168.0.1:443 rr
  -> 10.0.0.101:6443              Masq    1      0      0
  -> 10.0.0.102:6443              Masq    1      0      0
  -> 10.0.0.103:6443              Masq    1      0      0
TCP  192.168.0.10:53 rr
  -> 172.16.1.27:53               Masq    1      0      0
  -> 172.16.3.71:53               Masq    1      0      0
TCP  192.168.0.10:9153 rr
  -> 172.16.1.27:9153             Masq    1      0      0
  -> 172.16.3.71:9153             Masq    1      0      0
TCP  192.168.68.146:80 rr
  -> 172.16.1.56:80               Masq    1      0      0
  -> 172.16.2.93:80               Masq    1      0      0
  -> 172.16.3.106:80              Masq    1      0      0
UDP  192.168.0.10:53 rr
  -> 172.16.1.27:53               Masq    1      0      0
  -> 172.16.3.71:53               Masq    1      0      0
```

```
[root@master1 ~]#cat service-nodeport.yaml
```

```
kind: Service
apiVersion: v1
metadata:
  name: service-nodeport
spec:
  type: NodePort
  selector:
    app: myweb
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30066
```

```
[root@master1 ~]#kubectl apply -f service-nodeport.yaml
```

#查看IPVS规则

```
[root@master1 ~]#ipvsadm -Ln
IP Virtual Server version 1.2.1 (size=4096)
Prot LocalAddress:Port Scheduler Flags
  -> RemoteAddress:Port           Forward Weight ActiveConn InActConn
TCP  172.16.0.0:30066 rr
  -> 172.16.1.56:80               Masq    1      0      0
  -> 172.16.2.93:80               Masq    1      0      0
  -> 172.16.3.106:80              Masq    1      0      0
TCP  172.17.0.1:30066 rr
  -> 172.16.1.56:80               Masq    1      0      0
  -> 172.16.2.93:80               Masq    1      0      0
  -> 172.16.3.106:80              Masq    1      0      0
TCP  192.168.0.1:443 rr
  -> 10.0.0.101:6443              Masq    1      0      0
  -> 10.0.0.102:6443              Masq    1      0      0
  -> 10.0.0.103:6443              Masq    1      0      0
TCP  192.168.0.10:53 rr
```

```

-> 172.16.1.27:53          Masq    1      0      0
-> 172.16.3.71:53         Masq    1      0      0
TCP 192.168.0.10:9153 rr
-> 172.16.1.27:9153       Masq    1      0      0
-> 172.16.3.71:9153       Masq    1      0      0
TCP 192.168.62.141:80 rr
-> 172.16.1.56:80         Masq    1      0      0
-> 172.16.2.93:80         Masq    1      0      0
-> 172.16.3.106:80        Masq    1      0      0
TCP 192.168.68.146:80 rr
-> 172.16.1.56:80         Masq    1      0      0
-> 172.16.2.93:80         Masq    1      0      0
-> 172.16.3.106:80        Masq    1      0      0
TCP 10.0.0.101:30066 rr
-> 172.16.1.56:80         Masq    1      0      0
-> 172.16.2.93:80         Masq    1      0      0
-> 172.16.3.106:80        Masq    1      0      0
UDP 192.168.0.10:53 rr
-> 172.16.1.27:53         Masq    1      0      0
-> 172.16.3.71:53         Masq    1      0      0

```

1.4 综合案例：wordpress

```

[root@master1 ~]#cat wordpress-mysql-svc-deployment.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: demo
---
apiVersion: v1
kind: Service
metadata:
  name: wordpress
  namespace: demo
spec:
  ports:
    - port: 80
      protocol: TCP
      targetPort: 80
  selector:
    app: wordpress
  #type: NodePort
  type: LoadBalancer #有前提，需要部署LoadBalancer软件
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress
  namespace: demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: wordpress
  template:

```

```

    metadata:
      labels:
        app: wordpress
    spec:
      containers:
        - image: registry.cn-beijing.aliyuncs.com/wangxiaochun/wordpress:php8.2-
apache
    name: wordpress
    env:
      - name: WORDPRESS_DB_HOST
        #value: mysql.demo.svc.cluster.local.
        value: mysql
      - name: WORDPRESS_DB_USER
        value: wordpress
      - name: WORDPRESS_DB_PASSWORD
        value: "123456"
      - name: WORDPRESS_DB_NAME
        value: wordpress
---
apiVersion: v1
kind: Service
metadata:
  name: mysql
  namespace: demo
spec:
  ports:
    - port: 3306
      protocol: TCP
      targetPort: 3306
  selector:
    app: mysql
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
  namespace: demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - image: registry.cn-beijing.aliyuncs.com/wangxiaochun/mysql:8.0.29-oracle
          name: mysql
          env:
            - name: MYSQL_ROOT_PASSWORD
              value: "123456"
            - name: MYSQL_DATABASE
              value: "wordpress"
            - name: MYSQL_USER
              value: wordpress

```

```
- name: MYSQL_PASSWORD
  value: "123456"
```

2 域名解析

2.1 服务发现机制

在传统的系统部署中，服务运行在一个固定的已知的 IP 和端口上，如果一个服务需要调用另外一个服务，可以通过地址直接调用

在Kubernetes 集群中，基于clusterip地址来访问每service是很不方便的

虽然通过配置DNS可以实现名称解析来访问，但是在Kubernetes集群中，服务实例的启动和销毁是很频繁的，服务地址在动态的变化，所以传统的方式配置DNS解析记录就很不友好了。

将请求发送到动态变化的服务实例上，可以通过以下两个步骤来实现：

- 服务注册 — 创建服务实例后，主动将当前服务实例的信息，存储到一个集中式的服务管理中心。
- 服务发现 — 当A服务需要找未知的B服务时，先去服务管理中心查找B服务地址，然后根据该地址找到B服务

对于docker本身来说，可以基于自定义网络，或者--link参数方式实现有限的发现功能

Kubernetes主要有两种服务发现机制：

- 环境变量
- DNS解析

2.2 环境变量

环境变量主要有两种实现方式：

- Kubernetes Service 环境变量

Kubernetes为每个Service资源生成包括以下形式的环境变量在内一系列环境变量

在同一名称空间中后续创建的Pod对象都会自动拥有这些变量

注意：此方式不支持Service的动态变化，即在创建Pod对象以后，后续Service发生变化也不会更新已经生成相关的环境变量，生产此方式不太常见

Service 相关的环境变量形式如下：

```
{SVCNAME}_SERVICE_HOST、{SVCNAME}_PORT
```

示例：default名称空间创建名为test的Service，default名称空间下的每个Pod内部会被自动注入和service相关的变量

```
TEST_SERVICE_HOST=ClusterIP
TEST_PORT=tcp://ClusterIP:80
#注意：如果先创建pod然后关联到service是不生效的
```

- Docker Link形式的环境变量

Docker使用--link选项实现容器连接时所设置的环境变量形式

在创建Pod对象时，kubernetes也会把与此形式兼容的一系列环境变量注入到Pod对象中。

由于Kubernetes v1.24版本禁用dockershim，所以后续的Kubernetes环境中，可能不再支持这种方式。

范例:

#先创建service环境

```
[root@master1 ~]#cat service-test.yaml
```

```
kind: Service
apiVersion: v1
metadata:
  name: service-test
spec:
  selector:
    app: myweb
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 80
```

#先创建service环境

```
[root@master1 ~]#kubectl apply -f service-test.yaml
```

#查看service

```
[root@master1 ~]#kubectl get svc service-test
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service-test	ClusterIP	192.168.193.236	<none>	80/TCP	35m

#再创建pod对象，通过标签将其添加到service环境中

```
[root@master1 ~]#kubectl create deployment myweb --image=registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1 --replicas=3
```

#注意：必须先创建service，后创建deployment，否则的话，无法演示后面的效果

#确认效果

```
[root@master1 ~]#kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
myweb-5df49fdd5-brxcp	1/1	Running	0	26m
myweb-5df49fdd5-xnrt4	1/1	Running	0	26m
myweb-5df49fdd5-z7d67	1/1	Running	0	26m

#注意：看到的KUBERNETES_<xxx> 都是默认coredns在pod中添加进去的环境变量信息，包括service信息

```
[root@master1 ~]#kubectl exec myweb-5df49fdd5-brxcp -- env
```

```
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

```
HOSTNAME=myweb-5df49fdd5-brxcp
```

```
SERVICE_TEST_SERVICE_PORT=80
```

#注意以下三行，自动生成service相关的环境变量

环境变量

```
SERVICE_TEST_PORT=tcp://192.168.193.236:80
```

```
SERVICE_TEST_PORT_80_TCP=tcp://192.168.193.236:80
```

```
KUBERNETES_SERVICE_HOST=192.168.0.1
```

```
KUBERNETES_SERVICE_PORT_HTTPS=443
```

```
KUBERNETES_PORT_443_TCP=tcp://192.168.0.1:443
```

```
KUBERNETES_PORT_443_TCP_PORT=443
```

```
SERVICE_TEST_SERVICE_PORT_HTTP=80
```

```
SERVICE_TEST_SERVICE_HOST=192.168.193.236
```

```
SERVICE_TEST_PORT_80_TCP_PROTO=tcp
```

```
SERVICE_TEST_PORT_80_TCP_PORT=80
```

```
SERVICE_TEST_PORT_80_TCP_ADDR=192.168.193.236
```

```
KUBERNETES_SERVICE_PORT=443
```

```
KUBERNETES_PORT=tcp://192.168.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_ADDR=192.168.0.1
DEPLOYENV=Production
RELEASE=Stable
PS1=[\u@\h \w]\$
HOME=/root

#在容器内部，可以直接使用变量名来进行服务的访问
[root@master1 ~]#kubectl exec -it myweb-5df49fdd5-brxcp -- sh

#访问环境变量
[root@myweb-5df49fdd5-brxcp /]# curl ${SERVICE_TEST_SERVICE_HOST}
kubernetes pod-test v0.1!! ClientIP: 172.16.1.57, ServerName: myweb-5df49fdd5-
xnrt4, ServerIP: 172.16.3.107!
[root@myweb-5df49fdd5-brxcp /]# echo ${SERVICE_TEST_SERVICE_HOST}
192.168.193.236
```

2.3 CoreDNS

2.3.1 CoreDNS 介绍和部署



专用于kubernetes集群中的服务注册和发现的解决方案就是KubeDNS。

kubeDNS自从Kubernetes诞生以来，其方案的具体实现方案前后经历了三代，分别是 SkyDNS、KubeDNS、CoreDNS。

Kubernetes-v1.3之前使用SkyDNS, 之后到Kubernetes-v1.13之前使用KubeDNS,当前默认使用CoreDNS

CoreDNS 是由 CNCF（Cloud Native Computing Foundation，云原生计算基金会）旗下的项目，最初主要是由 CoreOS（后被 Red Hat 收购）开发的，现在是一个开源的、社区驱动的项目，有众多开发者参与贡献代码、持续完善它。

CoreDNS 是基于Go语言实现的一个DNS服务器，由于其灵活性，它可以在多种环境中使用。

CoreDNS 是一种灵活且高性能的 DNS 服务器，在 Kubernetes 等云原生环境中被广泛应用来进行服务发现等相关功能实现。

CoreDNS 通过 Kubernetes 插件与 Kubernetes 集成，或者通过etcd插件与etcd 集成,实现服务发现

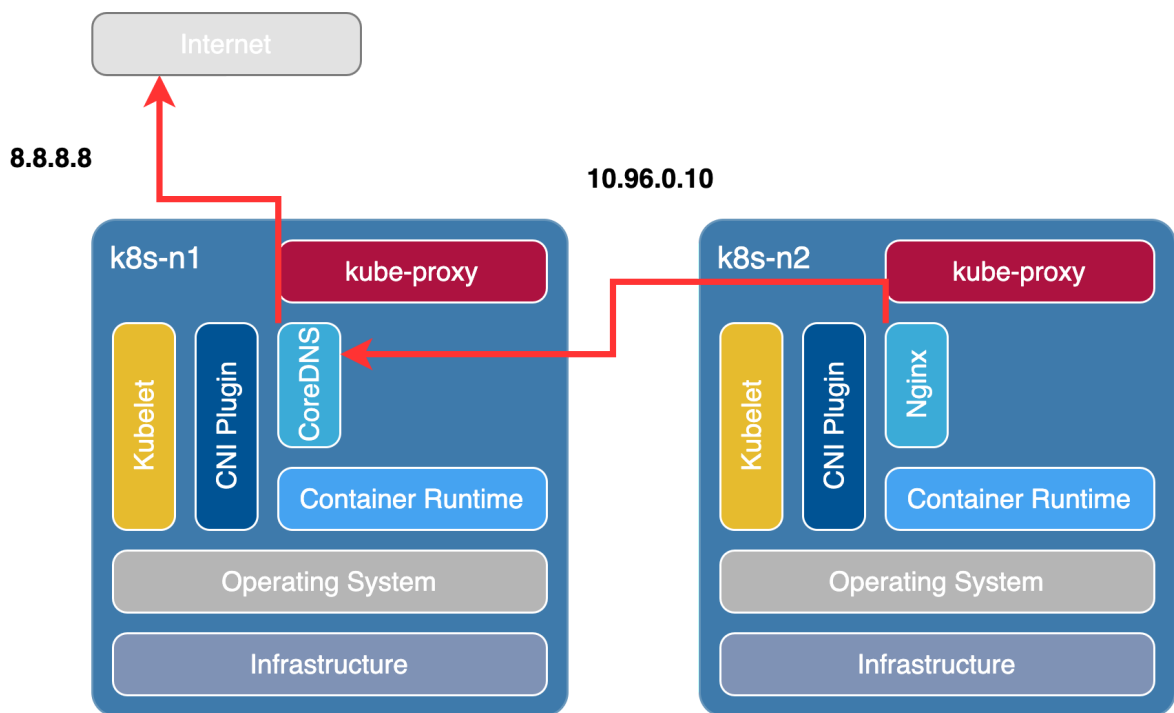
CoreDNS 在 Apache License Version 2下获得许可，并且完全开源。

CoreDNS 是一个云原生计算基金会已毕业的项目。

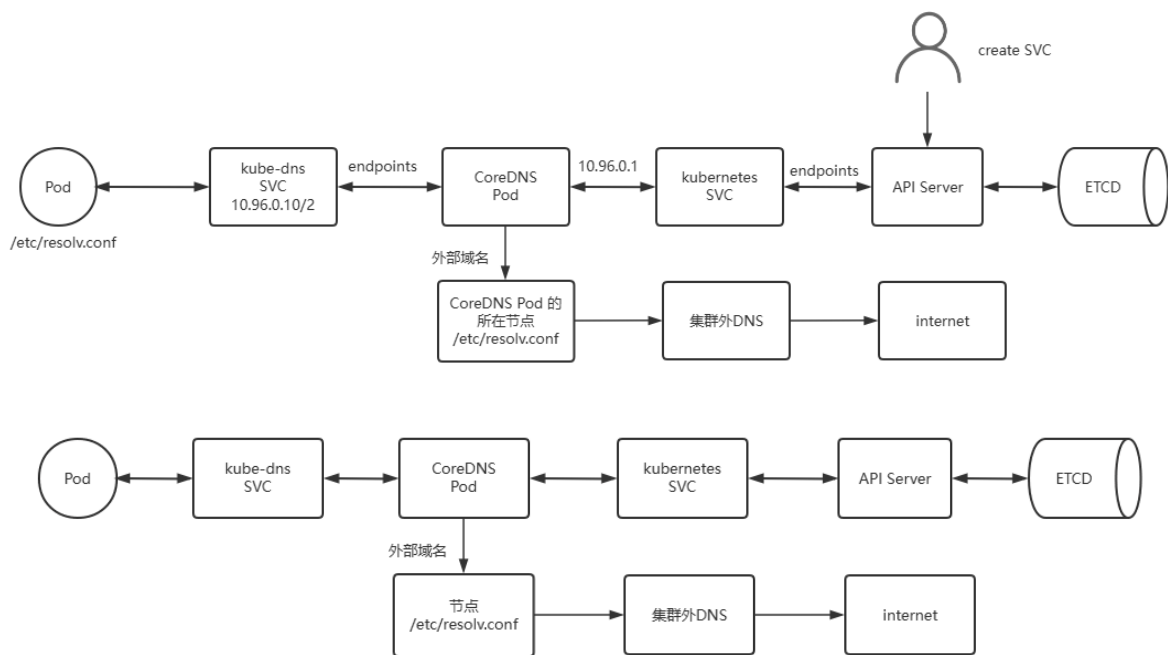
CoreDNS 官方网站

```
https://coredns.io/
https://github.com/coredns/coredns
```

CoreDNS 解析流程



CoreDNS 通过访问名为 kubernetes 的 Service,找到 API Server 进而连接到 ETCD, 从而实现 Kubernetes 集群中的 Service, Endpoint, Pod 等资源的查找



- Client Pod 查询自身的/etc/resolv.conf 文件中指向的DNS服务器地址,此地址为kube-dns service 的地址,即将解析请求转发给名为 kube-dns的 service
- kube-dns service会将请求转发到后端CoreDNS Pod,为了DNS的高可用,通常有两个CoreDNS Pod, 并位于kube-system名称空间
- CoreDNS Pod 根据Corefile的配置会连接到在default名称空间的名为kubernetes的service,而 kubernetes service对应的Endpoints为所有kube-apiserver:6443的地址
- kubernetes service 监视service IP的变动, 维护DNS解析记录,并将变化发送至ETCD实现DNS记录的存储
- CoreDNS查询到service name对应的IP后返回给客户端
- 如果查询的是外部域名, CoreDNS无法解析, 就转发给指定的域名服务器, 一般是CoreDNS Pod 所在宿主主机节点上/etc/resolv.conf中的服务器解析

范例: 基于二进制kubernetes集群中的 CoreDNS 部署

注意: 使用Kubeadm方式安装的Kubernetes集群默认已经安装CoreDNS无需再安装

```
[root@master1 ~]#wget -O coredns.yaml
https://raw.githubusercontent.com/coredns/deployment/master/kubernetes/coredns.
yaml.sed

#注意:文件中的大写字母的内容需要修改
[root@master1 ~]#cat coredns.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: coredns
  namespace: kube-system
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  labels:
    kubernetes.io/bootstrapping: rbac-defaults
  name: system:coredns
rules:
- apiGroups:
  - ""
  resources:
  - endpoints
  - services
  - pods
  - namespaces
  verbs:
  - list
  - watch
- apiGroups:
  - discovery.k8s.io
  resources:
  - endpointslices
  verbs:
  - list
  - watch
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  annotations:
    rbac.authorization.kubernetes.io/autoupdate: "true"
  labels:
    kubernetes.io/bootstrapping: rbac-defaults
  name: system:coredns
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: system:coredns
subjects:
- kind: ServiceAccount
  name: coredns
  namespace: kube-system
---
apiVersion: v1
```

```

kind: ConfigMap
metadata:
  name: coredns
  namespace: kube-system
data:
  Corefile: |
    .:53 {
      errors
      health {
        lameduck 5s
      }
      ready
      #此处CLUSTER_DOMAIN修改cluster.local,REVERSE_CIDRS 修改为 in-addr.arpa
      ip6.arpa
      kubernetes CLUSTER_DOMAIN REVERSE_CIDRS {
        fallthrough in-addr.arpa ip6.arpa
      }
      prometheus :9153
      forward . UPSTREAMNAMESERVER {      #此处UPSTREAMNAMESERVER修改为
/etc/resolv.conf
        max_concurrent 1000
      }
      cache 30
      loop
      reload
      loadbalance
    }STUBDOMAINS
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: coredns
  namespace: kube-system
  labels:
    k8s-app: kube-dns
    kubernetes.io/name: "CoreDNS"
spec:
  # replicas: not specified here:
  # 1. Default is 1.
  # 2. Will be tuned in real time if DNS horizontal auto-scaling is turned on.
  replicas: 2      #添加此行为多副本,默认为1
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
  selector:
    matchLabels:
      k8s-app: kube-dns
  template:
    metadata:
      labels:
        k8s-app: kube-dns
    spec:
      priorityClassName: system-cluster-critical
      serviceAccountName: coredns
      tolerations:
        - key: "CriticalAddonsOnly"
          operator: "Exists"

```

```
nodeSelector:
  kubernetes.io/os: linux
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
            - key: k8s-app
              operator: In
              values: ["kube-dns"]
        topologyKey: kubernetes.io/hostname
containers:
- name: coredns
  image: coredns/coredns:1.9.4
  imagePullPolicy: IfNotPresent
  resources:
    limits:
      memory: 170Mi #此处的资源限制修改为合适的值,比如:4096Mi
    requests:
      cpu: 100m #此处的资源限制修改为合适的值
      memory: 70Mi #此处的资源限制修改为合适的值
  args: [ "-conf", "/etc/coredns/Corefile" ]
  volumeMounts:
  - name: config-volume
    mountPath: /etc/coredns
    readOnly: true
  ports:
  - containerPort: 53
    name: dns
    protocol: UDP
  - containerPort: 53
    name: dns-tcp
    protocol: TCP
  - containerPort: 9153
    name: metrics
    protocol: TCP
  securityContext:
    allowPrivilegeEscalation: false
    capabilities:
      add:
      - NET_BIND_SERVICE
      drop:
      - all
    readOnlyRootFilesystem: true
  livenessProbe:
    httpGet:
      path: /health
      port: 8080
      scheme: HTTP
    initialDelaySeconds: 60
    timeoutSeconds: 5
    successThreshold: 1
    failureThreshold: 5
  readinessProbe:
    httpGet:
      path: /ready
      port: 8181
      scheme: HTTP
```

```

    dnsPolicy: Default
    volumes:
      - name: config-volume
        configMap:
          name: coredns
          items:
            - key: Corefile
              path: Corefile
---
apiVersion: v1
kind: Service
metadata:
  name: kube-dns
  namespace: kube-system
  annotations:
    prometheus.io/port: "9153"
    prometheus.io/scrape: "true"
  labels:
    k8s-app: kube-dns
    kubernetes.io/cluster-service: "true"
    kubernetes.io/name: "CoreDNS"
spec:
  selector:
    k8s-app: kube-dns
  clusterIP: CLUSTER_DNS_IP #修改此处为kube-dns SVC的地址,比如:10.96.0.10,可通过查看
Pod的/etc/resolv.conf 获取
  ports:
    - name: dns
      port: 53
      protocol: UDP
    - name: dns-tcp
      port: 53
      protocol: TCP
    - name: metrics
      port: 9153
      protocol: TCP

#修改上面文件内容汇总
[root@master1 ~]#vim coredns.yaml
CLUSTER_DOMAIN: cluster.local
REVERSE_CIDRS: in-addr.arpa ip6.arpa
UPSTREAMNAMESERVER: /etc/resolv.conf
CLUSTER_DNS_IP: 10.96.0.2

#应用创建coredns的Pod
[root@master1 ~]#kubectl apply -f coredns.yaml

```

2.3.2 CoreDNS 工作机制

2.3.2.1 CoreDNS 域名解析



Cluster DNS (CoreDNS) 是Kubernetes集群的必备附件, 负责为Kubernetes提供名称解析和服务发现

每个Service资源对象，在CoreDNS上都会自动生成如下格式的名称，结合该名称会生成对应的一些不同类型的DNS资源记录

```
<service>.<ns>.<svc>.<zone>
```

```
<service>: #当前Service对象的名称
<ns>:      #当前Service对象所属的名称空间
<zone>:    #当前Kubernetes集群使用的域名后缀，默认为“cluster.local”，用 kubeadm init
--service-dns-domain 指定
```

范例：kubeadm安装方式时查看默认Zone名称

```
[root@master1 ~]#kubeadm config print init-defaults |grep dns
dns: {}
    dnsDomain: cluster.local
```

CoreDNS会持续监视API Server上的Service资源对象的变动，并实时反映到相关的DNS资源记录中

Pod中各容器内部默认会在其 /etc/resolv.conf中，将nameserver指向CoreDNS相关的Service的ClusterIP，默认为service网段的第10个IP，比如：10.96.0.10，其后面的Endpoint是coredns对应的Pod的IP，此配置由kubelet创建Pod时根据指定的配置自动注入

Kubelet会为创建的每一个容器于/etc/resolv.conf配置文件中生成DNS查询客户端依赖到的必要配置

相关的配置信息源自于kubelet的配置参数，各容器的DNS服务器由clusterDNS参数的值设定，它的取值为kube-system名称空间中的Service对象kube-dns的ClusterIP，默认为10.96.0.10，而DNS搜索域的值由clusterDomain参数的值设定

ClusterDNS服务的默认名称：kube-dns.kube-system.svc.cluster.local

ClusterDNS服务的默认地址：Cluster Network 网段的第10个IP，默认为：10.96.0.10

若部署Kubernetes集群时未特别指定，其值将为cluster.local、svc.cluster.local和NAMESPACENAME.svc.cluster.local

范例：集群上的一个随机选择的Pod中的容器查看DNS客户端配置

```
[root@master1 ~]#kubect1 run client-test-$RANDOM --image registry.cn-
beijing.aliyuncs.com/wangxiaochun/admin-box:v0.1 --restart=Never --rm -it --
command -- cat /etc/resolv.conf
nameserver 10.96.0.10
search default.svc.cluster.local svc.cluster.local cluster.local wang.org
options ndots:5
```

#上述search参数中指定的DNS各搜索域，是以次序指定的几个域名后缀，它们各自的如下所示。

#<ns>.<svc>.<zone>：附带有特定名称空间的域名，例如default.svc.cluster.local

#<svc>.<zone>：附带了Kubernetes标识Service专用子域svc的域名，例如svc.cluster.local；

#<zone>：集群本地域名，例如cluster.local。

#ndots:5，表示如果手工查询时候给的域名包含的点“.”不超过5个，那么进行DNS查找时将使用非完全限定名称，即用search指定的域名补全

#即 <手工输入域名> 或者 <手工输入域名>.<search 部分给定的域名后缀>

#如果你查询的域名包含点数大于等于5，那么DNS查询，默认会使用绝对域名进行查询。即使用 <手工输入域名>

2.3.2.2 Service 资源对应的DNS资源记录

基于DNS的服务发现，对于每个Service对象，都会具有以下3个类型的DNS资源记录A/AAAA，PTR和SRV

- 根据ClusterIP的地址类型，为IPv4生成固定格式的 A记录，为IPv6生成AAAA记录

```
<service>.<ns>.svc.<zone>. <ttl> IN A <cluster-ip>
<service>.<ns>.svc.<zone>. <ttl> IN AAAA <cluster-ip>
```

#示例:

service-test.default.svc.cluster.local.

#注意: cluster.local 是默认zone名称，在初始化Kubernetes集群中，自己通过dnsDomain属性定制的。

- 对于每个给定的A记录或AAAA记录都要生成PTR记录，格式如下所示:

```
<d>.<c>.<b>.<a>.in-addr.arpa. <ttl> IN PTR <service>.<ns>.svc.<zone>.
h4.h3.h2.h1.g4.g3.g2.g1.f4.f3.f2.f1.e4.e3.e2.e1.d4.d3.d2.d1.c4.c3.c2.c1.b4.b3.b2
.b1.a4.a3.a2.a1.ip6.arpa <ttl> IN PTR <service>.<ns>.svc.<zone>.
```

- 为每个定义了名称的端口生成一个SRV记录，未命名的端口号则不具有该记录

```
_<port_name>._<proto>.<service>.<ns>.svc.<zone>. <ttl> IN SRV <weight>
<priority> <port-number> <service>.<ns>.svc.<zone>.
```

例如

#在default名称空间中创建Service对象service-test的地址为10.97.72.1，且为TCP协议的80端口取名http，对于默认的cluster.local域名来说，此它会拥有如下3个DNS资源记录。

A记录: service-test.default.svc.cluster.local. 30 IN A 10.97.72.1;
PTR记录: 1.72.97.10.in-addr.arpa. 30 IN PTR service-test.default.svc.cluster.local.
SRV记录: _http._tcp.service-test.default.svc.cluster.local. 30 IN SRV 0 100 80 service-test.default.svc.cluster.local.

范例: 查看 CoreDNS 默认配置

```
[root@master1 ~]#kubectl get svc -A
NAMESPACE      NAME           TYPE           CLUSTER-IP      EXTERNAL-IP      PORT(S)
AGE
default        kubernetest    ClusterIP      10.96.0.1       <none>           443/TCP
35d
default        pod-test      NodePort       10.108.29.167   <none>           80:30383/TCP
17h
kube-system    kube-dns      ClusterIP      10.96.0.10      <none>           53/UDP,53/TCP,9153/TCP
35d
```

#查看kube-dns svc

```
[root@master1 ~]#kubectl describe svc kube-dns -n kube-system
Name:          kube-dns
Namespace:     kube-system
Labels:        k8s-app=kube-dns
```

```
kubernetes.io/cluster-service=true
kubernetes.io/name=CoreDNS
Annotations:    prometheus.io/port: 9153
                prometheus.io/scrape: true
Selector:       k8s-app=kube-dns
Type:           ClusterIP
IP Family Policy: SingleStack
IP Families:    IPv4
IP:             10.96.0.10
IPs:            10.96.0.10
Port:           dns 53/UDP
TargetPort:     53/UDP
Endpoints:      10.244.1.78:53,10.244.1.79:53
Port:           dns-tcp 53/TCP
TargetPort:     53/TCP
Endpoints:      10.244.1.78:53,10.244.1.79:53
Port:           metrics 9153/TCP
TargetPort:     9153/TCP
Endpoints:      10.244.1.78:9153,10.244.1.79:9153
Session Affinity: None
Events:         <none>
```

```
[root@master1 ~]#kubectl get svc -n kube-system kube-dns -o yaml
```

```
apiVersion: v1
kind: Service
metadata:
  annotations:
    prometheus.io/port: "9153"
    prometheus.io/scrape: "true"
  creationTimestamp: ""
  labels:
    k8s-app: kube-dns
    kubernetes.io/cluster-service: "true"
    kubernetes.io/name: CoreDNS
  name: kube-dns
  namespace: kube-system
  resourceVersion: "239"
  uid: 84729fbb-2e5f-426d-aaf9-64c8b01cb762
spec:
  clusterIP: 10.96.0.10
  clusterIPs:
  - 10.96.0.10
  internalTrafficPolicy: Cluster
  ipFamilies:
  - IPv4
  ipFamilyPolicy: SingleStack
  ports:
  - name: dns
    port: 53
    protocol: UDP
    targetPort: 53
  - name: dns-tcp
    port: 53
    protocol: TCP
    targetPort: 53
  - name: metrics
    port: 9153
    protocol: TCP
```



```
targetPort: 9153
selector:
  k8s-app: kube-dns
sessionAffinity: None
type: ClusterIP
status:
  loadBalancer: {}
```

#对应一个名称为coredns的deployment中两个副本的Pod

```
[root@master1 ~]#kubectl get deployments.apps -n kube-system coredns -o yaml
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  annotations:
```

```
    deployment.kubernetes.io/revision: "1"
```

```
  creationTimestamp: "2021-03-04T15:36:36Z"
```

```
  generation: 1
```

```
  labels:
```

```
    k8s-app: kube-dns
```

```
  name: coredns
```

```
  namespace: kube-system
```

```
  resourceVersion: "493653"
```

```
  uid: b215bd59-bf75-4693-9a9b-dfeadb28c4f
```

```
spec:
```

```
  progressDeadlineSeconds: 600
```

```
  replicas: 2 #两个副本Pod
```

```
  revisionHistoryLimit: 10
```

```
  selector:
```

```
    matchLabels:
```

```
      k8s-app: kube-dns
```

```
  strategy:
```

```
    rollingUpdate:
```

```
      maxSurge: 25%
```

```
      maxUnavailable: 1
```

```
    type: RollingUpdate
```

```
template:
```

```
  metadata:
```

```
    creationTimestamp: null
```

```
    labels:
```

```
      k8s-app: kube-dns
```

```
  spec:
```

```
    affinity:
```

```
      podAntiAffinity:
```

```
        preferredDuringSchedulingIgnoredDuringExecution:
```

```
          - podAffinityTerm:
```

```
            labelSelector:
```

```
              matchExpressions:
```

```
                - key: k8s-app
```

```
                  operator: In
```

```
                  values:
```

```
                    - kube-dns
```

```
            topologyKey: kubernetes.io/hostname
```

```
            weight: 100
```

```
    containers:
```

```
      - args:
```

```
        - -conf
```

```
        - /etc/coredns/Corefile
```

```
      image: registry.aliyuncs.com/google_containers/coredns:v1.9.3
```

```
imagePullPolicy: IfNotPresent
livenessProbe:
  failureThreshold: 5
  httpGet:
    path: /health
    port: 8080
    scheme: HTTP
  initialDelaySeconds: 60
  periodSeconds: 10
  successThreshold: 1
  timeoutSeconds: 5
name: coredns
ports:
- containerPort: 53
  name: dns
  protocol: UDP
- containerPort: 53
  name: dns-tcp
  protocol: TCP
- containerPort: 9153
  name: metrics
  protocol: TCP
readinessProbe:
  failureThreshold: 3
  httpGet:
    path: /ready
    port: 8181
    scheme: HTTP
  periodSeconds: 10
  successThreshold: 1
  timeoutSeconds: 1
resources:
  limits:
    memory: 170Mi
  requests:
    cpu: 100m
    memory: 70Mi
securityContext:
  allowPrivilegeEscalation: false
  capabilities:
    add:
    - NET_BIND_SERVICE
    drop:
    - all
  readOnlyRootFilesystem: true
terminationMessagePath: /dev/termination-log
terminationMessagePolicy: File
volumeMounts:
- mountPath: /etc/coredns
  name: config-volume
  readOnly: true
dnsPolicy: Default
nodeSelector:
  kubernetes.io/os: linux
priorityClassName: system-cluster-critical
restartPolicy: Always
schedulerName: default-scheduler
securityContext: {}
```

```

    serviceAccount: coredns
    serviceAccountName: coredns
    terminationGracePeriodSeconds: 30
    tolerations:
    - key: CriticalAddonsOnly
      operator: Exists
    - effect: NoSchedule
      key: node-role.kubernetes.io/control-plane
    volumes:
    - configMap:
        defaultMode: 420
        items:
        - key: Corefile
          path: Corefile
        name: coredns
      name: config-volume
status:
  availableReplicas: 2
  conditions:
  - lastTransitionTime: "2021-03-04T16:18:38Z"
    lastUpdateTime: "2021-03-04T16:18:39Z"
    message: ReplicaSet "coredns-5bbd96d687" has successfully progressed.
    reason: NewReplicaSetAvailable
    status: "True"
    type: Progressing
  - lastTransitionTime: "2021-04-01T23:39:30Z"
    lastUpdateTime: "2021-04-01T23:39:30Z"
    message: Deployment has minimum availability.
    reason: MinimumReplicasAvailable
    status: "True"
    type: Available
  observedGeneration: 1
  readyReplicas: 2
  replicas: 2
  updatedReplicas: 2

```

#查看SVC后面的Pod地址信息

```

[root@master1 ~]#kubectl get endpoints kube-dns -n kube-system
NAME           ENDPOINTS                                                                 AGE
kube-dns       10.244.1.78:53,10.244.1.79:53,10.244.1.78:53 + 3 more... 34d

```

```

[root@master1 ~]#kubectl describe ep -n kube-system kube-dns
Name:          kube-dns
Namespace:     kube-system
Labels:        k8s-app=kube-dns
               kubernetes.io/cluster-service=true
               kubernetes.io/name=CoreDNS
Annotations:   endpoints.kubernetes.io/last-change-trigger-time:
Subsets:
  Addresses:    10.244.1.78,10.244.1.79
  NotReadyAddresses: <none>
  Ports:

```

Name	Port	Protocol
dns-tcp	53	TCP
dns	53	UDP
metrics	9153	TCP

Events: <none>

```
[root@master1 ~]#kubectl get pod -n kube-system -l k8s-app=kube-dns
```

NAME	READY	STATUS	RESTARTS	AGE
coredns-5bbd96d687-8bpgx	1/1	Running	10 (86m ago)	34d
coredns-5bbd96d687-15k81	1/1	Running	11 (86m ago)	34d

```
[root@master1 ~]#kubectl get pod -o wide -A |grep coredns
```

kube-system		coredns-5bbd96d687-8bpgx			1/1
Running	10 (83m ago)	34d	10.244.1.79	node1.wang.org	<none>
		<none>			
kube-system		coredns-5bbd96d687-15k81			1/1
Running	11 (83m ago)	34d	10.244.1.78	node1.wang.org	<none>
		<none>			

#查看DNS名称

```
[root@master1 ~]#nslookup
```

```
> server 10.96.0.10
```

```
Default server: 10.96.0.10
```

```
Address: 10.96.0.10#53
```

```
> 10.96.0.10
```

```
10.0.96.10.in-addr.arpa name = kube-dns.kube-system.svc.cluster.local.
```

```
[root@node1 ~]#docker ps |grep coredns
```

```
[root@node1 ~]#docker inspect 5c445eaf80a4
```

```
.....
    {
        "Type": "bind",
        "Source": "/var/lib/kubelet/pods/50da8812-5573-4afb-9130-
e919bdb0fc35/volumes/kubernetes.io~configmap/config-volume",
        "Destination": "/etc/coredns",
        "Mode": "ro",
        "RW": false,
        "Propagation": "rprivate"
    }
.....
```

```
[root@node1 ~]#cat /var/lib/kubelet/pods/50da8812-5573-4afb-9130-
e919bdb0fc35/volumes/kubernetes.io~configmap/config-volume/Corefile.:53 {
```

```
  errors
  health {
    lameduck 5s
  }
  ready
  kubernet.es cluster.local in-addr.arpa ip6.arpa {
    pods insecure
    fallthrough in-addr.arpa ip6.arpa
    ttl 30
  }
  prometheus :9153
  forward . /etc/resolv.conf {
    max_concurrent 1000
  }
  cache 30
```

```

loop
reload
loadbalance
}

```

范例: 所有Pod指向DNS对应的svc 服务kube-dns的Cluster-IP地址

#查看集群默认的dns服务

```
[root@master1 ~]#kubectl get svc -n kube-system
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kube-dns	ClusterIP	192.168.0.10	<none>	53/UDP,53/TCP,9153/TCP	27d

```
[root@master1 ~]#kubectl describe svc kube-dns -n kube-system
```

.....

Endpoints: 172.16.1.58:53,172.16.3.109:53

```
[root@master1 ~]#kubectl get pod -o wide -l Kubernetes-app=kube-dns -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE	IP
coredns-7d89d9b6b8-6wzsl	1/1	Running	9 (43m ago)	22d	172.16.1.58
node2.wang.org	<none>	<none>			
coredns-7d89d9b6b8-kcltj	1/1	Running	2 (43m ago)	44h	172.16.3.109
node1.wang.org	<none>	<none>			

#这些dns的解析服务的地址是通过文件的方式注入到pod的环境中的

```
[root@master1 ~]#kubectl run pod-$RANDOM --image=registry.cn-
```

```
beijing.aliyuncs.com/wangxiaochun/admin-box:v0.1 -it --rm --command -- /bin/bash
If you don't see a command prompt, try pressing enter.
```

```
[root@pod-26908 /]$ cat /etc/resolv.conf
```

```
nameserver 192.168.0.10
```

```
search default.svc.cluster.local svc.cluster.local cluster.local magedu.com
```

```
options ndots:5
```

```
[root@pod-26908 /]$ cat /etc/hosts
```

```
# Kubernetes-managed hosts file.
```

```
127.0.0.1 localhost
```

```
::1 localhost ip6-localhost ip6-loopback
```

```
fe00::0 ip6-localnet
```

```
fe00::0 ip6-mcastprefix
```

```
fe00::1 ip6-allnodes
```

```
fe00::2 ip6-allrouters
```

```
172.16.2.96 pod-26908
```

```
[root@pod-26908 /]$ curl service-test
```

```
kubernetes pod-test v0.1!! ClientIP: 172.16.2.96, ServerName: myweb-5df49fdd5-z7d67, ServerIP: 172.16.2.95!
```

```
[root@pod-26908 /]$ curl service-test.default.svc.cluster.local
```

```
kubernetes pod-test v0.1!! ClientIP: 172.16.2.96, ServerName: myweb-5df49fdd5-z7d67, ServerIP: 172.16.2.95!
```

```
[root@pod-26908 /]$ curl service-test.default.svc.cluster.local.
```

```
kubernetes pod-test v0.1!! ClientIP: 172.16.2.96, ServerName: myweb-5df49fdd5-xnrt4, ServerIP: 172.16.3.108!
```

范例: 记录解析

#记录解析

```
[root@master1 ~]#kubectl run pod-$RANDOM --image=registry.cn-  
beijing.aliyuncs.com/wangxiaochun/admin-box:v0.1 -it --rm --command -- /bin/bash  
[root@pod-26908 /]$ nslookup  
> service-test.default.svc.cluster.local. #注意最后的.必须要添加  
Server:      192.168.0.10  
Address:     192.168.0.10#53
```

```
Name:   service-test.default.svc.cluster.local  
Address: 192.168.193.236  
> kube-dns.kube-system.svc.cluster.local. #正向解析service名称的结果为ClusterIP  
Server:      192.168.0.10  
Address:     192.168.0.10#53
```

```
Name:   kube-dns.kube-system.svc.cluster.local  
Address: 192.168.0.10
```

#反解析记录

```
> 172.16.2.95  
95.2.16.172.in-addr.arpa    name = 172-16-2-95.service-  
test.default.svc.cluster.local.  
> 172.16.1.59  
59.1.16.172.in-addr.arpa    name = 172-16-1-59.service-  
test.default.svc.cluster.local.
```

#测试解析

```
[root@master1 ~]#kubectl run pod-$RANDOM --image=registry.cn-  
beijing.aliyuncs.com/wangxiaochun/admin-box:v0.1 -it --rm --command -- /bin/bash  
root@pod-24105 /# host service-test  
service-test.default.svc.cluster.local has address 10.111.183.208  
root@pod-24105 /# host -t PTR 10.111.183.208  
208.183.111.10.in-addr.arpa domain name pointer service-  
test.default.svc.cluster.local.  
root@pod-24105 /# host -t A kube-dns  
Host kube-dns.wang.org not found: 2(SERVFAIL)  
root@pod-24105 /# host -t A kube-dns.kube-system  
kube-dns.kube-system.svc.cluster.local has address 10.96.0.10  
root@pod-24105 /# host -t SRV _http._tcp.service-test.default.svc.cluster.local  
_http._tcp.service-test.default.svc.cluster.local has SRV record 0 100 80  
service-test.default.svc.cluster.local.
```

```
root@pod-24105 /# dig -t SRV _http._tcp.myapp.default.svc.cluster.local.
```

```
; <<>> DiG 9.14.12 <<>> -t SRV _http._tcp.myapp.default.svc.cluster.local.  
;; global options: +cmd  
;; Got answer:  
;; WARNING: .local is reserved for Multicast DNS  
;; You are currently testing what happens when an mDNS query is leaked to DNS  
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 40397  
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 2  
;; WARNING: recursion requested but not available
```

```
;; OPT PSEUDOSECTION:  
; EDNS: version: 0, flags:; udp: 4096  
; COOKIE: 1cd2314ba79746b3 (echoed)  
;; QUESTION SECTION:  
;_http._tcp.myapp.default.svc.cluster.local. IN SRV
```

```
;; ANSWER SECTION:
```

```

_http._tcp.myapp.default.svc.cluster.local. 30 IN SRV 0 100 80
myapp.default.svc.cluster.local.

;; ADDITIONAL SECTION:
myapp.default.svc.cluster.local. 30 IN A 10.107.129.47

root@pod-24084 /# dig -x 10.104.87.84

; <<>> DiG 9.14.12 <<>> -x 10.104.87.84
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 49821
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: a709af85c2271fe3 (echoed)
;; QUESTION SECTION:
;84.87.104.10.in-addr.arpa. IN PTR

;; ANSWER SECTION:
84.87.104.10.in-addr.arpa. 30 IN PTR metrics-server.kube-
system.svc.cluster.local.

;; Query time: 10 msec
;; SERVER: 10.96.0.10#53(10.96.0.10)
;; WHEN: Fri Mar 28 02:12:21 UTC 2025
;; MSG SIZE rcvd: 149

root@pod-24084 /# host -t ptr 10.104.87.84
84.87.104.10.in-addr.arpa domain name pointer metrics-server.kube-
system.svc.cluster.local.
root@pod-24084 /# host -t ptr 84.87.104.10.in-addr.arpa
84.87.104.10.in-addr.arpa domain name pointer metrics-server.kube-
system.svc.cluster.local.

root@pod-24084 /# dig -x 10.104.87.84
.....
84.87.104.10.in-addr.arpa. 30 IN PTR metrics-server.kube-
system.svc.cluster.local.

root@pod-24084 /# dig -t ptr 84.87.104.10.in-addr.arpa
.....
84.87.104.10.in-addr.arpa. 30 IN PTR metrics-server.kube-
system.svc.cluster.local.

;; Query time: 6 msec
;; SERVER: 10.96.0.10#53(10.96.0.10)
;; WHEN: Fri Mar 28 02:13:01 UTC 2025
;; MSG SIZE rcvd: 149

```

2.3.2.3 Pod的 DNS 解析策略和配置

Kubernetes支持在单个Pod资源规范上自定义DNS解析策略和配置，并组合生效

- pod.spec.dnsPolicy: 解析策略
- pod.spec.dnsConfig: 名称解析机制

DNS解析策略: pod.spec.dnsPolicy

- Default: 从当前Pod运行的所在宿主机节点/etc/resolv.conf继承DNS名称解析相关的配置
- ClusterFirst: 此为默认值，优先使用K8s集群内DNS服务上解析集群域内的名称，其他域名的解析则交由coreDNS的Pod所在宿主机节点/etc/resolv.conf的名称服务器即使用Default策略（注意：和Default不同在于，使用是CoreDNS的Pod的所在宿主机节点的/etc/resolv.conf文件）
- ClusterFirstWithHostNet: 专用于在设置了hostNetwork（使用宿主机的网络）的Pod对象上并不会使用宿主机节点网络的DNS，仍然使用ClusterFirst策略
- None: 用于忽略Kubernetes集群的默认设定，而仅使用由dnsConfig自定义的配置

DNS解析机制: pod.spec.dnsConfig

- nameservers <[]string>: DNS名称服务器列表，附加于由dnsPolicy生成的DNS名称服务器之后
- searches <[]string>: DNS名称解析时的搜索域，附加由于dnsPolicy生成的搜索域之后
- options <[]Object>: DNS解析选项列表，同dnsPolicy生成的解析选项合并成最终生效的定义

范例: 查看Pod的 DNS 解析策略和配置

```
[root@master1 ~]#kubectl explain pods.spec
.....
dnsConfig    <Object>
  Specifies the DNS parameters of a pod. Parameters specified here will be
  merged to the generated DNS configuration based on DNSPolicy.

dnsPolicy    <string>
  Set DNS policy for the pod. Defaults to "ClusterFirst". Valid values are
  'ClusterFirstWithHostNet', 'ClusterFirst', 'Default' or 'None'. DNS
  parameters given in DNSConfig will be merged with the policy selected with
  DNSPolicy. To have DNS options set along with hostNetwork, you have to
  specify DNS policy explicitly to 'ClusterFirstWithHostNet'.

  Possible enum values:
  - "ClusterFirst" indicates that the pod should use cluster DNS first
  unless hostNetwork is true, if it is available, then fall back on the
  default (as determined by kubelet) DNS settings.
  - "ClusterFirstWithHostNet" indicates that the pod should use cluster DNS
  first, if it is available, then fall back on the default (as determined by
  kubelet) DNS settings.
  - "Default" indicates that the pod should use the default (as determined
  by kubelet) DNS settings.
  - "None" indicates that the pod should use empty DNS settings. DNS
  parameters such as nameservers and search paths should be defined via
  DNSConfig.
.....
```

范例: dnsPolicy 的ClusterFirst的解析策略

```
#安装集群外的DNS服务
[root@ubuntu2404 ~]#bash install_dns.sh
[root@ubuntu2404 ~]#vim /etc/bind/wang.org.zone
```



```
www      A      6.6.6.6
```

#指定节点使用批定的DNS地址

```
[root@node2 ~]#vim /etc/netplan/01-netcfg.yaml
```

```
network:
```

```
  version: 2
```

```
  renderer: networkd
```

```
  ethernet:
```

```
    eth0:
```

```
      addresses:
```

```
      - 10.0.0.103/24
```

```
      gateway4: 10.0.0.2
```

```
      nameservers:
```

```
        #addresses: [180.76.76.76, 223.6.6.6]
```

```
        addresses: [10.0.0.100] #指定DNS地址
```

```
[root@node2 ~]#netplan apply
```

#清单文件

```
[root@master1 service]#cat service-pod-with-dnspolicy-clusterfirst.yaml
```

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: service-pod-with-dnspolicy-clusterfirst
```

```
  namespace: default
```

```
spec:
```

```
  nodeName: node2.wang.org #指定面上的节点地址
```

```
  containers:
```

```
    - name: demo
```

```
      image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
```

```
      imagePullPolicy: IfNotPresent
```

```
    #dnsPolicy: ClusterFirst #使用默认的策略
```

```
[root@master1 service]#kubectl apply -f service-pod-with-dnspolicy-clusterfirst.yaml
```

#确认

```
[root@master1 service]#kubectl get pod service-pod-with-dnspolicy-clusterfirst -o yaml|grep dnsPolicy:
```

```
  dnsPolicy: ClusterFirst
```

#测试访问，发现并没有使用Pod所在节点的DNS配置，而使用是的CoreDNS所在节点的DNS服务

```
[root@master1 service]#kubectl exec -it service-pod-with-dnspolicy-clusterfirst -- sh
```

```
[root@service-pod-with-dnspolicy-clusterfirst /]# host www.wang.org
```

```
www.wang.org has address 205.178.189.129
```

范例：dnsPolicy 的 default 的解析策略

#安装集群外的DNS服务

```
[root@ubuntu2404 ~]#bash install_dns.sh
```

```
[root@ubuntu2404 ~]#vim /etc/bind/wang.org.zone
```

```
test      A      6.6.6.6
```

#修改指定节点的DNS配置指向上面的DNS服务器

```
[root@node2 ~]#cat /etc/netplan/01-netcfg.yaml
```

```
# This file describes the network interfaces available on your system
```

```
# For more information, see netplan(5).
```

```
network:
```

```

version: 2
renderer: networkd
ethernets:
  eth0:
    addresses:
      - 10.0.0.102/24
    gateway4: 10.0.0.2
    nameservers:
      addresses: [10.0.0.200]

[root@master1 service]#cat service-pod-with-dnspolicy-default.yaml
apiVersion: v1
kind: Pod
metadata:
  name: service-pod-with-dnspolicy-default
  namespace: default
spec:
  nodeName: node2.wang.org
  containers:
  - name: demo
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    imagePullPolicy: IfNotPresent
    dnsPolicy: Default #指定dns策略，默认为ClusterFirst

[root@master1 service]#kubectl apply -f service-pod-with-dnspolicy-default.yaml

#发现Pod的DNS指定Pod所有宿主节点的DNS
[root@master1 ~]#kubectl exec -it service-pod-with-dnspolicy-default -- cat
/etc/resolv.conf
nameserver 10.0.0.200
[root@master1 ~]#kubectl exec -it service-pod-with-dnspolicy-default -- host
test.wang.org
test.wang.org has address 6.6.6.6

[root@master1 service]#kubectl delete -f service-pod-with-dnspolicy-default.yaml

```

范例：dnsPolicy 的 None 的解析策略

```

[root@master1 ~]#cat service-pod-with-dnspolicy-none.yaml
apiVersion: v1
kind: Pod
metadata:
  name: service-pod-with-dnspolicy
  namespace: default
spec:
  containers:
  - name: demo
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    imagePullPolicy: IfNotPresent
  dnsPolicy: None
  dnsConfig:
    nameservers:
      - 10.96.0.10
      - 180.76.76.76
      - 223.6.6.6
    searches:

```

```
- svc.cluster.local
- cluster.local
- wang.org
options:
- name: ndots
  value: "5"    #意味着如果域名中只有5个或更少的点，则系统会尝试在其末尾添加搜索域。
```

#应用

```
[root@master1 ~]#kubectl apply -f service-pod-with-dnspolicy-none.yaml
```

```
[root@master1 ~]#kubectl get pod service-pod-with-dnspolicy
NAME                                READY   STATUS    RESTARTS   AGE
service-pod-with-dnspolicy         1/1     Running   0           78s
```

#查看Pod的DNS配置

```
[root@master1 ~]#kubectl exec service-pod-with-dnspolicy -- cat /etc/resolv.conf
nameserver 10.96.0.10
nameserver 180.76.76.76
nameserver 223.6.6.6
search svc.cluster.local cluster.local wang.org
options ndots:5 #意味着如果域名中只有5个或更少的点，则系统会尝试在其末尾添加搜索域。
```

#清理环境

```
[root@master1 ~]#kubectl delete -f service-pod-with-dnspolicy-none.yaml
```

2.3.3 CoreDNS 配置

CoreDNS的配置都存储在名为coredns的ConfigMap对象中，该对象位于kube-system名称空间中

服务器配置段(Server Blocks)，用于定义负责解析的权威区域，配置段放置于其后的花括号{}中

服务器配置段也可以指定要监听的端口号，端口号之前需要使用一个冒号，默认为53

若在同一端口上配置了多个区域，CoreDNS将使用最长匹配机制来评估并挑出最适配的服务器配置段；

随后，请求将由该服务器配置段中的插件链按特定顺序（在构建CoreDNS程序时由plugin.cfg文件指定）进行路由；

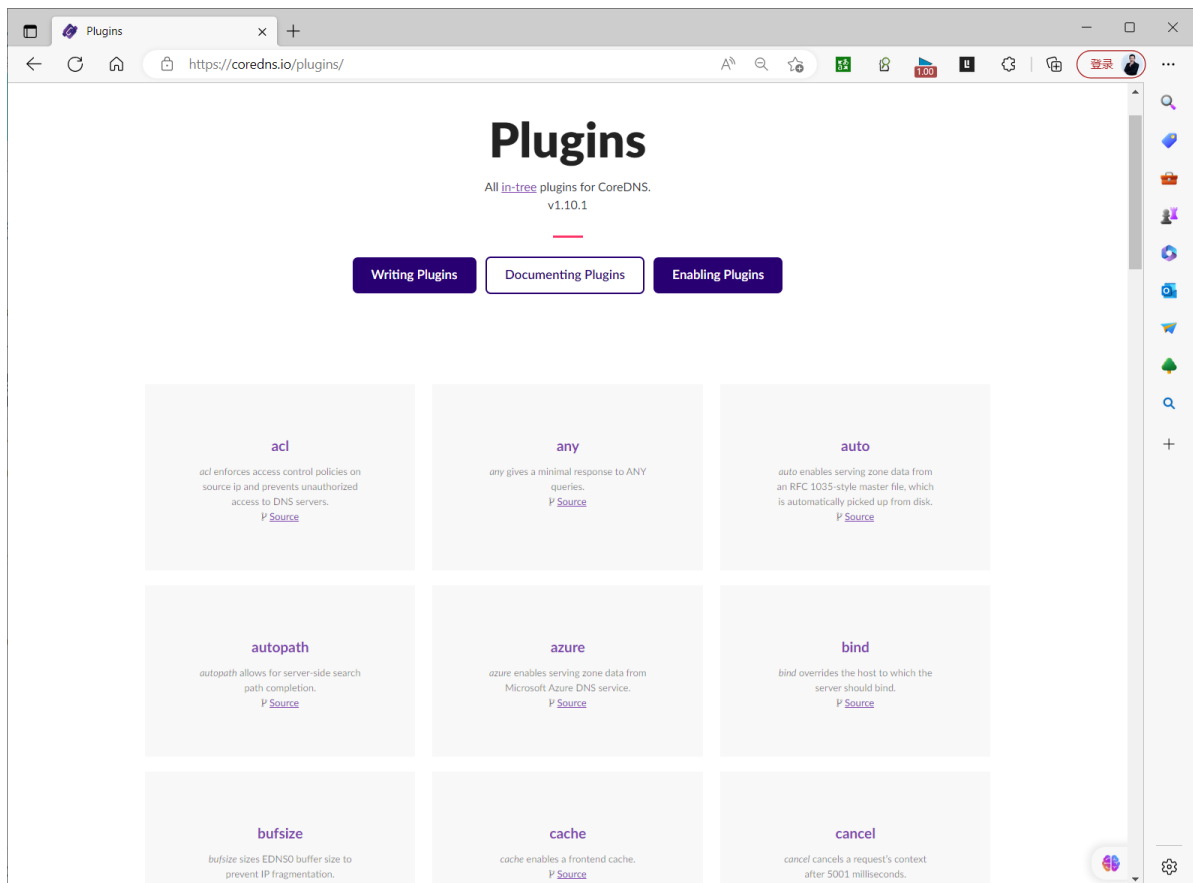
针对一个DNS查询，相应配置段插件链中的每个插件都会检查并确定是否应该对其进行处理

CoreDNS查询路由概述CoreDNS处理DNS查询的方式

- 针对每个套接字，CoreDNS会生成一个服务器(dnsserver.Server)
- 一个套接字上存在多个配置段(Server Block)时，CoreDNS将组合这些配置段生成单个服务器上的多个独立配置段，并将发往该套接字的DNS查询请求路由到最佳匹配的目标配置段
- 若不存在匹配的服务器配置段，则返回SERVERFAIL

CoreDNS的支持丰富的插件

```
https://coredns.io/plugins
```



CoreDNS的插件可大致分为两类

- 负责处理请求的常规插件(Normal Plugin)，这些插件才是插件链中的有效组成部分，例如:errors、kubernetes和forward等
- 不处理请求的特殊插件，它们仅用来修改Server或Server Block中的配置，例如:health、tls、startup、shutdown和root等插件，这类插件不会作为服务器配置段的插件链中的有效组成部署

常规插件可再划分为两种类型

- 负责以某种方式处理请求的插件
这类插件不是区域数据源，它们的运行方式是在自身处理完成后，会将查询传递给下一个插件
- 后端插件
用于配置区域数据的来源，例如etcd、file和kubernetes等
这类插件通常要为其负责的区域生成最终结果，它要么能正常响应查询，要么返回NXDOMAIN
但也可以使用fallthrough改变这种响应行为:未发现查询的目标资源记录时，将DNS查询请求转交给插件链中的下一个插件，随后终结于另一个后端插件，或再次由fallthrough改变这种行为

配置解析

```
#coredns的配置是存放在 configmap中
[root@master1 ~]#kubectl get cm -n kube-system
NAME          DATA  AGE
coredns       1      27d
.....

#查看配置内容
[root@master1 ~]#kubectl get cm coredns -n kube-system -o yaml
apiVersion: v1
data:
  Corefile: |
    .:53 {
```

```

    errors
    health {
        lameduck 5s
    }
    ready
    kubernetes cluster.local in-addr.arpa ip6.arpa {
        pods insecure
        fallthrough in-addr.arpa ip6.arpa
        ttl 30
    }
    prometheus :9153
    forward . /etc/resolv.conf {
        max_concurrent 1000
    }
    cache 30
    loop
    reload
    loadbalance
}
kind: ConfigMap
metadata:
  creationTimestamp: "2021-03-04T15:36:36Z"
  name: coredns
  namespace: kube-system
  resourceVersion: "219"
  uid: d6307d21-8c84-4302-9b95-a8058188333e

```

[root@master1 ~]# kubectl describe cm coredns -n kube-system

...

Data

====

Corefile:

```

.:53 {                                     #包括根区域的所有区域对应的监听端口进行解析
    errors                                 #将错误信息进行输出
    health {                              #livenessProbe检测,http://localhost:8080/health 实现
        lameduck 5s
    }
    ready                                 #readinessProbe检测,http://localhost:8181/ready coredns就绪返回200
    kubernetes cluster.local in-addr.arpa ip6.arpa { #基于kubernetes的service名称进行查询并返回查询结果
        pods insecure
        fallthrough in-addr.arpa ip6.arpa #如果in-addr.arpa ip6.arpa区域解析失败,交由后续的插件进行解析
        ttl 30
    }
    prometheus :9153                       #配置访问端口给prometheus实现监控,URL:
http://10.96.0.10:9153/metrics
    forward . /etc/resolv.conf {          #forward 转发配置,如果集群内部无法解析,交由coreDNS的Pod所在宿主机的DNS解析,也可IP地址
        max_concurrent 1000              #最大连接性,提高此值可以提高并发性
    }
    cache 30                              #启用缓存,单位s
    loop                                  #检测发现环路时会重建coredns对应的Pod显示CrashLoopBackOff状态而停止查询,比如CoreDNS直接将请求转发给上游服务器,后者再将请求转发回CoreDNS

```

```

    reload                                # 检测corefile是否变化,修改
configmap后默认2M后会自动加载
    loadbalance                            # 基于随机算法实现DNS查询记录负载均衡
}
...

#对于企业内的dns解决方案,可以通过 forward 来实现,格式如下
    forward <域名> <转发至外部DNS的地址> {    #转发配置, 如果集群内部无法解析,交由宿主机文件
或外部DNS的IP解析
        max_concurrent 最大连接配置
        except 排除的域名
    }
#示例: 转发域名解析至集群外的DNS服务器
forward . 10.0.0.10 10.0.0.20 {
    prefer_udp    #优先使用UDP
}
forward wang.org 10.0.0.200 {
    prefer_udp    #优先使用UDP, 多个forward谁在前谁优先生效
}

#注意: 如果仅仅对某个域名进行转发的话,只需要将 <域名> 部分设置为指定的域名即可。
#生产中不推荐直接将 "." 的转发地址使用公网的dns地址,推荐在当前主机的/etc/resolv.conf中配置
外网,实现间接的效果

# 添加特定主机的正向解析记录,类似于/etc/hosts文件功能,此优先级比上面的forward高
    hosts {
        192.168.10.100 www.example.com
        10.0.0.101  gitlab.wang.org nfs.wang.org
        10.0.0.102  jenkins.wang.org
        10.0.0.100  harbor.wang.org
        fallthrough
    }

```

范例: 配置CoreDNS的域名解析转发至指定的DNS Server

```

#创建集群外的DNS
[root@ubuntu2404 ~]#bash install_dns.sh
[root@ubuntu2404 ~]#cat /etc/bind/wang.org.zone
$TTL 1D
@   IN SOA  master admin (
                        1   ; serial
                        1D   ; refresh
                        1H   ; retry
                        1w   ; expire
                        3H )   ; minimum
      NS      master
master  A      10.0.0.200
www     A      10.0.0.200
harbor  A      10.0.0.100
test    A      6.6.6.6
a       A      1.1.1.1
b       A      2.2.2.2

[root@ubuntu2404 ~]#rndc reload

```

```
[root@master1 ~]#kubectl edit -n kube-system cm coredns
apiVersion: v1
data:
  Corefile: |
    .:53 {
      errors
      health {
        lameduck 5s
      }
      ready
      kubernetes cluster.local in-addr.arpa ip6.arpa {
        pods insecure
        fallthrough in-addr.arpa ip6.arpa
        ttl 30
      }
      prometheus :9153
      forward wang.org 10.0.0.200 { #添加三行
        prefer_udp
      }
      forward . /etc/resolv.conf { #两个forward谁在前谁优先生效
        max_concurrent 1000
      }

      cache 30 {
        disable success cluster.local
        disable denial cluster.local
      }
      loop
      reload
      loadbalance
    }
  }
```

#重新创建coredns的Pod生效

```
[root@master1 ~]#kubectl rollout restart deployment -n kube-system coredns
deployment.apps/coredns restarted
```

```
[root@master1 service]#kubectl run client-test-$RANDOM --image registry.cn-
beijing.aliyuncs.com/wangxiaochun/admin-box:v0.1 --restart=Never --rm -it --
command -- /bin/bash
```

If you don't see a command prompt, try pressing enter.

```
root@client-test-908 /# host a.wang.org
```

```
a.wang.org has address 1.1.1.1
```

```
Host a.wang.org not found: 3(NXDOMAIN)
```

```
root@client-test-908 /# host b.wang.org
```

```
b.wang.org has address 2.2.2.2
```

```
Host b.wang.org not found: 3(NXDOMAIN)
```

范例: 不使用默认的转发策略, 使用自定义的转发策略

#修改配置文件

```
root@master1:~# kubectl edit cm coredns -n kube-system
```

```
...
```

```
      rewrite name myapp.wang.org myapp.default.svc.cluster.local #将集群外的域名
重写为集群内的域名再进行解析
```

```
      kubernetes cluster.local in-addr.arpa ip6.arpa
```

```
      forward . /etc/resolv.conf {
```

```

max_concurrent 1000
except www.baidu.com. #排除www.baidu.com, 不进行解析,
注意: 最后面的点
}
forward wang.org 10.0.0.200 { #添加三行
    prefer_udp
}
hosts { #添加三行, 实现指定域名的解析,此优先级比上面的forward高
    10.0.0.100 harbor.cluster.local harbor.wang.org
    10.0.0.101 nfs.wang.org
    fallthrough
}
...
#注意: 多个dns地址间用空格隔开,排除的域名最好在末尾添加“.”,对于之前的旧版本来说可能会出现无法查询的现象

#重建CoreDNS, 加快DNS配置信息生效
#方法1: 重新创建coredns的Pod生效
[root@master1 ~]#kubectl rollout restart deployment -n kube-system coredns
deployment.apps/coredns restarted
#方法2:
#新版
[root@master1 ~]#kubectl delete pod -l k8s-app=kube-dns -n kube-system
#旧版
[root@master1 ~]#kubectl delete pod -l kubernetes-app=kube-dns -n kube-system

#pod中测试效果
[root@master1 ~]#kubectl run pod-$RANDOM --image=registry.cn-
beijing.aliyuncs.com/wangxiaochun/admin-box:v0.1 -it --rm --command -- /bin/bash
If you don't see a command prompt, try pressing enter.
[root@pod-16460 /]$ host myapp.wang.org
myapp.wang.org has address 10.111.183.208
[root@pod-16460 /]$ host www.baidu.com
Host www.baidu.com not found: 3(NXDOMAIN)
[root@pod-16460 /]ping -c 1 harbor.wang.org
PING harbor.wang.org (harbor.cluster.local): 56 data bytes
64 bytes from harbor.cluster.local: seq=0 ttl=63 time=1.085 ms

```

3 Headless Service

3.1 无头服务机制

用户访问service资源对象的流程如下



- kube-proxy其实是将用户访问的域名解析给了service对象所对应的ClusterIP
- 然后再由ClusterIP根据Label 选择器将请求转发给后端的pod
- 然后在交给pod内部的容器

Client --> ServiceName --> ClusterIP --> 所有Pod的IP地址,这样一个请求经过了两层转发才到容器里面的服务,流程有些繁琐。

Service中 ClusterIP、NodePort和LoadBalancer 三种类型都为其Service配置一个ClusterIP

在CoreDNS上将这些Service对象的A记录最终解析为对应的ClusterIP,此ClusterIP 称为头Head

目前集群的资源调度主要是针对的是无状态的服务，但是对于一些特定场景的状态服务，虽然可以基于会话粘滞来实现，但是毕竟需要经过两层跳转，如果转发量太大，性能不太好。

可以使用如下方式可以减少中间的流程

Client --> ServiceName --> 所有Pod的IP地址

在有状态的Pod都有专有的名称,那么可以无需经过ClusterIP，直接利用Pod名称解析到Pod_ip即可，那么和普通Service相比，这种没有配置ClusterIP项的Service，称之为Headless Service。



无头服务场景下，Kubernetes会将一个集群内部的所有Pod成员提供唯一的DNS域名来作为每个成员的网络标识，集群内部成员之间使用域名通信，这个时候，就特别依赖service的selector属性配置了。

广义上Headless Service，它们又可以为分两种情形

- 有标签选择器，或者没有标签选择器,但有着与Service对象同名的Endpoint资源
Service的DNS名称直接解析为后端各就绪状态的Pod的IP地址
调度功能也将由DNS完成
各Pod IP相关PTR记录将解析至Pod名称，假设Pod IP为a.b.c.d，则其Pod名称为a-b-c-d...SVC.
这种类型也就是狭义上的Headless Service
主要应用于有状态服务的statefulSet资源对象
- 无标签选择器且也没有与Service对象同名的Endpoint资源
用于集群外部 ExternalName 类型的Service
Service的DNS名称将会生成一条CNAME记录，对应值由Service对象上的spec.externalName字段指定

注意: headless service是一个四层调度，因为iptables/ipvs都是四层的，所以如果要建立一个https服务的话，每一个服务都必须配置一个https的主机，因为四层调度是无法卸载https会话的

主要的应用场景

- ServiceName --> (label Selector, Pod) --> 所有Pod的IP地址，此方式又称为狭义的Headless Service，主要应用在 StatefulSet
- ServiceName --> CName (ExternalServiceName) --> ExternalService IP，此方式称为狭义的 External Service

无头服务管理的域名是如下的格式：

```
$(service_name).$(Kubernetes_namespace).svc.cluster.local
```

DNS 解析记录

#A记录

<a>--<c>-<d>.<service>.<ns>.svc.<zone> A PodIP

#PodIP的PTR反解析记录

<d>.<c>..<a>.in-addr.arpa IN PTR <用横线分隔的PodIP>.<service>.<ns>.svc.<zone>

#关键点:

正向解析:svc_name的解析结果从常规Service的ClusterIP, 转为解析成各个Pod的IP地址

反向解析:从常规的clusterip解析为service name, 转为从podip到hostname, <a>--<c>-<d>.

<service>.<ns>.svc.<zone>

<hostname>指的是a-b-c-d格式, 而非Pod自己的主机名;

3.2 案例

范例: Headless Service

#命令行方式

```
[root@master1 ~]#kubectl create service clusterip service-headless-cmd --clusterip="None"
```

#创建文件

```
[root@master1 ~]#cat service-headless.yaml
```

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: service-headless
```

```
spec:
```

```
  selector:
```

```
    app: myweb
```

```
  clusterIP: "None" #无头服务
```

#应用文件

```
[root@master1 ~]#kubectl apply -f service-headless.yaml
```

#查看两个service, Cluster-IP显示为None,注意和ExternalName Service的<none>不同

```
[root@master1 ~]#kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	192.168.0.1	<none>	443/TCP	46h
service-headless	ClusterIP	None	<none>	<none>	79s
service-headless-cmd	ClusterIP	None	<none>	<none>	2s

```
[root@master1 ~]#kubectl describe service service-headless
```

```
Name: service-headless
```

```
Namespace: default
```

```
Labels: <none>
```

```
Annotations: <none>
```

```
Selector: app=myweb
```

```
Type: ClusterIP
```

```
IP Family Policy: SingleStack
```

```
IP Families: IPv4
```

```
IP: None #注意: 没有ClusterIP
```

```
IPs: None
```

```
Session Affinity: None
```

```
Events: <none>
```

#注意：因为dns的原因，这里的无头服务可能无法显示对应的subnets相关的信息，但是不影响无头服务的正常使用

#重新创建deployment

```
[root@master1 ~]#kubectl delete deployment myweb
[root@master1 ~]#kubectl create deployment myweb --image=registry.cn-
beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1 --replicas=3
```

```
[root@master1 ~]#kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
myweb-5df49fdd5-fwc82	1/1	Running	0	63s	172.16.2.98	
node3.wang.org	<none>	<none>				
myweb-5df49fdd5-rfsth	1/1	Running	0	63s	172.16.3.111	
node1.wang.org	<none>	<none>				
myweb-5df49fdd5-wh86t	1/1	Running	0	63s	172.16.1.61	
node2.wang.org	<none>	<none>				

#进入新pod中测试效果

```
[root@master1 ~]#kubectl run pod-$RANDOM --image=registry.cn-
beijing.aliyuncs.com/wangxiaochun/admin-box:v0.1 -it --rm --command -- /bin/bash
root@pod-8431 /# host service-headless
```

#无头服务的正向解析结果直接为后端Pod的地址，不再解析为service的IP，导致Service的负载均衡功能失效，但可以利用DNS实现

```
service-headless.default.svc.cluster.local has address 10.244.2.55
service-headless.default.svc.cluster.local has address 10.244.1.83
service-headless.default.svc.cluster.local has address 10.244.1.85
```

```
root@pod-8431 /# host -t A service-headless
service-headless.default.svc.cluster.local has address 10.244.2.55
service-headless.default.svc.cluster.local has address 10.244.1.85
service-headless.default.svc.cluster.local has address 10.244.1.83
```

```
root@pod-8431 /# host -t PTR 10.244.1.85
85.1.244.10.in-addr.arpa domain name pointer 10-244-1-85.service-
headless.default.svc.cluster.local.
```

```
root@pod-8431 /# host 10-244-1-85.service-headless.default.svc.cluster.local.
10-244-1-85.service-headless.default.svc.cluster.local has address 10.244.1.85
```

```
root@pod-8431 /# curl service-headless
kubernetes pod-test v0.1!! ClientIP: 10.244.3.20, ServerName: myweb-5d78b4dcdbd-
9kfth, ServerIP: 10.244.1.83!
root@pod-8431 /# curl service-headless
kubernetes pod-test v0.1!! ClientIP: 10.244.3.20, ServerName: myweb-5d78b4dcdbd-
96r94, ServerIP: 10.244.1.85!
root@pod-8431 /# curl service-headless
kubernetes pod-test v0.1!! ClientIP: 10.244.3.20, ServerName: myweb-5d78b4dcdbd-
9kfth, ServerIP: 10.244.1.83!
root@pod-8431 /# curl service-headless
kubernetes pod-test v0.1!! ClientIP: 10.244.3.20, ServerName: myweb-5d78b4dcdbd-
6rgv4, ServerIP: 10.244.2.55!
```

```
root@pod-8431 /# nslookup
```

#正向解析

```
> service-headless
Server:      10.96.0.10
```

```

Address: 10.96.0.10#53

Name: service-headless.default.svc.cluster.local
Address: 10.244.1.83
Name: service-headless.default.svc.cluster.local
Address: 10.244.1.85
Name: service-headless.default.svc.cluster.local
Address: 10.244.2.55

> service-headless.default.svc.cluster.local #注意最后的面没有一个点.，结果解析错误
Server: 10.96.0.10
Address: 10.96.0.10#53

Non-authoritative answer:
Name: service-headless.default.svc.cluster.local.wang.org
Address: 205.178.189.129
> service-headless.default.svc.cluster.local.
Server: 10.96.0.10
Address: 10.96.0.10#53

Name: service-headless.default.svc.cluster.local
Address: 10.244.1.83
Name: service-headless.default.svc.cluster.local
Address: 10.244.2.55
Name: service-headless.default.svc.cluster.local
Address: 10.244.1.85

> service-headless.default.svc.cluster.local. #注意最后的面有一个点.，结果解析错误
Server: 10.96.0.10
Address: 10.96.0.10#53

Name: service-headless.default.svc.cluster.local
Address: 10.244.2.55
Name: service-headless.default.svc.cluster.local
Address: 10.244.1.85
Name: service-headless.default.svc.cluster.local
Address: 10.244.1.83

#反向解析
> 10.244.2.55
55.2.244.10.in-addr.arpa name = 10-244-2-55.service-headless.default.svc.cluster.local.

#结果显示：由于没有对域名做定向解析，那么找请求的时候，就像无头苍蝇似的，到处乱串
#注意：也可以为 nslookup 添加 -query=A 方式

#清理环境
[root@master1 ~]#kubectl delete -f service-headless.yaml

```

范例：通过ExternalName 和无头服务结合CoreDNS及外部DNS实现集群内访问集群外的服务

```

#集群外安装Redis
[root@ubuntu2404 ~]#apt install redis
[root@ubuntu2404 ~]#vi /etc/redis/redis.conf
bind 0.0.0.0
requirepass 123456
[root@ubuntu2404 ~]#systemctl restart redis

```

```
[root@master1 service]#cat service-externalname-redis.yaml
```

```
kind: Service
apiVersion: v1
metadata:
  name: svc-externalname-redis
  namespace: default
spec:
  type: ExternalName
  externalName: redis.wang.org
  ports:
  - protocol: TCP
    port: 6379
    targetPort: 6379
    nodePort: 0
  selector: {}
```

```
[root@master1 service]#kubectl apply -f service-externalname-redis.yaml
```

```
[root@master1 service]#kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
svc-externalname-redis	ExternalName	<none>	redis.wang.org	6379/TCP 10m

#配置coreDNS

```
[root@master1 ~]#kubectl edit cm -n kube-system coredns
```

```
o o o o o
```

```
prometheus :9153
```

#方法1: 添加外部的域名解析转发至外部DNS, 需要提前集群外创建DNS服务和域名解析

```
forward wang.org 10.0.0.200 {
  prefer_udp
}
```

```
forward . /etc/resolv.conf {
  max_concurrent 1000
}
```

#方法2: 或者在coreDNS中直接解析外部域名

```
hosts {
  10.0.0.200 redis.wang.org
  fallthrough
}
```

```
[root@master1 ~]#kubectl rollout restart deployment -n kube-system coredns
```

#测试访问

```
[root@master1 ~]#kubectl run client-test-$RANDOM --image registry.cn-
beijing.aliyuncs.com/wangxiaochun/admin-box:v0.1 --restart=Never --rm -it --
command -- /bin/bash
```

```
root@client-test-29726 /# host redis.wang.org
```

```
redis.wang.org has address 10.0.0.200
```

```
root@client-test-29726 /# nc redis.wang.org 6379
```

```
auth 123456
```

```
+OK
```

```
info
```

